

Vivekanand Education Society's Institute of Technology

An Autonomous Institute Affiliated to University of Mumbai
Hashu Advani Memorial Complex, Collector Colony, Chembur East, Mumbai - 400074.



Department of Information Technology

CERTIFICATE

This is to certify that **Niraj Kothawade** of **D15A** semester **VI**, have successfully completed necessary experiments in the **MAD & PWA Lab** under my supervision in **VES Institute of Technology** during the academic year **2024-2025**.

Lab Assistant

Subject Teacher

Mrs. Kajal Joseph

Principal

Head of Department

Dr. Mrs. Shalu Chopra

Name of the Course : MAD & PWA Lab

Course Code : ITL604

Year/Sem/Class : D15A/D15B**A.Y.:** 24-25**Faculty Incharge** : Mrs. Kajal Joseph.**Lab Teachers** : Mrs. Kajal Joseph.**Email** : kajal.jewani@ves.ac.in**Programme Outcomes:** The graduate will be able to:

PO1) Basic Engineering knowledge: An ability to apply the fundamental knowledge in mathematics, science and engineering to solve problems in Computer engineering.

PO2) Problem Analysis: Identify, formulate, research literature and analyze computer engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences and computer engineering and sciences.

PO3) Design/ Development of Solutions: Design solutions for complex computer engineering problems and design system components or processes that meet specified needs with appropriate consideration for public health and safety, cultural, societal and environmental considerations.

PO4) Conduct investigations of complex engineering problems using research-based knowledge and research methods including design of experiments, analysis and interpretation of data and synthesis of information to provide valid conclusions.

PO5) Modern Tool Usage: Create, select and apply appropriate techniques, resources and modern computer engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

PO6) The Engineer and Society: Apply reasoning informed by contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to computer engineering practice.

PO7) Environment and Sustainability: Understand the impact of professional computer engineering solutions in societal and environmental contexts and demonstrate knowledge of and need for sustainable development.

PO8) Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of computer engineering practice.

PO9) Individual and Team Work: Function effectively as an individual, and as a member or leader in diverse teams and in multidisciplinary settings.

PO10) Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations and give and receive clear instructions.

PO11) Project Management and Finance: Demonstrate knowledge and understanding of computer engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO12) Life-long Learning: Recognize the need for and have the preparation and ability to engage in independent and lifelong learning in the broadest context of technological change.

Program specific Outcomes

PSO1) An ability to manage and analyze data / information effectively for making better decisions.

PSO2) Demonstrate the ability to use state of the art technologies and tools including Free and Open Source Software (FOSS) tools in developing software.

Lab Objectives:

Sr. No.	Lab Objectives
The Lab experiments aims:	
1	Learn the basics of the Flutter framework.
2	Develop the App UI by incorporating widgets, layouts, gestures and animation
3	Create a production ready Flutter App by including files and firebase backend service.
4	Learn the Essential technologies, and Concepts of PWAs to get started as quickly and efficiently as possible
5	Develop responsive web applications by combining AJAX development techniques with the jQuery JavaScript library.
6	Understand how service workers operate and also learn to Test and Deploy PWA.

Lab Outcomes:

Sr. No.	Lab Outcomes	Cognitive levels of attainment as per Bloom's Taxonomy
On Completion of the course the learner/student should be able to:		
1	Understand cross platform mobile application development using Flutter framework	L1, L2
2	Design and Develop interactive Flutter App by using widgets, layouts, gestures and animation	L3
3	Analyze and Build production ready Flutter App by incorporating backend services and deploying on Android / iOS	L3, L4
4	Understand various PWA frameworks and their requirements	L1, L2
5	Design and Develop a responsive User Interface by applying PWA Design techniques	L3
6	Develop and Analyse PWA Features and deploy it over app hosting solutions	L3, L4

Index

Sr. No	Experiment Title	LO	DOP	DOS	Grade
1.	To install and configure the Flutter Environment	LO1			
2.	To design Flutter UI by including common widgets.	LO2			
3.	To include icons, images, fonts in Flutter app	LO2			
4.	To create an interactive Form using form widget	LO2			
5.	To apply navigation, routing and gestures in Flutter App	LO2			
6.	To Connect Flutter UI with fireBase database	LO3			
7.	To write meta data of your Ecommerce PWA in a Web app manifest file to enable “add to homescreen feature”.	LO4			
8.	To code and register a service worker, and complete the install and activation process for a new service worker for the E-commerce PWA	LO5			
9.	To implement Service worker events like fetch, sync and push for E-commerce PWA	LO5			
10.	To study and implement deployment of Ecommerce PWA to GitHub Pages.	LO5			
11.	To use google Lighthouse PWA Analysis Tool to test the PWA functioning.	LO6			
12.	Assignment-1	LO1,LO2 ,LO3			
13.	Assignment-2	LO4,LO5 ,LO6			

MAD & PWA Lab

Journal

Experiment No.	01
Experiment Title.	To install and configure the Flutter Environment
Roll No.	24
Name	Niraj Kothawade
Class	D15A/D15B
Subject	MAD & PWA Lab
Lab Outcome	LO1: Understand cross platform mobile application development using Flutter framework
Grade:	

EXPERIMENT NO: - 01

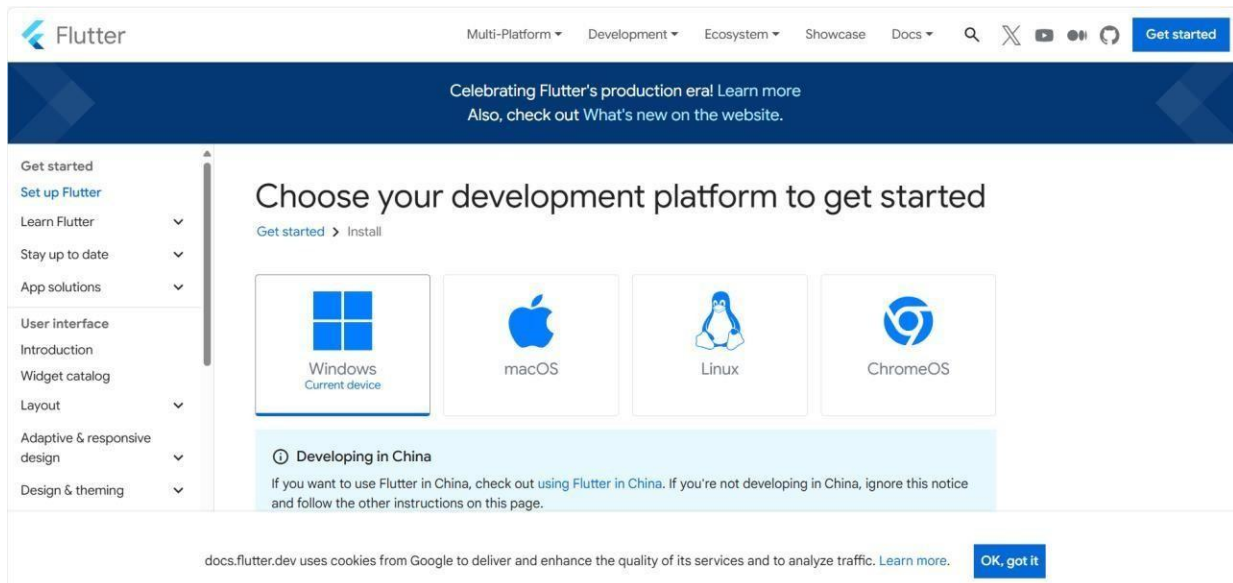
Name:- Niraj Kothawade

Class:- D15A

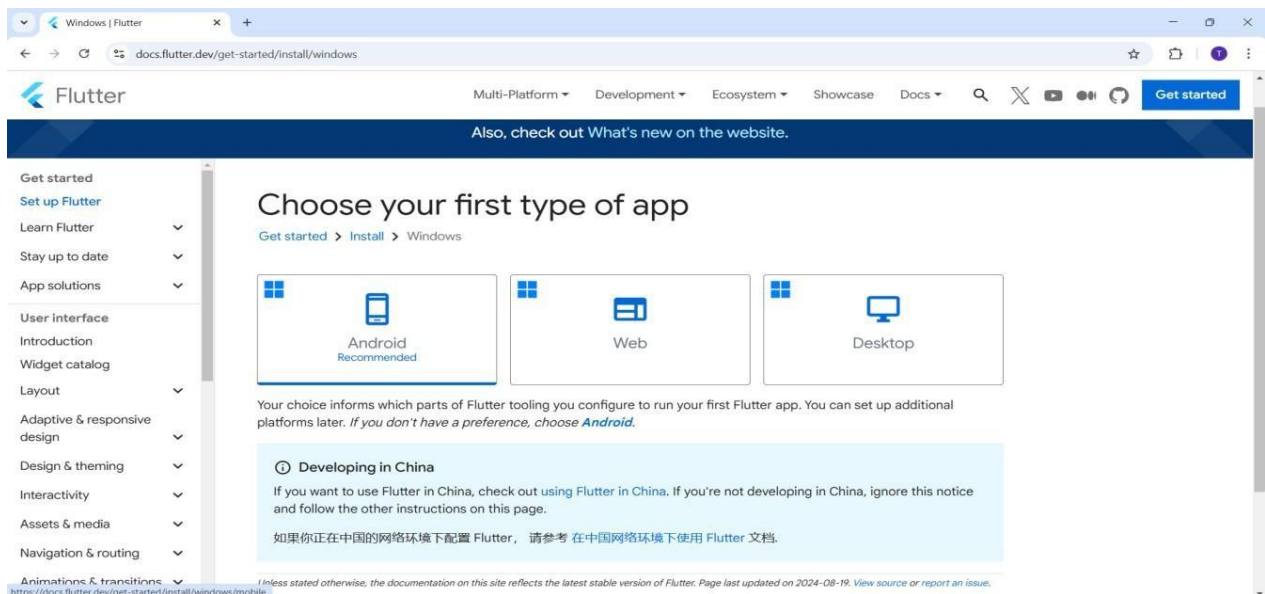
Roll:No:- 24

AIM:- Installation and Configuration of Flutter Environment.

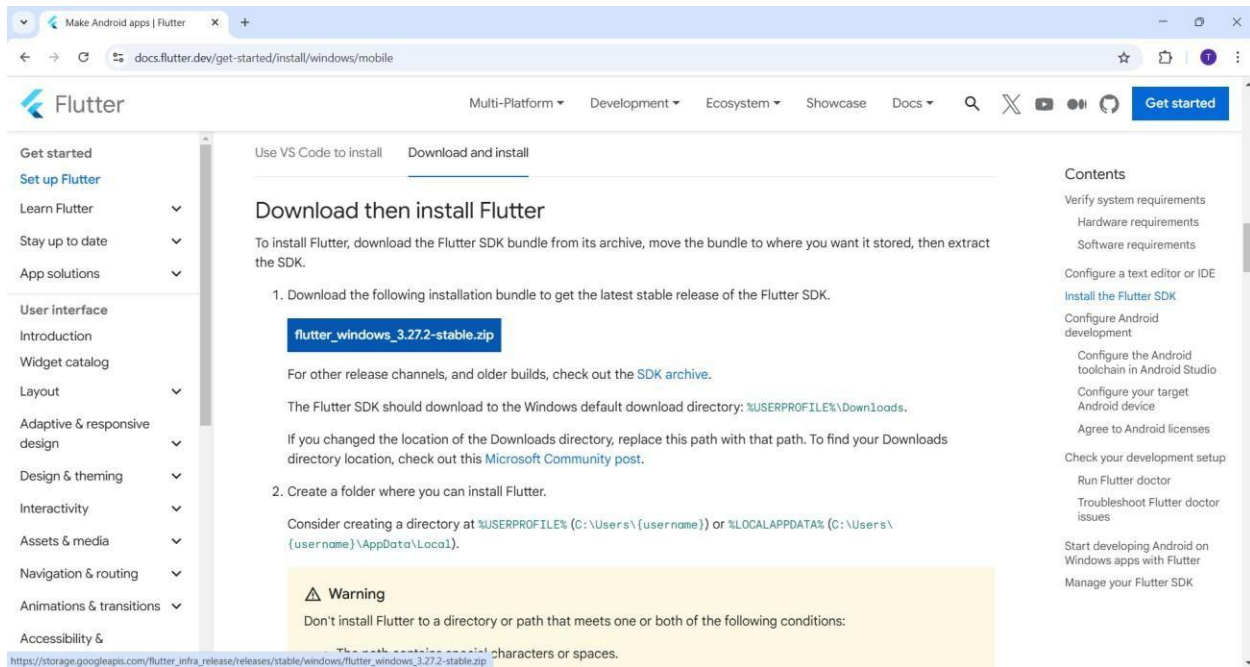
Step 1: Go to the official Flutter website: <https://docs.flutter.dev/get-started/install>



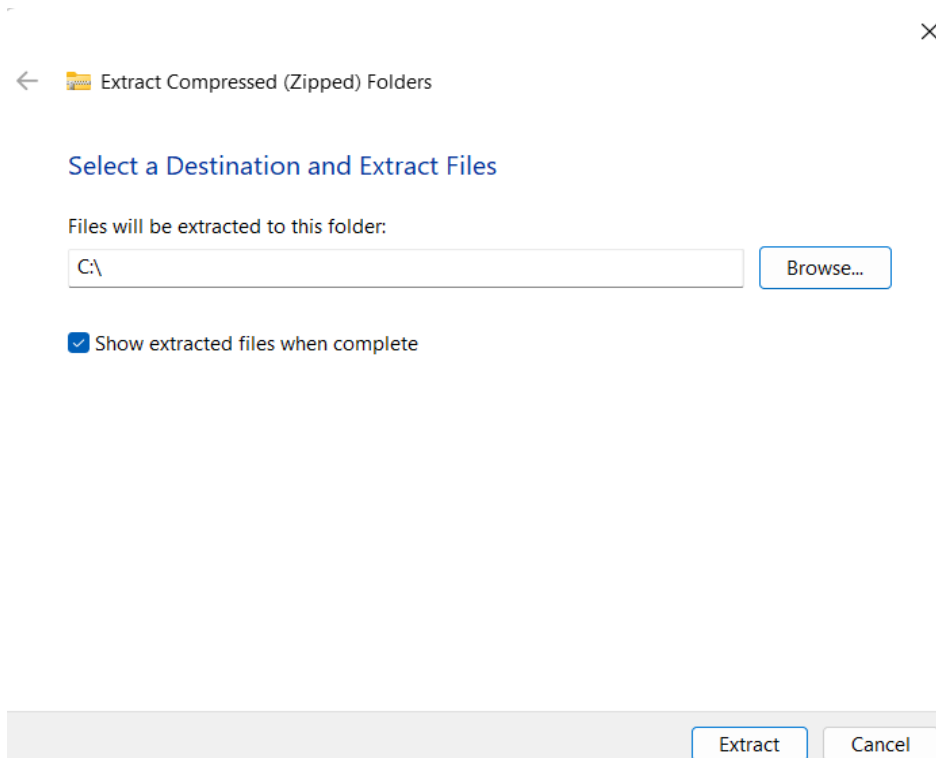
Step 2: To download the latest Flutter SDK, click on the Windows icon > Android



Step 3: For Windows, download the stable release (a .zip file).



Step 4: Extract the ZIP file to a folder (e.g., C:\flutter).

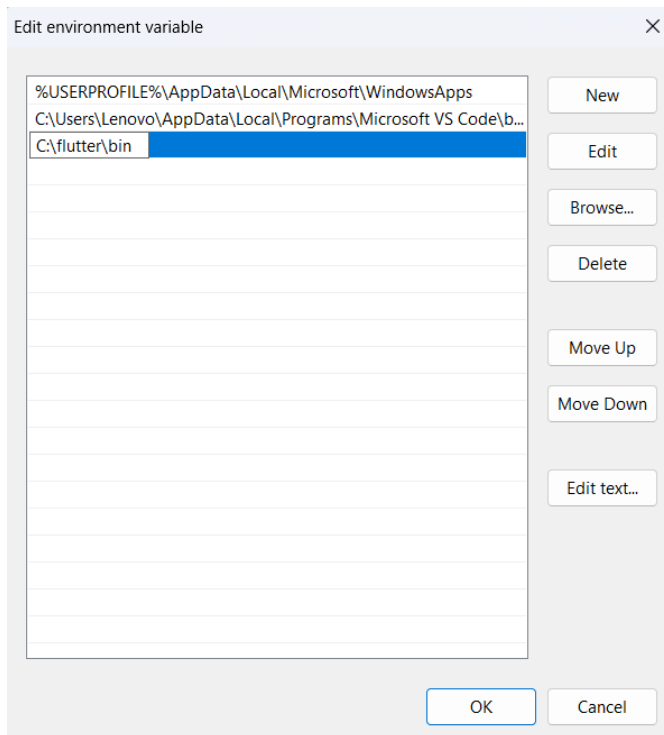


Step 5 :- Add Flutter to System PATH

Right-click on the Start Menu > System > Advanced system settings > Environment Variables.

Under System Variables, find Path and click Edit.

Add the full path to the flutter/bin directory (e.g., C:\flutter\bin).



Step 6 :- Now, run the \$ flutter command in command prompt.

```
Command Prompt - flutter
Microsoft Windows [Version 10.0.22631.4751]
(c) Microsoft Corporation. All rights reserved.

C:\Users\TEJAS>flutter
Manage your Flutter app development.

Common commands:

  flutter create <output directory>
    Create a new Flutter project in the specified directory.

  flutter run [options]
    Run your Flutter application on an attached device or in an emulator.

Usage: flutter <command> [arguments]

Global options:
-h, --help          Print this usage information.
-v, --verbose       Noisy logging, including all shell commands executed.
                    If used with "--help", shows hidden options. If used with "flutter doctor", shows additional
                    diagnostic information. (Use "-vv" to force verbose logging in those cases.)
-d, --device-id     Target device id or name (prefixes allowed).
--version           Reports the version of this tool.
--enable-analytics  Enable telemetry reporting each time a flutter or dart command runs.
--disable-analytics Disable telemetry reporting each time a flutter or dart command runs, until it is
                    re-enabled.
--suppress-analytics Suppress analytics reporting for the current CLI invocation.

Available commands:
```

Step 7:- Run the \$ flutter doctor command. This command checks for all the requirements of Flutter app development and displays a report of the status of your Flutter installation

```
Command Prompt - flutter - f x + v

You have received two consent messages because the flutter tool is migrating to a new analytics system. Disabling analytics collection will disable both the legacy and new analytics collection systems. You can disable analytics reporting by running 'flutter --disable-analytics'

C:\Users\TEJAS>flutter doctor
Doctor summary (to see all details, run flutter doctor -v):
[✓] Flutter (Channel stable, 3.27.2, on Microsoft Windows [Version 10.0.22631.4751], locale en-US)
[✓] Windows Version (Installed version of Windows is version 10 or higher)
[X] Android toolchain - develop for Android devices
    X Unable to locate Android SDK.
      Install Android Studio from: https://developer.android.com/studio/index.html
      On first launch it will assist you in installing the Android SDK components.
      (or visit https://flutter.dev/to/windows-android-setup for detailed instructions).
      If the Android SDK has been installed to a custom location, please use
      'flutter config --android-sdk' to update to that location.
[✓] Chrome - develop for the web
[X] Visual Studio - develop Windows apps
    X Visual Studio not installed; this is necessary to develop Windows apps.
      Download at https://visualstudio.microsoft.com/downloads/.
      Please install the "Desktop development with C++" workload, including all of its default components
[!] Android Studio (not installed)
[✓] VS Code (version 1.96.4)
[✓] Connected device (3 available)
[✓] Network resources

! Doctor found issues in 3 categories.
```

Step 8 :- Go to Android Studio and download the installer.

Developers

Essentials

Design & Plan

Develop

Google Play

Community

Search

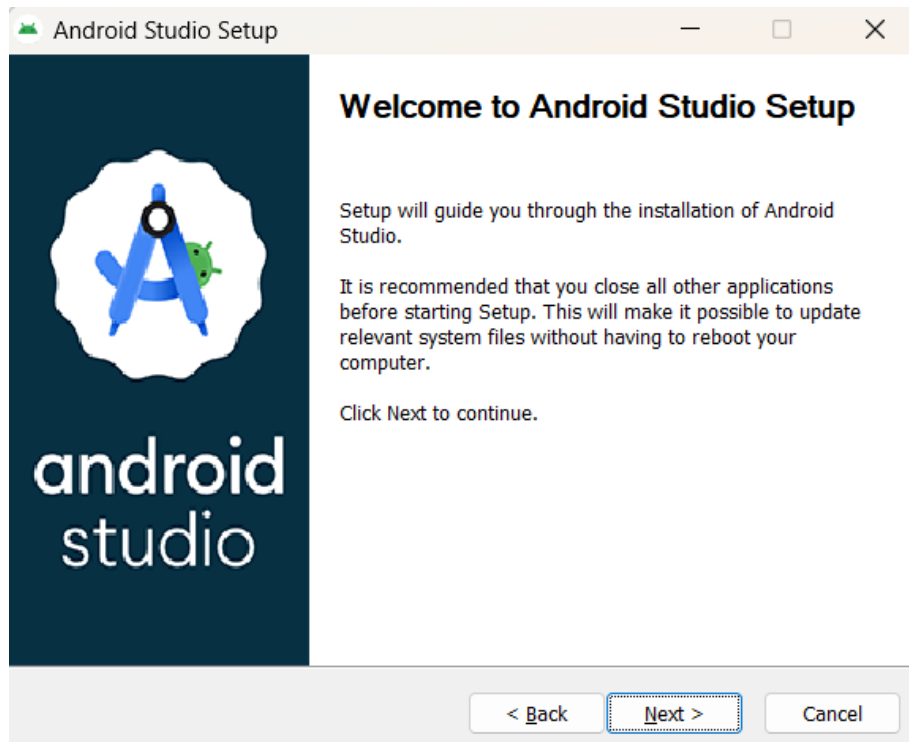
Language

Android Studio

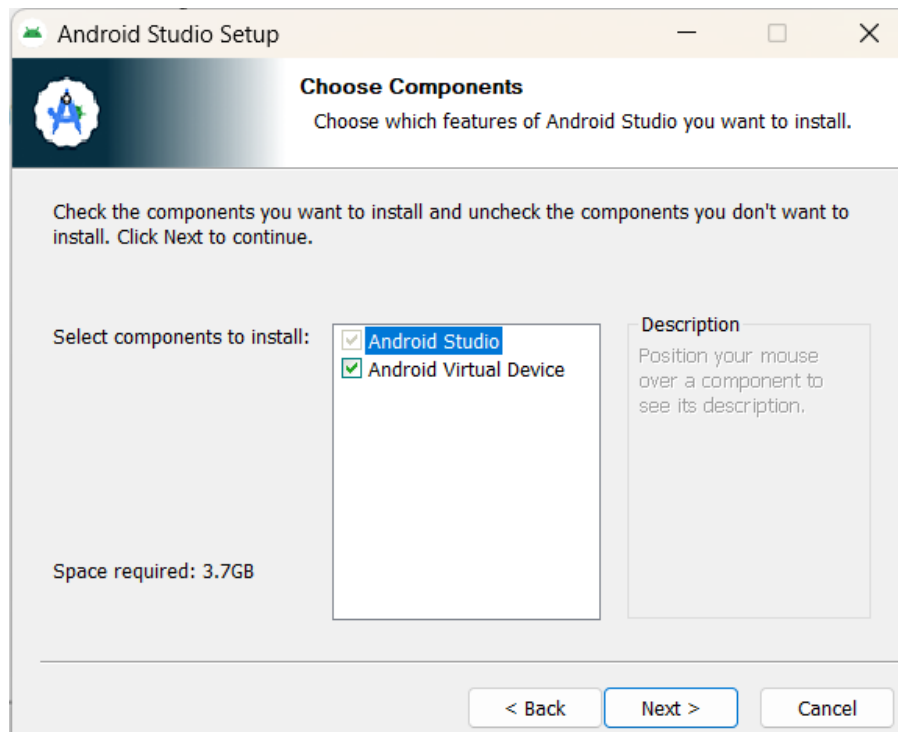
Download the latest version of Android Studio. For more information, see the [Android Studio release notes](#).

Platform	Android Studio package	Size	SHA-256 checksum
Windows (64-bit)	android-studio-2024.2.2.13-windows.exe Recommended	1.2 GB	7d93dd9bf3539f948f609b1968507bf1502bf6965d2d44bd38a7ff26cb5dd3e
Windows (64-bit)	android-studio-2024.2.2.13-windows.zip No .exe installer	1.2 GB	855945962ff9b84ea49ce39de0bf4189dbf451ae37a6fab7999da013b046b7f7
Mac (64-bit)	android-studio-2024.2.2.13-mac.dmg	1.3 GB	acfbb54d6ce8cf21f19b43510c7addcb9dde2824282f205fd1331be77d2e613
Mac (64-bit, ARM)	android-studio-2024.2.2.13-mac_arm.dmg	1.3 GB	688f8d007e612f3f0c18f316179079dc4565f93d8d1e6a7dad80c4fce356df7
Linux (64-bit)	android-studio-2024.2.2.13-linux.tar.gz	1.3 GB	b7fed4a7959bdaca7a8fd57461dbbf9a205eb23cc218ed828ed88e8b998cb5

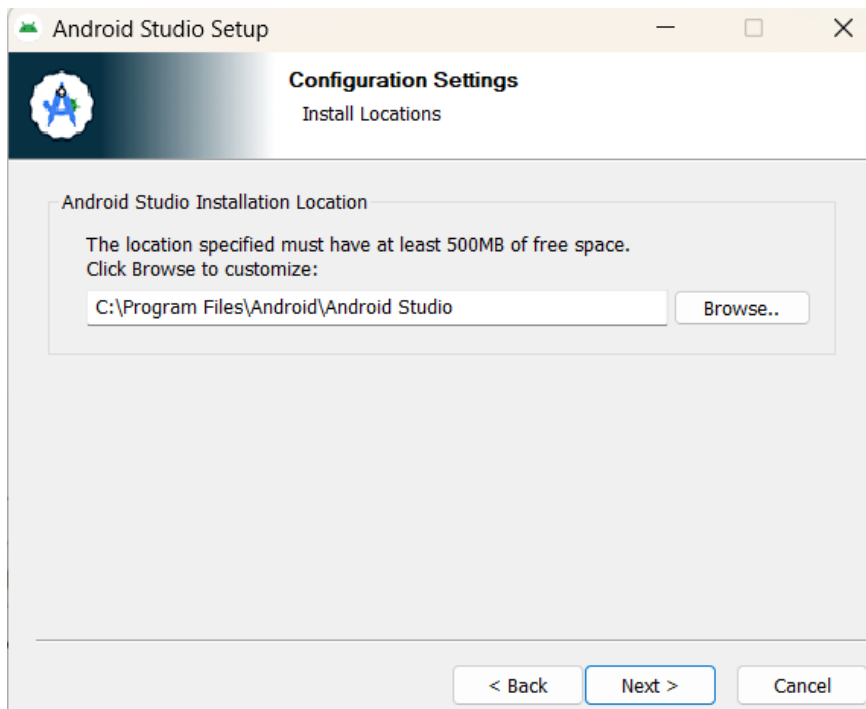
Step 8.1: - When the download is complete, open the .exe file and run it. You will get the following dialog box



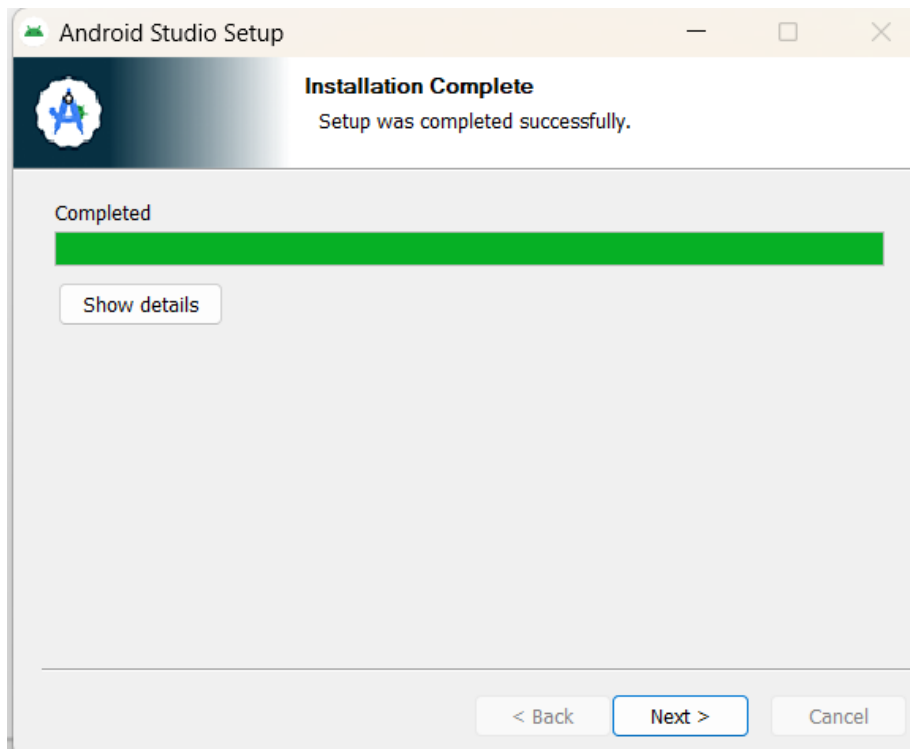
Step 8.2: - Select all the Checkboxes and Click on 'Next' Button.

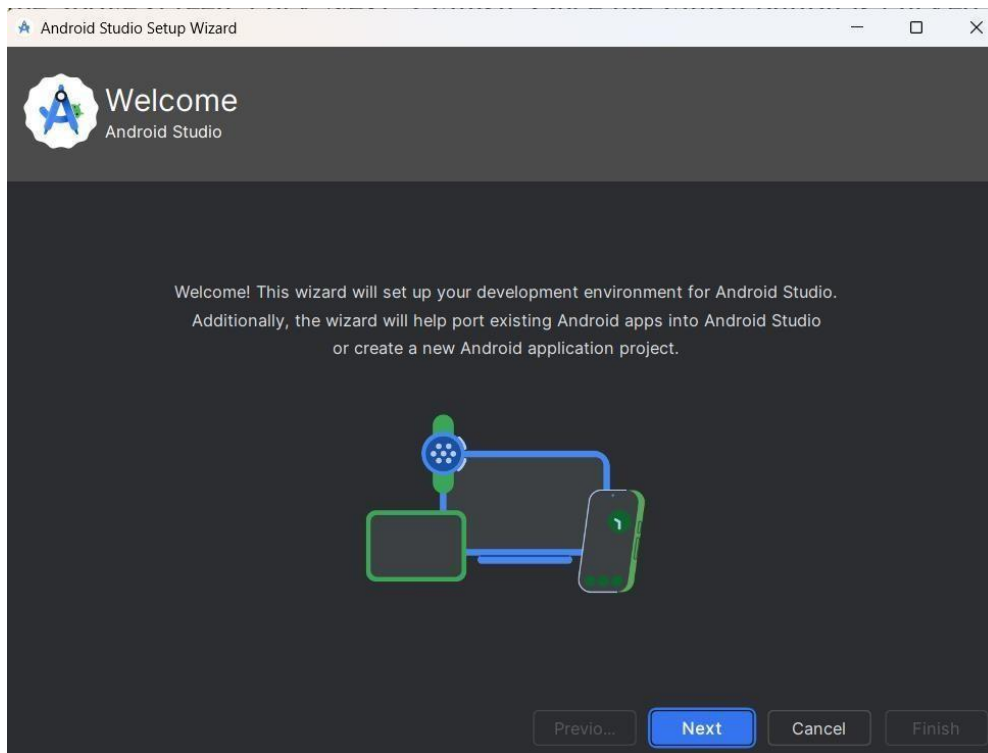


Step 8.3: - Change the destination as per your convenience and click on 'Next' Button.



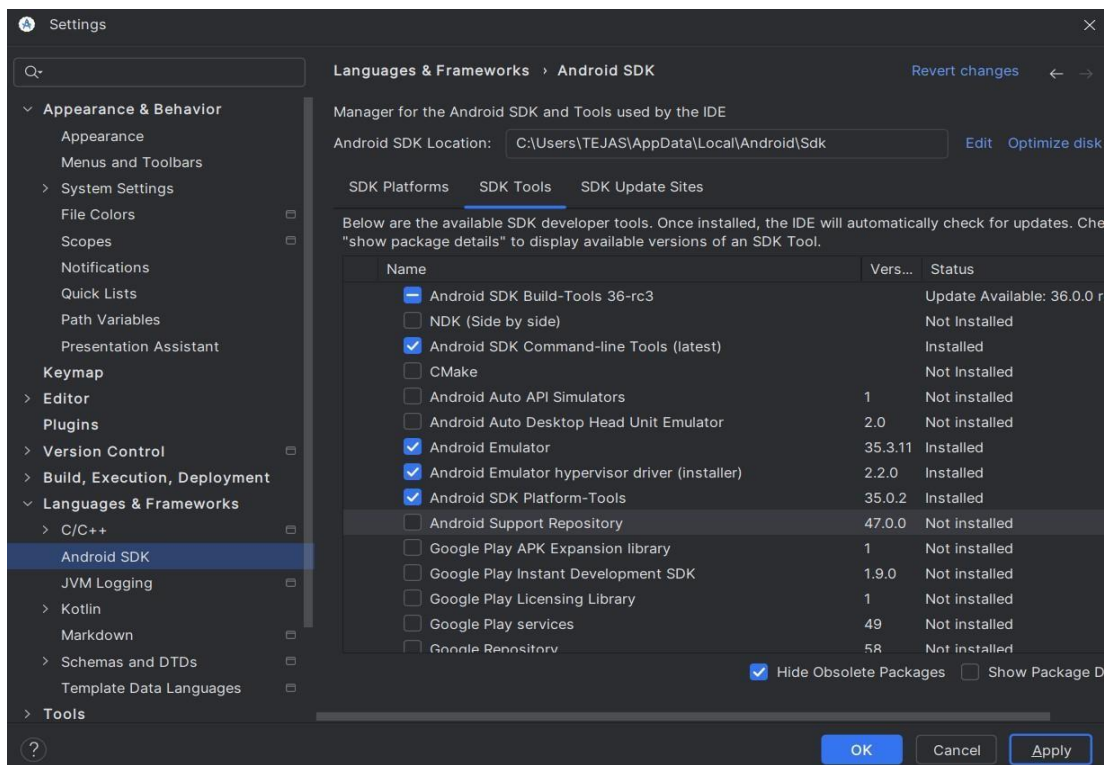
Step 8.4: - Follow the steps of the installation wizard. Once the installation wizard completes, you will get the following screen.





Step 8.5: - Go to Preferences > Appearance & Behavior > System Settings > Android SDK.

Select the SDK Tools tab and check Android SDK Command-line Tools and Install it.



Step 9: - Open a terminal and run the following command

```
Command Prompt - flutter - f  X + v

C:\Users\TEJAS>flutter doctor --android-licenses

Warning: Errors during XML parse:
Warning: Additionally, the fallback loader failed to parse the XML.ry...
Warning: Errors during XML parse:
Warning: Additionally, the fallback loader failed to parse the XML.
[=====] 100% Computing updates...
6 of 7 SDK package licenses not accepted.
Review licenses that have not been accepted (y/N)? y

1/6: License android-googletv-license:
-----
Terms and Conditions

This is the Google TV Add-on for the Android Software Development Kit License Agreement.

1. Introduction

1.1 The Google TV Add-on for the Android Software Development Kit (referred to in this License Agreement as the "Google TV Add-on" and specifically including the Android system files, packaged APIs, and Google APIs add-ons) is licensed to you subject to the terms of this License Agreement. This License Agreement forms a legally binding contract between you and Google in relation to your use of the Google TV Add-on.

1.2 "Google" means Google Inc., a Delaware corporation with principal place of business at 1600 Amphitheatre Parkway, Mountain View, CA 94043, United States.

2. Accepting this License Agreement

2.1 In order to use the Google TV Add-on, you must first agree to this License Agreement. You may not use the Google TV Add-on if you do not accept this License Agreement.
```

```
Command Prompt - flutter di  X + v

Doctor summary (to see all details, run flutter doctor -v):
[✓] Flutter (Channel stable, 3.27.2, on Microsoft Windows [Version 10.0.22631.4751], locale en-US)
[✓] Windows Version (Installed version of Windows is version 10 or higher)
[✓] Android toolchain - develop for Android devices (Android SDK version 35.0.1)
[✓] Chrome - develop for the web
[!] Visual Studio - develop Windows apps (Visual Studio Community 2022 17.12.4)
    X The current Visual Studio installation is incomplete.
      Please use Visual Studio Installer to complete the installation or reinstall Visual Studio.
[✓] Android Studio (version 2024.2)
[✓] VS Code (version 1.96.4)
[✓] Connected device (3 available)
[✓] Network resources

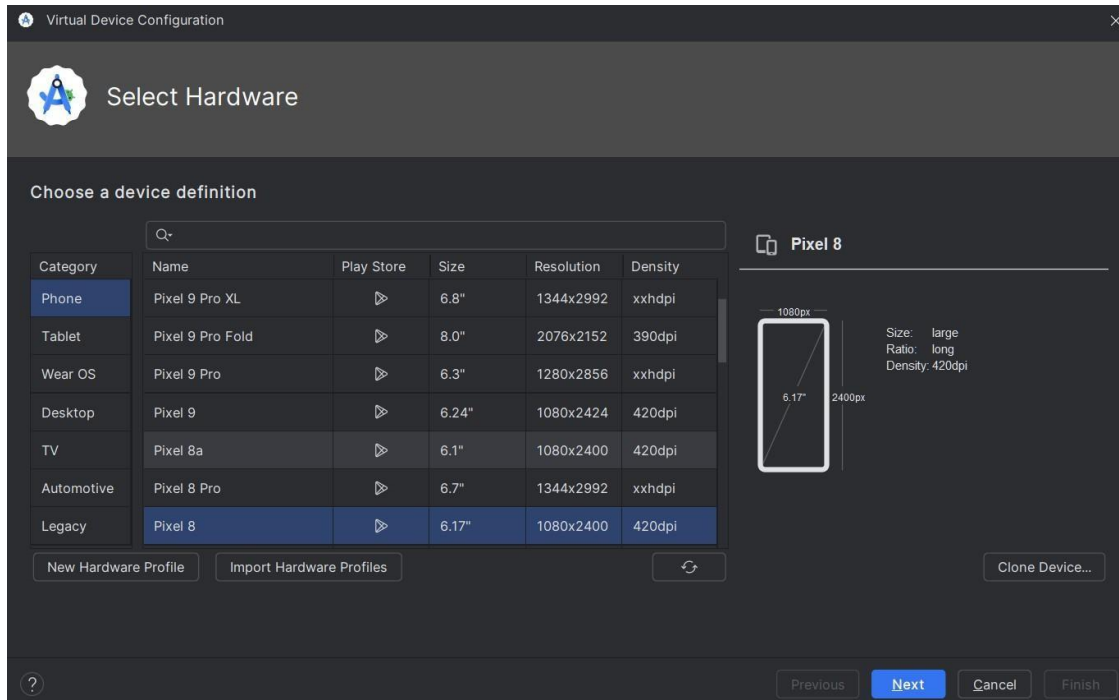
! Doctor found issues in 1 category.

C:\Users\TEJAS>flutter doctor
Doctor summary (to see all details, run flutter doctor -v):
[✓] Flutter (Channel stable, 3.27.2, on Microsoft Windows [Version 10.0.22631.4751], locale en-US)
[✓] Windows Version (Installed version of Windows is version 10 or higher)
[✓] Android toolchain - develop for Android devices (Android SDK version 35.0.1)
[✓] Chrome - develop for the web
[✓] Visual Studio - develop Windows apps (Visual Studio Community 2022 17.12.4)
[✓] Android Studio (version 2024.2)
[✓] VS Code (version 1.96.4)
[✓] Connected device (3 available)
[✓] Network resources

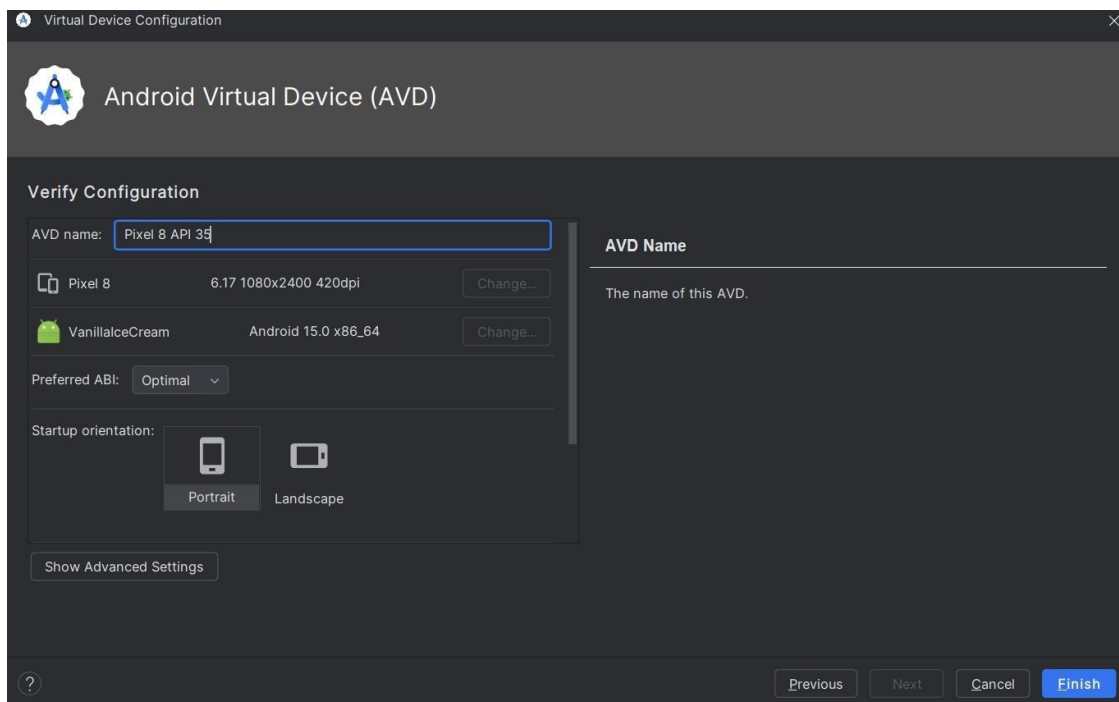
• No issues found!

C:\Users\TEJAS>
```

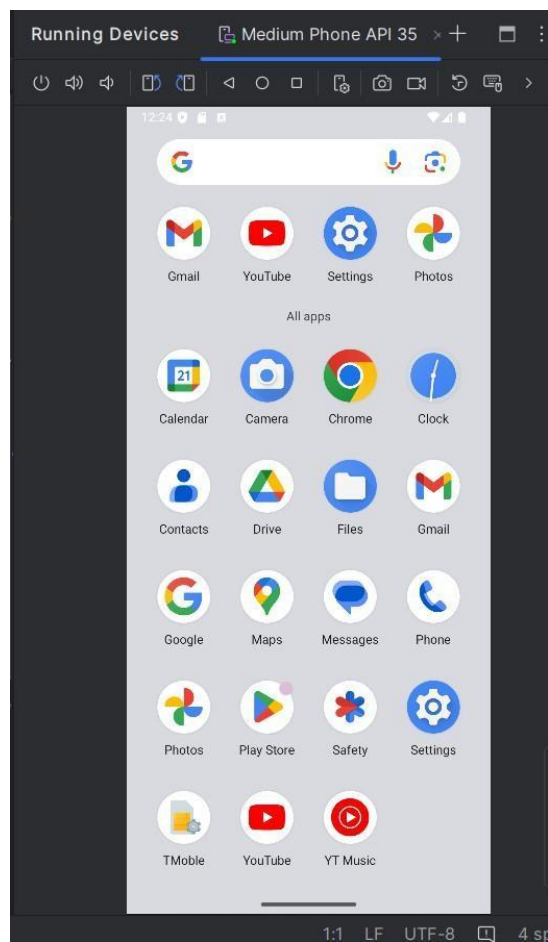
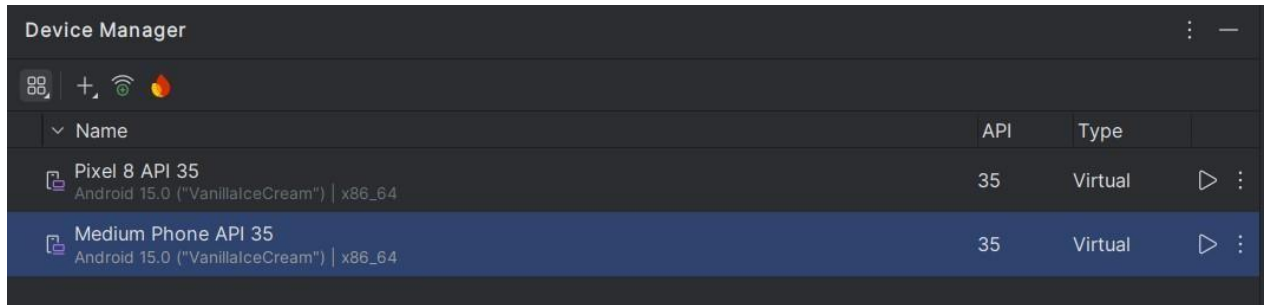
Step 10: - Next, you need to set up an Android emulator. It is responsible for running and testing the Flutter application



Step 10.1: - Open Android Studio and go to Tools > AVD Manager. Create a new virtual device.

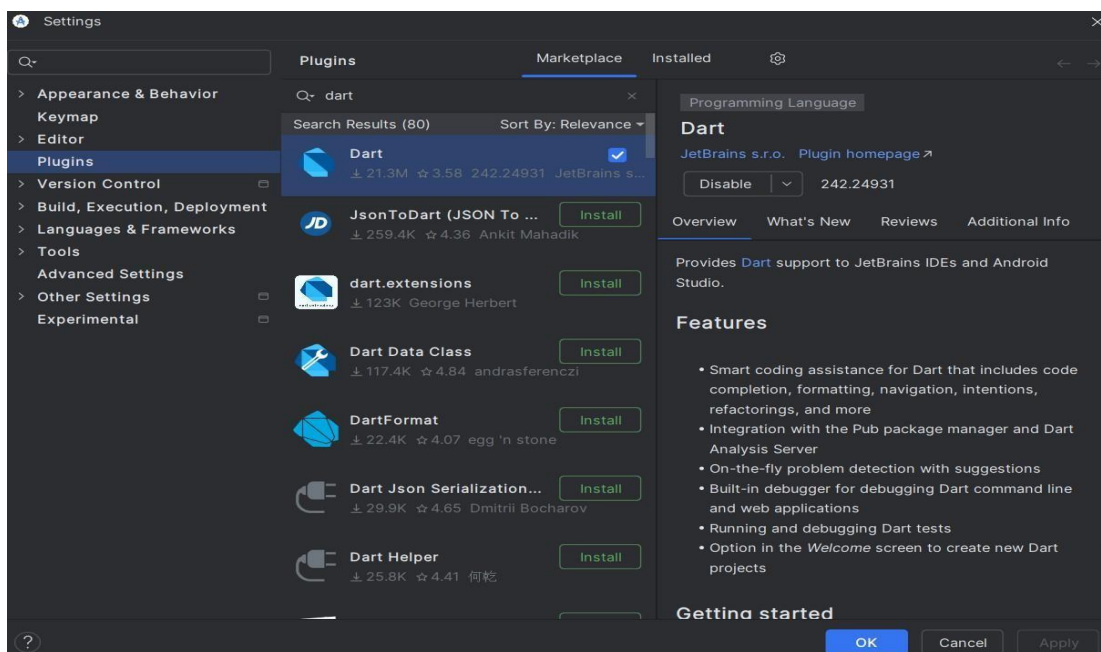
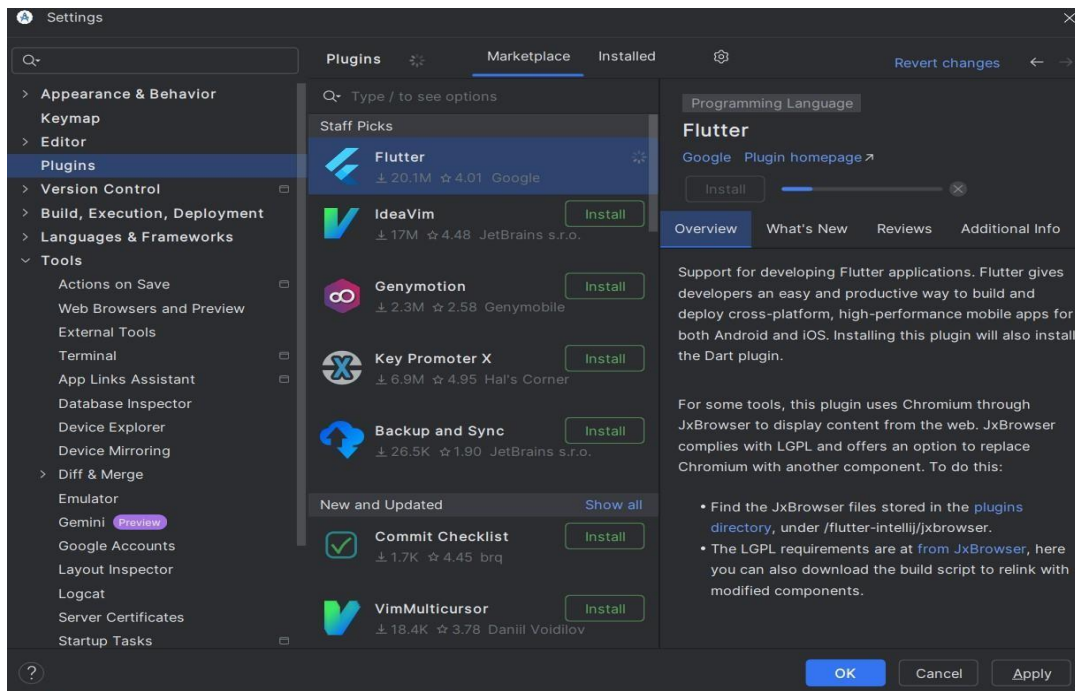


Step 10.2: - Click on the icon pointed into the red color rectangle. The Android emulator displayed as below screen



Step 11: - Now, install Flutter and Dart plugin for building Flutter application in Android Studio. These plugins provide a template to create a Flutter application, give an option to run and debug Flutter application in the Android Studio itself

Step 11.1: - Open the Android Studio and then go to File->Settings->Plugins. Now, search the Flutter plugin. If found, select Flutter plugin and click install



Step 11.2: - Restart the Android Studio

Step 12: - Go to File > New Project > Create Flutter Project, then select the project name and location, and click Next to proceed.

Code:

```
import 'package:flutter/material.dart';
```

```
void main() {  
  runApp(const MyApp());  
}
```

```
class MyApp extends StatelessWidget {  
  const MyApp({super.key});
```

```
  @override
```

```
  Widget build(BuildContext context) {
```

```
    return MaterialApp(  
      debugShowCheckedModeBanner: false,  
      title: 'Hello World App',  
      theme: ThemeData(  
        primarySwatch: Colors.blue,  
        visualDensity: VisualDensity.adaptivePlatformDensity,  
      ),  
      home: const HomePage(),  
    );  
  }  
}
```

```
class HomePage extends StatelessWidget {  
  const HomePage({super.key});
```

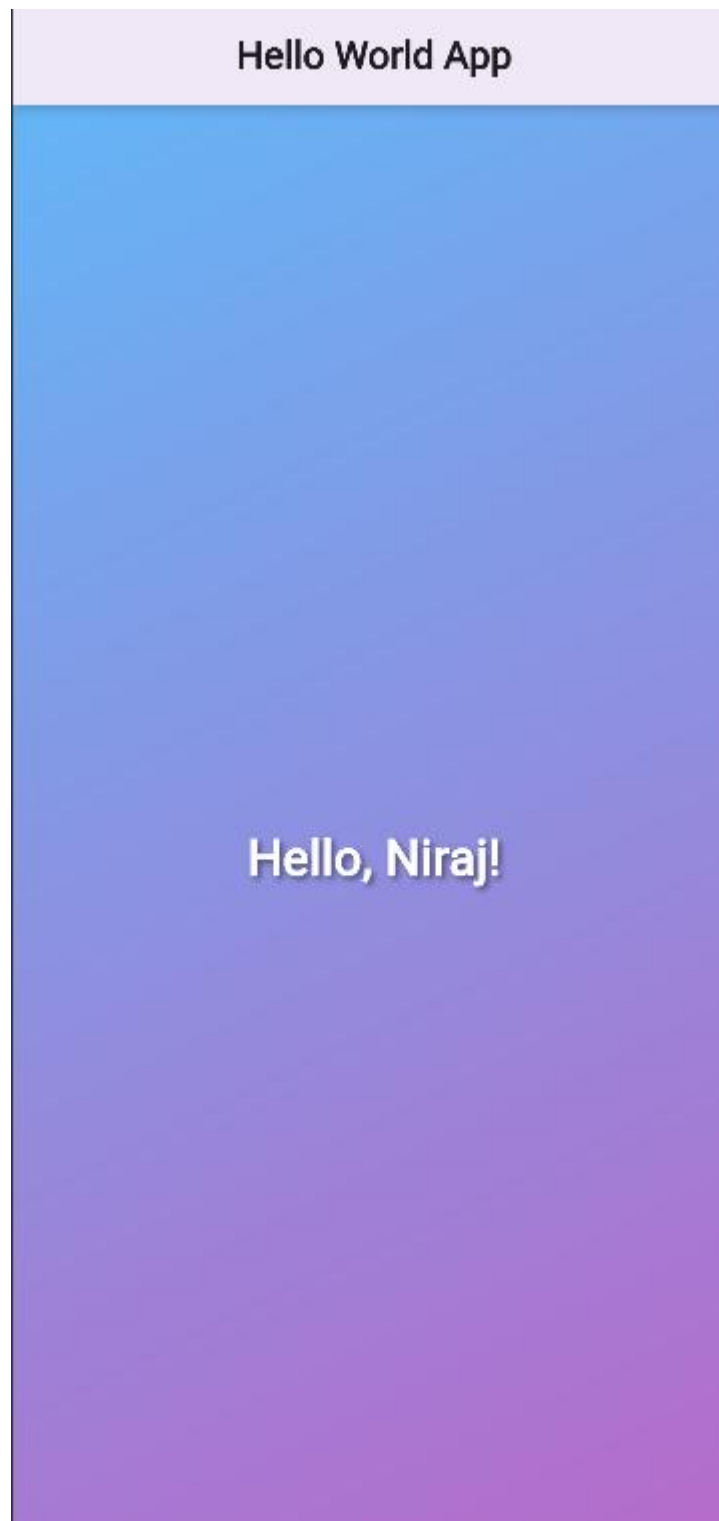
```
  @override
```

```
  Widget build(BuildContext context) {
```

```
    return Scaffold(  
      appBar: AppBar(  
        title: const Text(  
          'Hello World App',  
          style: TextStyle(fontWeight: FontWeight.bold),  
        ),  
      ),  
    );  
  }  
}
```

```
        centerTitle: true,
        elevation: 4,
        shadowColor: Colors.black54,
    ),
    body: Container(
      decoration: BoxDecoration(
        gradient: LinearGradient(
          begin: Alignment.topLeft,
          end: Alignment.bottomRight,
          colors: [Colors.blue.shade300, Colors.purple.shade300],
        ),
      ),
    child: const Center(
      child: Text(
        'Hello, Niraj!',
        style: TextStyle(
          fontSize: 28,
          fontWeight: FontWeight.bold,
          color: Colors.white,
          shadows: [
            Shadow(
              blurRadius: 4,
              color: Colors.black45,
              offset: Offset(2, 2),
            ),
          ],
        ),
      ),
    ),
  );
}
```

Output:



MAD & PWA Lab

Journal

Experiment No.	02
Experiment Title.	To design Flutter UI by including common widgets.
Roll No.	24
Name	Niraj Kothawade
Class	D15A/D15B
Subject	MAD & PWA Lab
Lab Outcome	LO2: Design and Develop interactive Flutter App by using widgets, layouts, gestures and animation
Grade:	

MAD & PWA LAB

Experiment 2

Name: Niraj Kothawade

Class: D15A

Roll no: 24

Aim: To design Flutter UI by including common widgets.

Code:

Landing page:

```
import 'package:flutter/material.dart';

import
'package:whatsapp_messenger/common/
routes/routes.dart';

import
'package:whatsapp_messenger/common/
utils/coloors.dart';

import
'package:whatsapp_messenger/common/
widgets/custom_elevated_button.dart';

import
'package:whatsapp_messenger/feature/w
elcome/widgets/language_button.dart';

import
'package:whatsapp_messenger/feature/w
elcome/widgets/privacy_and_terms.dart';

class WelcomePage extends
StatelessWidget {
  const WelcomePage({super.key});

  navigateToLoginPage(context) {
    Navigator.of(context)
      .pushNamedAndRemoveUntil(Routes.
login, (route) => false);
  }
}
```

```
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    body: Column(
      children: [
        Expanded(
          child: Align(
            alignment:
Alignment.bottomCenter,
            child: Padding(
              padding: const
EdgeInsets.symmetric(horizontal: 50,
vertical: 10),
              child:
Image.asset('assets/images/circle.png',
                color: Colors.greenDark),
            ),
          ),
        const SizedBox(height: 40),
        Expanded(
          child: Column(
```

```

children: [
  const Text(
    'Welcome to WhatsApp',
    style: TextStyle(
      fontSize: 22,
      fontWeight: FontWeight.bold,
    ),
  ),
  const PrivacyAndTerms(),
  CustomElevatedButton(
    onPressed: () =>
navigateToLoginPage(context),
    text: 'AGREE AND CONTINUE',
  ),
  const SizedBox(height: 50),
  LanguageButton(),
],
),
],
),
);
}
}

```

Authentication page:

```

import 'package:flutter/material.dart';

import
'package:flutter_riverpod/flutter_riverpod
.dart';

```

```

import
'package:whatsapp_messenger/common/
extension/custom_theme_extension.dart';

import
'package:whatsapp_messenger/feature/a
uth/controller/auth_controller.dart';

import
'package:whatsapp_messenger/feature/a
uth/widgets/custom_text_field.dart';

class VerificationPage extends
ConsumerWidget {
  const VerificationPage({
    super.key,
    required this.smsCodeId,
    required this.phoneNumber,
  });

  final String smsCodeId;
  final String phoneNumber;

  Future<void> verifySmsCode(
    BuildContext context,
    WidgetRef ref,
    String smsCode,
  ) async {
    await
ref.read(authControllerProvider).verifySms
Code(
    context: context,
    smsCodeId: smsCodeId,
    smsCode: smsCode,

```

```

    );
}

@override
Widget build(BuildContext context,
WidgetRef ref) {
  return Scaffold(
    appBar: AppBar(
      backgroundColor:
Theme.of(context).scaffoldBackgroundCol
or,
      elevation: 0,
      centerTitle: true,
      title: Text(
        'Verify your number',
        style: TextStyle(
          color:
context.theme.authAppBarTextColor,
        ),
      ),
      actions: [
        IconButton(
          onPressed: () {},
          icon: const Icon(Icons.more_vert),
        ),
      ],
    ),
    body: SingleChildScrollView(
      padding: const
EdgeInsets.symmetric(horizontal: 20),
      child: Column(

```

```

        children: [
          Padding(
            padding: const
EdgeInsets.symmetric(horizontal: 10),
            child: RichText(
              textAlign: TextAlign.center,
              text: TextSpan(
                style: TextStyle(color:
context.theme.greyColor),
                children: [
                  const TextSpan(
                    text:
                      "You've tried to register
before requesting an SMS or Call with your
code.",
                    ),
                  TextSpan(
                    text: " Wrong number?",
                    style: TextStyle(
                      color:
context.theme.blueColor,
                    ),
                  ),
                ],
              ),
            ),
          const SizedBox(height: 20),
          Container(
            padding: const
EdgeInsets.symmetric(horizontal: 80),

```

```

        child: CustomTextField(
            hintText: "--- ---",
            fontSize: 30,
            autoFocus: true,
            keyboardType:
TextInputType.number,
            onChanged: (value) async {
                if (value.length == 6) {
                    await verifySmsCode(context,
ref, value);
                }
            },
        ),
    ),
    const SizedBox(height: 20),
    Text(
        'Enter 6-digit code',
        style: TextStyle(color:
context.theme.greyColor),
    ),
    const SizedBox(height: 30),
    Row(
        children: [
            Icon(Icons.message, color:
context.theme.greyColor),
            const SizedBox(width: 20),
            Text(
                'Resend SMS',
                style: TextStyle(
                    color: context.theme.greyColor,
                ),
            ),
        ],
    ),
    const SizedBox(height: 10),
    Divider(
        color:
context.theme.greyColor!.withOpacity(.2),
    ),
    const SizedBox(height: 10),
    Row(
        children: [
            Icon(Icons.phone, color:
context.theme.greyColor),
            const SizedBox(width: 20),
            Text(
                'Call Me',
                style: TextStyle(
                    color: context.theme.greyColor,
                ),
            ),
        ],
    ),
    ],
);
}
}

```

Chats page:

```
import
'package:cached_network_image/cached_network_image.dart';

import 'package:flutter/material.dart';

import
'package:flutter_riverpod/flutter_riverpod.dart';

import 'package:intl/intl.dart';

import
'package:whatsapp_messenger/common/extension/custom_theme_extension.dart';

import
'package:whatsapp_messenger/common/models/last_message_model.dart';

import
'package:whatsapp_messenger/common/models/user_model.dart';

import
'package:whatsapp_messenger/common/routes/routes.dart';

import
'package:whatsapp_messenger/common/utils/colors.dart';

import
'package:whatsapp_messenger/feature/chat/controller/chat_controller.dart';

class ChatHomePage extends ConsumerWidget {

  const ChatHomePage({super.key});

  List<LastMessageModel>
  getDemoChats() {

    return [
```

```
      LastMessageModel(
        username: 'Dev Stack',
        contactId: '1',
        profileImageUrl:
        'https://picsum.photos/seed/1/200',
        lastMessage: 'Hi Everyone 🙌',
        timeSent:
        DateTime.now().subtract(const Duration(minutes: 4)),
      ),
      LastMessageModel(
        username: 'Kishor',
        contactId: '2',
        profileImageUrl:
        'https://picsum.photos/seed/2/200',
        lastMessage: 'Hi Kishor',
        timeSent:
        DateTime.now().subtract(const Duration(hours: 1)),
      ),
      LastMessageModel(
        username: 'Dev Server Chat',
        contactId: '3',
        profileImageUrl:
        'https://picsum.photos/seed/3/200',
        lastMessage: 'Alice: The new API is ready for testing',
        timeSent:
        DateTime.now().subtract(const Duration(hours: 2)),
      ),
      LastMessageModel(
```

```

        username: 'Collins',
        contactId: '4',
        imageUrl:
'https://picsum.photos/seed/4/200',
        lastMessage: 'Hey Dev Stack! How\'s
the project going?',
        timeSent:
DateTime.now().subtract(const
Duration(hours: 4)),
    ),
    LastMessageModel(
        username: 'Friends',
        contactId: '5',
        imageUrl:
'https://picsum.photos/seed/5/200',
        lastMessage: 'John: Anyone up for
coffee? ☕ ',
        timeSent:
DateTime.now().subtract(const
Duration(hours: 5)),
    ),
    LastMessageModel(
        username: 'Flutter Community',
        contactId: '6',
        imageUrl:
'https://picsum.photos/seed/6/200',
        lastMessage: 'New flutter 3.0 features
announced! 🎉 ',
        timeSent:
DateTime.now().subtract(const
Duration(hours: 10)),
    ),
    LastMessageModel(

```

```

        username: 'Balram Rathore',
        contactId: '7',
        imageUrl:
'https://picsum.photos/seed/7/200',
        lastMessage: 'happy coding
🔥 🔥 🔥 ',
        timeSent:
DateTime.now().subtract(const
Duration(hours: 10)),
    ),
  ];
}

@override
Widget build(BuildContext context,
WidgetRef ref) {
  return Scaffold(
    backgroundColor: const
Color(0xFF121B22),
    body: ListView.builder(
      itemCount: getDemoChats().length,
      shrinkWrap: true,
      itemBuilder: (context, index) {
        final lastMessageData =
getDemoChats()[index];

        return Column(
          children: [
            ListTile(
              onTap: () {
                Navigator.pushNamed(

```

```

        context,
        Routes.chat,
        arguments: UserModel(
          username:
lastMessageData.username,
          uid:
lastMessageData.contactId,
          profileImageUrl:
lastMessageData.profileImageUrl,
          active: true,
          lastSeen: 0,
          phoneNumber: '0',
          groupId: [],
        ),
      );
    },
    title: Row(
      mainAxisAlignment:
MainAxisAlignment.spaceBetween,
      children: [
        Expanded(
          child: Text(
            lastMessageData.username,
            style: const TextStyle(
              fontWeight:
FontWeight.w500,
              color: Colors.white,
            ),
            overflow:
TextOverflow.ellipsis,
          ),

```

```

        ),
        Text(
          DateFormat.Hm().format(last
MessageData.timeSent),
          style: const TextStyle(
            fontSize: 13,
            color: Colors.grey,
          ),
        ),
      ],
    ),
    subtitle: Padding(
      padding: const
EdgeInsets.only(top: 3),
      child: Row(
        children: [
          const Icon(
            Icons.done_all,
            size: 16,
            color: Colors.greenDark,
          ),
          const SizedBox(width: 3),
          Expanded(
            child: Text(
              lastMessageData.lastMessa
ge,
              maxLines: 1,
              overflow:
TextOverflow.ellipsis,
              style: const TextStyle(
                color: Colors.grey,

```



```

        ),
        ),
        ),
    ],
    ),
    ),
    leading: CircleAvatar(
        backgroundImage:
CachedNetworkImageProvider(
        lastMessageData.profileImage
Url,
    ),
    radius: 24,
    ),
    ),
    Divider(
        color: Colors.grey[900],
        indent: 80,
        height: 0,
    ),
    ],
    );
},
),
floatingActionButton:
FloatingActionButton(
    backgroundColor: Colors.greenDark,
    onPressed: () {
        Navigator.pushNamed(context,
Routes.contact);
    },

```

```

        child: const Icon(
            Icons.message,
            color: Colors.white,
        ),
    ),
    );
}
}

```

Status Page:

```

import 'dart:html' as html;
import 'package:flutter/material.dart';
import
'package:image_picker/image_picker.dart'
;

```

// Enhanced Status Model

```

class Status {
    final String id;
    final String name;
    final String imageUrl; // Changed from
imagePath to imageUrl
    final DateTime timestamp;
    final bool isViewed;
    final StatusType type;
    final String? caption;

    Status({
        required this.id,

```

```

    required this.name,
    required this.imageUrl,
    required this.timestamp,
    this.isViewed = false,
    this.type = StatusType.image,
    this.caption,
  });
}

enum StatusType { image, text }

class StatusHomePage extends
StatefulWidget {
  const StatusHomePage({super.key});

  @override
  State<StatusHomePage> createState() =>
    _StatusHomePageState();
}

class _StatusHomePageState extends
State<StatusHomePage> {
  final ImagePicker _picker = ImagePicker();
  List<Status> recentUpdates = [];
  List<Status> viewedUpdates = [];
  Status? myCurrentStatus;

  @override
  void initState() {
    super.initState();

    _loadDemoStatuses();
  }

  void _loadDemoStatuses() {
    // Load demo statuses with web-
    compatible URLs
    recentUpdates = [
      Status(
        id: '1',
        name: 'Balram Rathore',
        imageUrl:
        'https://picsum.photos/400/600', // Using
        placeholder image
        timestamp: DateTime.now(),
      ),
      Status(
        id: '2',
        name: 'Kishor Kumar',
        imageUrl:
        'https://picsum.photos/400/601', //
        Different placeholder
        timestamp: DateTime.now(),
      ),
    ];

    viewedUpdates = [
      Status(
        id: '3',
        name: 'Saket Sinha',
        imageUrl:
        'https://picsum.photos/400/602',

```

```

        timestamp:
DateTime.now().subtract(const
Duration(hours: 1)),
        isViewed: true,
    ),
];
}

Future<void> _addImageStatus() async {
    try {
        final XFile? image = await
_picker.pickImage(source:
ImageSource.gallery);
        if (image == null) return;

        // Create object URL for web
        final bytes = await
image.readAsBytes();
        final blob = html.Blob([bytes]);
        final url =
html.Url.createObjectUrlFromBlob(blob);

        setState(() {
            myCurrentStatus = Status(
                id: DateTime.now().toString(),
                name: 'My Status',
                imageUrl: url,
                timestamp: DateTime.now(),
                type: StatusType.image,
            );

```

```

        // Add to recent updates
        recentUpdates.insert(0,
myCurrentStatus!);
    });
} catch (e) {
    showDialog(
        context: context,
        builder: (context) => AlertDialog(
            title: const Text('Error'),
            content: Text('Failed to add status:
$e'),
            actions: [
                TextButton(
                    onPressed: () =>
Navigator.pop(context),
                    child: const Text('OK'),
                ),
            ],
        ),
    );
}

Future<void> _addTextStatus() async {
    final TextEditingController textController
= TextEditingController();
    final result = await showDialog<String>(
        context: context,
        builder: (context) => AlertDialog(
            backgroundColor: const
Color(0xFF121B22),

```

```

        title: const Text('Add Text Status',
          style: TextStyle(color:
Colors.white)),
        content: TextField(
          controller: textController,
          style: const TextStyle(color:
Colors.white),
          maxLines: 3,
          decoration: const InputDecoration(
            hintText: 'Enter your status...',
            hintStyle: TextStyle(color:
Colors.grey),
            border: OutlineInputBorder(),
          ),
        ),
        actions: [
          TextButton(
            onPressed: () =>
Navigator.pop(context),
            child: const Text('Cancel'),
          ),
          TextButton(
            onPressed: () =>
Navigator.pop(context,
textController.text),
            child: const Text('Post'),
          ),
        ],
      ),
    );

```

```

    if (result != null && result.isNotEmpty) {
      setState(() {
        myCurrentStatus = Status(
          id: DateTime.now().toString(),
          name: 'My Status',
          imageUrl: '', // Empty for text status
          timestamp: DateTime.now(),
          type: StatusType.text,
          caption: result,
        );

        // Add to recent updates
        recentUpdates.insert(0,
myCurrentStatus!);
      });
    }
  }

  void _viewStatus(Status status) {
    Navigator.push(
      context,
      MaterialPageRoute(
        builder: (context) =>
StatusViewPage(status: status),
      ),
    );

    // Mark status as viewed
    if (!status.isViewed &&
recentUpdates.contains(status)) {

```

```

    setState(() {
      final index =
recentUpdates.indexOf(status);

      final viewedStatus = Status(
        id: status.id,
        name: status.name,
        imageUrl: status.imageUrl,
        timestamp: status.timestamp,
        isViewed: true,
        type: status.type,
        caption: status.caption,
      );

      recentUpdates.removeAt(index);
      viewedUpdates.insert(0,
viewedStatus);
    });
  }
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    backgroundColor: const
Color(0xFF121B22),
    body: SingleChildScrollView(
      child: Column(
        crossAxisAlignment:
CrossAxisAlignment.start,
        children: [

          // My Status Section

```

```

ListTile(
  leading: Stack(
    children: [
      CircleAvatar(
        radius: 30,
        backgroundColor: Colors.grey,
        backgroundImage:
myCurrentStatus?.type ==
StatusType.image
          ?
NetworkImage(myCurrentStatus!.imageUr
l)
          : null,
        child: myCurrentStatus == null
          ? const Icon(Icons.person,
            color: Colors.white, size:
35)
          : myCurrentStatus?.type ==
StatusType.text
            ? const
Icon(Icons.text_fields, color: Colors.white)
            : null,
      ),
    ],
  ),
  Positioned(
    bottom: 0,
    right: 0,
    child: Container(
      padding: const
EdgeInsets.all(4),
      decoration: BoxDecoration(
        color: Colors.green,

```

<pre> borderRadius: BorderRadius.circular(12),), child: const Icon(Icons.add, color: Colors.white, size: 20,),),),],), title: const Text('My status', style: TextStyle(color: Colors.white, fontWeight: FontWeight.bold,),), subtitle: Text(myCurrentStatus != null ? 'Tap to view your status' : 'Tap to add status update', style: const TextStyle(color: Colors.grey),), onTap: () { if (myCurrentStatus != null) { _viewStatus(myCurrentStatus!); } } </pre>	<pre> },), if (recentUpdates.isNotEmpty) ...[const Padding(padding: EdgeInsets.symmetric(horizontal: 15, vertical: 8), child: Text('Recent updates', style: TextStyle(color: Colors.grey, fontWeight: FontWeight.bold,),),), ListView.builder(shrinkWrap: true, physics: const NeverScrollableScrollPhysics(), itemCount: recentUpdates.length, itemBuilder: (context, index) => StatusTile(status: recentUpdates[index], onTap: () => _viewStatus(recentUpdates[index]),),),], </pre>
---	---

```

    if (viewedUpdates.isNotEmpty) ...[
      const Padding(
        padding:
EdgeInsets.symmetric(horizontal: 15,
vertical: 8),
        child: Text(
          'Viewed updates',
          style: TextStyle(
            color: Colors.grey,
            fontWeight: FontWeight.bold,
          ),
        ),
      ),
    ],
    ListView.builder(
      shrinkWrap: true,
      physics: const
NeverScrollableScrollPhysics(),
      itemCount:
viewedUpdates.length,
      itemBuilder: (context, index) =>
StatusTile(
        status: viewedUpdates[index],
        onTap: () =>
_viewStatus(viewedUpdates[index]),
      ),
    ],
  ],
),
),
floatingActionButton: Column(

```

```

      mainAxisAlignment:
MainAxisAlignment.end,
      children: [
        FloatingActionButton(
          mini: true,
          onPressed: _addTextStatus,
          backgroundColor: Colors.grey[700],
          child: const Icon(Icons.edit),
        ),
        const SizedBox(height: 16),
        FloatingActionButton(
          onPressed: _addImageStatus,
          backgroundColor: Colors.teal[700],
          child: const Icon(Icons.camera_alt),
        ),
      ],
    ),
  );
}
}

```

```

class StatusTile extends StatelessWidget {
  final Status status;
  final VoidCallback onTap;

  const StatusTile({
    super.key,
    required this.status,
    required this.onTap,
  });
}

```

```

@override
Widget build(BuildContext context) {
  return ListTile(
    leading: Container(
      decoration: BoxDecoration(
        shape: BoxShape.circle,
        border: Border.all(
          color: status.isViewed ? Colors.grey
: Colors.green,
          width: 2,
        ),
      ),
    child: CircleAvatar(
      radius: 28,
      backgroundColor: Colors.grey,
      backgroundImage: status.type ==
StatusType.image
        ? NetworkImage(status.imageUrl)
        : null,
    child: status.type == StatusType.text
        ? const Icon(Icons.text_fields,
color: Colors.white)
        : null,
    ),
  ),
  title: Text(
    status.name,
    style: const TextStyle(
      color: Colors.white,

```

```

fontWeight: FontWeight.bold,
    ),
  ),
  subtitle: Text(
    'Today,
    ${status.timestamp.hour}:${status.timesta
mp.minute.toString().padLeft(2, '0')}',
    style: const TextStyle(color:
Colors.grey),
  ),
  onTap: onTap,
);
}
}

```

```

class StatusViewPage extends
StatelessWidget {

```

```

  final Status status;

```

```

  const StatusViewPage({super.key,
required this.status});

```

```

@override

```

```

Widget build(BuildContext context) {

```

```

  return Scaffold(

```

```

    backgroundColor: Colors.black,

```

```

    appBar: AppBar(

```

```

      backgroundColor: Colors.transparent,

```

```

      title: Text(status.name),

```

```

      iconTheme: const

```

```

IconThemeData(color: Colors.white),

```



```

    ),
    body: Center(
      child: status.type == StatusType.image
        ? Image.network(
            status.imageUrl,
            fit: BoxFit.contain,
            loadingBuilder: (context, child,
loadingProgress) {
              if (loadingProgress == null)
return child;

              return const
CircularProgressIndicator();
            },
            errorBuilder: (context, error,
stackTrace) {
              return const Icon(
                Icons.error,
                color: Colors.red,
                size: 48,
              );
            },
        )
      : Container(
          padding: const EdgeInsets.all(20),
          color: Colors.teal,
          child: Center(
            child: Text(
              status.caption ?? "",
              style: const TextStyle(
                color: Colors.white,
                fontSize: 24,

```

```

            ),
            textAlign: TextAlign.center,
          ),
        ),
      ),
    );
  }
}

```

Call page:

```

import 'package:flutter/material.dart';

class Call {
  final String name;
  final String avatarUrl;
  final DateTime timestamp;
  final bool isIncoming;
  final bool isMissed;
  final bool isVideo;

  Call({
    required this.name,
    required this.avatarUrl,
    required this.timestamp,
    this.isIncoming = true,
    this.isMissed = false,
    this.isVideo = false,
  });
}

```

```

class CallHomePage extends StatefulWidget {
  const CallHomePage({super.key});

  @override
  State<CallHomePage> createState() =>
    _CallHomePageState();
}

```

```

class _CallHomePageState extends
State<CallHomePage> {

  // Expanded demo call data with Picsum
  images

  final List<Call> calls = [

    Call(
      name: "Mike Johnson",
      avatarUrl:
        "https://picsum.photos/seed/mike/200",
      timestamp: DateTime.now().subtract(const
        Duration(minutes: 5)),
      isIncoming: true,
      isVideo: false,
    ),
    Call(
      name: "Sarah Wilson",
      avatarUrl:
        "https://picsum.photos/seed/sarah/200",
      timestamp: DateTime.now().subtract(const
        Duration(minutes: 15)),
      isIncoming: false,
      isVideo: true,
    ),
    Call(

```

```

      name: "Alex Thompson",
      avatarUrl:
        "https://picsum.photos/seed/alex/200",
      timestamp: DateTime.now().subtract(const
        Duration(hours: 2)),
      isIncoming: true,
      isMissed: true,
      isVideo: false,
    ),
    Call(
      name: "Emma Davis",
      avatarUrl:
        "https://picsum.photos/seed/emma/200",
      timestamp: DateTime.now().subtract(const
        Duration(hours: 3)),
      isIncoming: false,
      isVideo: true,
    ),
    Call(
      name: "James Miller",
      avatarUrl:
        "https://picsum.photos/seed/james/200",
      timestamp: DateTime.now().subtract(const
        Duration(hours: 5)),
      isIncoming: true,
      isMissed: false,
      isVideo: true,
    ),
    Call(
      name: "Lisa Brown",
      avatarUrl:
        "https://picsum.photos/seed/lisa/200",
      timestamp: DateTime.now().subtract(const
        Duration(hours: 6)),

```

```

        isIncoming: false,

        isMissed: true,

        isVideo: false,
    ),
    Call(
        name: "David Clark",

        avatarUrl:
        "https://picsum.photos/seed/david/200",

        timestamp: DateTime.now().subtract(const
        Duration(hours: 8)),

        isIncoming: true,

        isVideo: false,
    ),
    Call(
        name: "Rachel Green",

        avatarUrl:
        "https://picsum.photos/seed/rachel/200",

        timestamp: DateTime.now().subtract(const
        Duration(days: 1)),

        isIncoming: false,

        isVideo: true,
    ),
    Call(
        name: "Tom Wilson",

        avatarUrl:
        "https://picsum.photos/seed/tom/200",

        timestamp: DateTime.now().subtract(const
        Duration(days: 1, hours: 3)),

        isIncoming: true,

        isMissed: true,

        isVideo: false,
    ),
    Call(

```

```

        name: "Emily White",

        avatarUrl:
        "https://picsum.photos/seed/emily/200",

        timestamp: DateTime.now().subtract(const
        Duration(days: 1, hours: 6)),

        isIncoming: false,

        isVideo: true,
    ),
];

```

```

void _showOutgoingCallDialog(
    BuildContext context, String name, bool
    isVideo) {
    showDialog(
        context: context,
        barrierDismissible: false,
        builder: (context) => Align(
            alignment: Alignment.topCenter,
            child: OutgoingCallDialog(name: name,
            isVideo: isVideo),
        ),
    );
}

```

```

String _getTimeString(DateTime timestamp) {
    final now = DateTime.now();

    final difference =
    now.difference(timestamp);

    if (difference.inDays > 0) {
        return "Yesterday,
        ${timestamp.hour.toString().padLeft(2,
        '0')}:${timestamp.minute.toString().padLeft(2,
        '0')}";
    }
}

```

```

    } else if (difference.inHours > 0) {
      return "${difference.inHours}h ago";
    } else {
      return
"${timestamp.hour.toString().padLeft(2,
'0')}:${timestamp.minute.toString().padLeft(2,
'0')}";
    }
  }
}

```

```

@override
Widget build(BuildContext context) {
  return Scaffold(
    backgroundColor: const
Color(0xFF1F2C34),
    body: SafeArea(
      child: Column(
        children: [
          Expanded(
            child: ListView.builder(
              itemCount: calls.length,
              itemBuilder: (context, index) {
                final call = calls[index];
                return ListTile(
                  leading: CircleAvatar(
                    radius: 25,
                    backgroundImage:
NetworkImage(call.avatarUrl),
                    backgroundColor:
Colors.grey[700],
                  ),
                  title: Text(
                    call.name,

```

```

style: const TextStyle(
  color: Colors.white,
  fontWeight: FontWeight.w500,
),
),
subtitle: Row(
  children: [
    Icon(
      call.isIncoming
        ? Icons.call_received
        : Icons.call_made,
      size: 15,
      color: call.isMissed ? Colors.red
: Colors.green,
    ),
    const SizedBox(width: 4),
    Text(
      _getTimeString(call.timestamp),
      style: TextStyle(
        color: Colors.grey[400],
        fontSize: 13,
      ),
    ),
  ],
),
trailing: IconButton(
  icon: Icon(
    call.isVideo ? Icons.videocam :
Icons.call,
    color: const Color(0xFF00A884),
  ),
  onPressed: () {

```

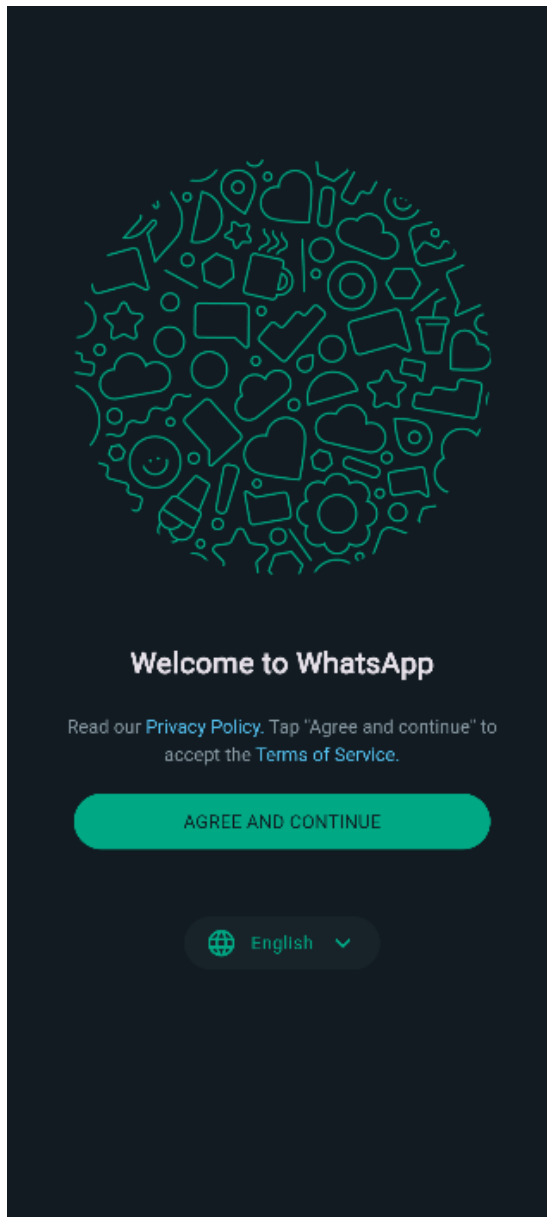


```

        color: Colors.white,
      },
      fontSize: 16,
    },
    fontWeight: FontWeight.bold,
  ),
),
Text(
  "Calling${isVideo ? ' video' : ''}...",
  style: TextStyle(
    color: Colors.grey[400],
    fontSize: 14,
  ),
),
],
),
),
IconButton(
  icon: const Icon(Icons.close, color:
Colors.red),
  onPressed: () =>
Navigator.of(context).pop(),
),
IconButton(
  icon: Icon(
    isVideo ? Icons.videocam : Icons.call,
    color: Colors.green,
  ),
  onPressed: () =>
Navigator.of(context).pop(),
),
],
),
);

```

Output:



Verify your number

You've tried to register before requesting an SMS or Call with your code. [Wrong number?](#)

Enter 6-digit code

 Resend SMS

 Call Me

 Privacy & Terms

WhatsApp

CHATS

STATUS

CALLS

 **Dev Stack** 01:47
✓ Hi Everyone 🧑

 **Kishor** 00:51
✓ Hi Kishor

 **Dev Server Chat** 23:51
✓ Alice: The new API is ready for testing

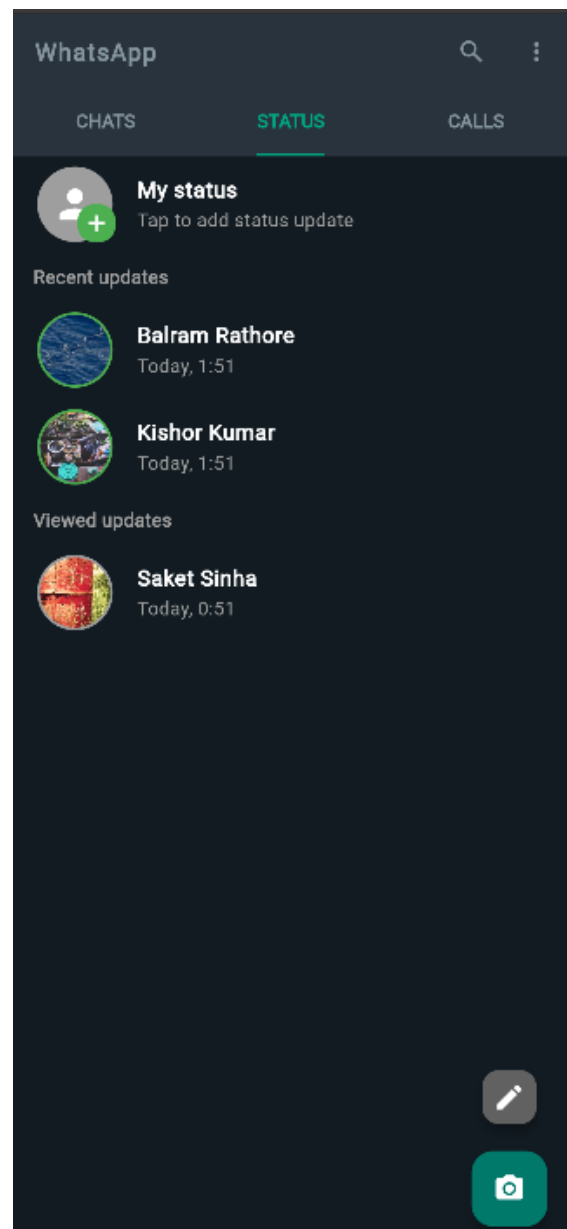
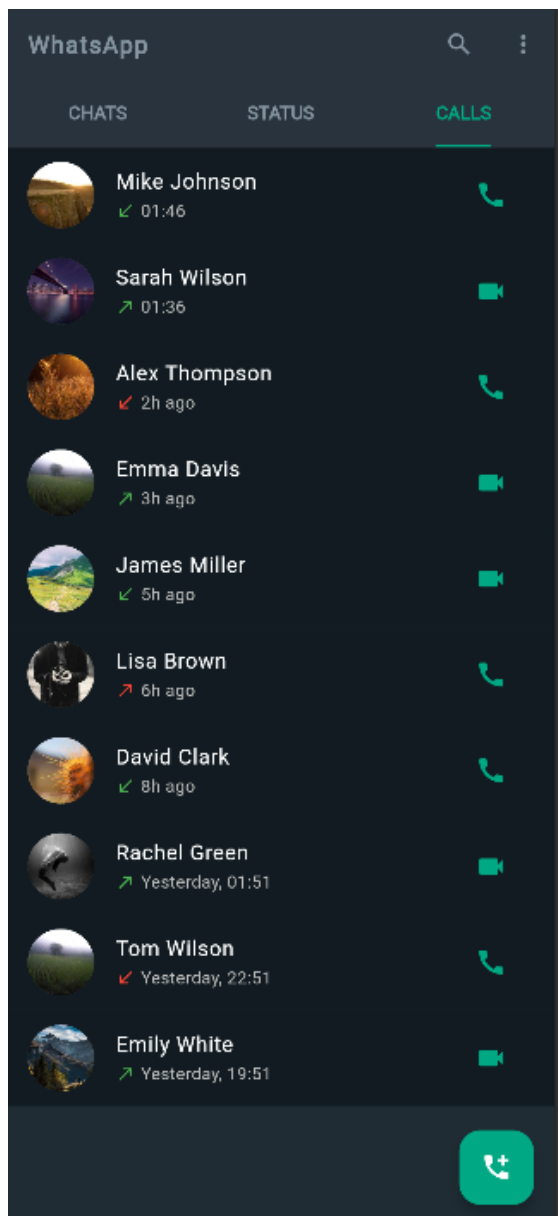
 **Collins** 21:51
✓ Hey Dev Stack! How's the project going?

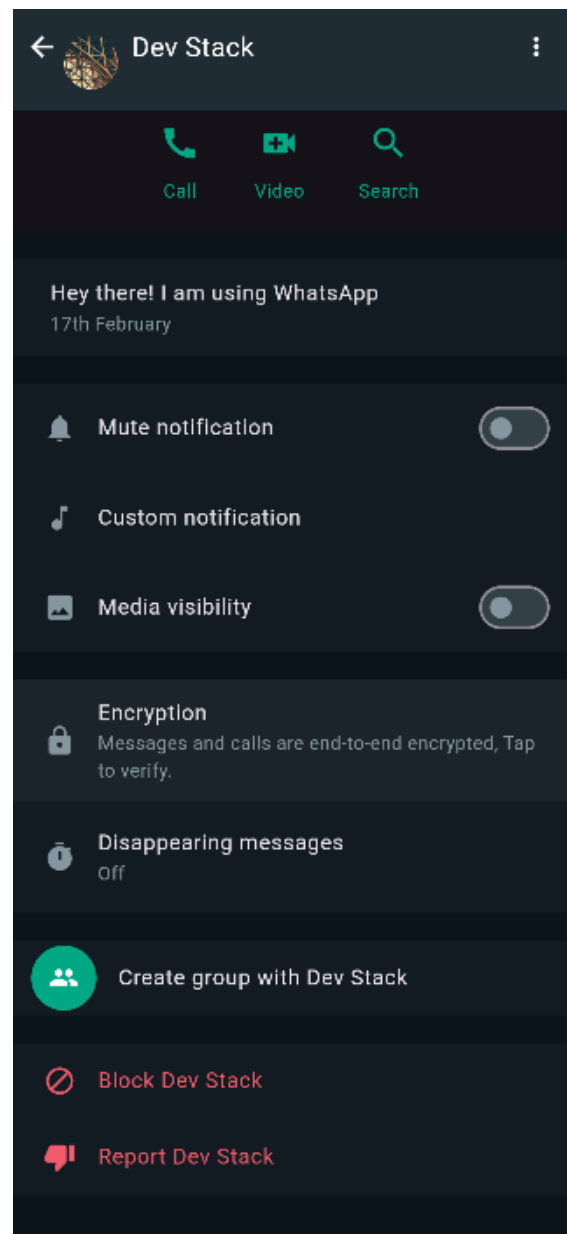
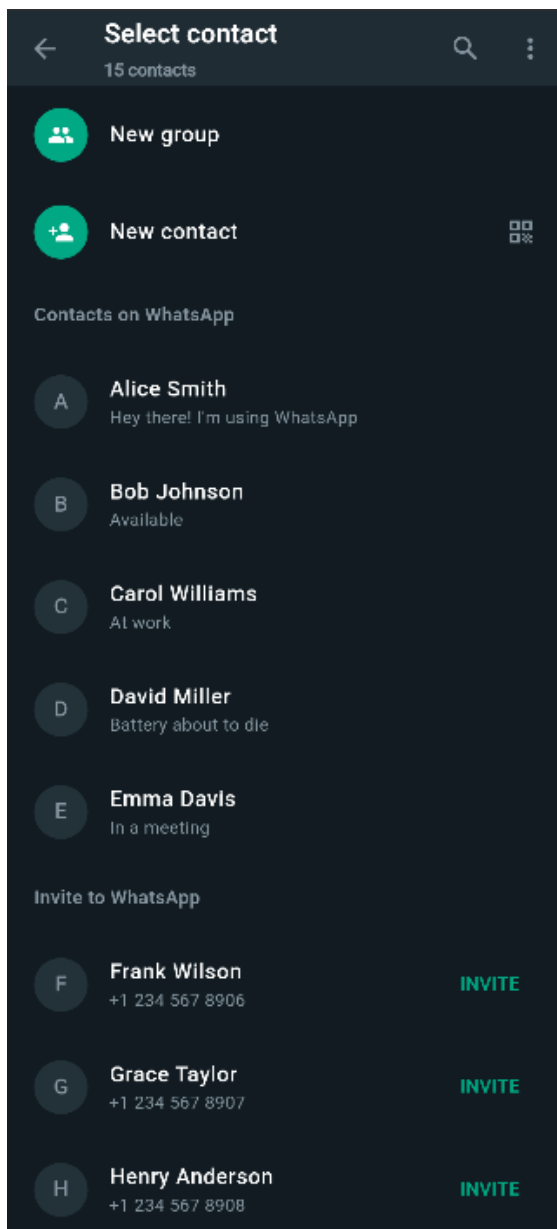
 **Friends** 20:51
✓ John: Anyone up for coffee? ☕

 **Flutter Community** 15:51
✓ New flutter 3.0 features announced! 🚀

 **Balram Rathore** 15:51
✓ happy coding 🔥🔥🔥







MAD & PWA Lab

Journal

Experiment No.	03
Experiment Title.	To include icons, images, fonts in Flutter app
Roll No.	24
Name	Niraj Kothawade
Class	D15A/D15B
Subject	MAD & PWA Lab
Lab Outcome	LO2: Design and Develop interactive Flutter App by using widgets, layouts, gestures and animation
Grade:	

MAD & PWA LAB

Experiment 3

Name: Niraj Kothawade

Class: D15A

Roll no:24

Aim: To include icons, [images](#), fonts in Flutter app.

Code:

```
import 'package:flutter/material.dart';
import 'package:google_fonts/google_fonts.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'WhatsApp UI Elements',
      theme: ThemeData.dark().copyWith(
        primaryColor: const Color(0xFF075E54), // WhatsApp primary green
        scaffoldBackgroundColor:
          const Color(0xFF121B22), // WhatsApp dark mode background
        appBarTheme: const AppBarTheme(
          backgroundColor: Color(0xFF1F2C34), // WhatsApp dark mode app bar
          elevation: 0,
```

```

    ),
    ),
    home: const WhatsAppUIPage(),
  );
}
}

```

```

class WhatsAppUIPage extends StatelessWidget {
  const WhatsAppUIPage({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text(
          'WhatsApp',
          style: GoogleFonts.heebo(
            // WhatsApp uses Heebo font
            fontWeight: FontWeight.w600,
            fontSize: 20,
            color: const Color(0xFF8696A0), // WhatsApp dark mode text color
          ),
        ),
        actions: [
          IconButton(
            icon: const Icon(Icons.search, color: Color(0xFF8696A0)),
            onPressed: () {},
          ),
          IconButton(

```

```
        icon: const Icon(Icons.more_vert, color: Color(0xFF8696A0)),
        onPressed: () {},
      ),
    ],
  ),
  body: Padding(
    padding: const EdgeInsets.all(16.0),
    child: Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        Text(
          'Main Navigation Icons:',
          style: GoogleFonts.heebo(
            fontSize: 16,
            fontWeight: FontWeight.w600,
            color: const Color(0xFF8696A0),
          ),
        ),
        const SizedBox(height: 16),
        Row(
          mainAxisAlignment: MainAxisAlignment.spaceEvenly,
          children: const [
            Icon(Icons.camera_alt, // Camera icon
              size: 24,
              color: Color(0xFF8696A0)),
            Icon(Icons.chat, // Chats icon
              size: 24,
              color: Color(0xFF00A884)), // WhatsApp accent green
            Icon(Icons.group, // Status icon
```

```

        size: 24,
        color: Color(0xFF8696A0)),
    Icon(Icons.call, // Calls icon
        size: 24,
        color: Color(0xFF8696A0)),
    ],
),
const SizedBox(height: 32),

Text(
    'Chat Actions:',
    style: GoogleFonts.heebo(
        fontSize: 16,
        fontWeight: FontWeight.w600,
        color: const Color(0xFF8696A0),
    ),
),
const SizedBox(height: 16),
Row(
    mainAxisAlignment: MainAxisAlignment.spaceEvenly,
    children: const [
        Icon(Icons.mic, // Voice message
            size: 24,
            color: Color(0xFF00A884)),
        Icon(Icons.attach_file, // Attachments
            size: 24,
            color: Color(0xFF8696A0)),
        Icon(Icons.photo_camera, // Camera in chat
            size: 24,
```

```

        color: Color(0xFF8696A0)),
Icon(Icons.emoji_emotions, // Emoji
    size: 24,
    color: Color(0xFF8696A0)),
],
),
const SizedBox(height: 32),

// Profile/Status Image Example
Text(
    'Profile/Status Style:',
    style: GoogleFonts.heebo(
        fontSize: 16,
        fontWeight: FontWeight.w600,
        color: const Color(0xFF8696A0),
    ),
),
const SizedBox(height: 16),
Row(
    children: [
        Container(
            width: 50,
            height: 50,
            decoration: BoxDecoration(
                shape: BoxShape.circle,
                border: Border.all(
                    color: const Color(0xFF00A884),
                    width: 2,
                ),
            ),

```



```
        color: const Color(
          0xFF233239), // WhatsApp dark mode secondary background
      ),
      child: const Icon(
        Icons.person,
        color: Color(0xFF8696A0),
        size: 30,
      ),
    ),
    const SizedBox(width: 16),
    Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        Text(
          'Contact Name',
          style: GoogleFonts.heebo(
            fontSize: 16,
            fontWeight: FontWeight.w600,
            color: Colors.white,
          ),
        ),
        Text(
          'Last seen today at 12:00 PM',
          style: GoogleFonts.heebo(
            fontSize: 13,
            color: const Color(0xFF8696A0),
          ),
        ),
      ],
    ),
  ],
```

```

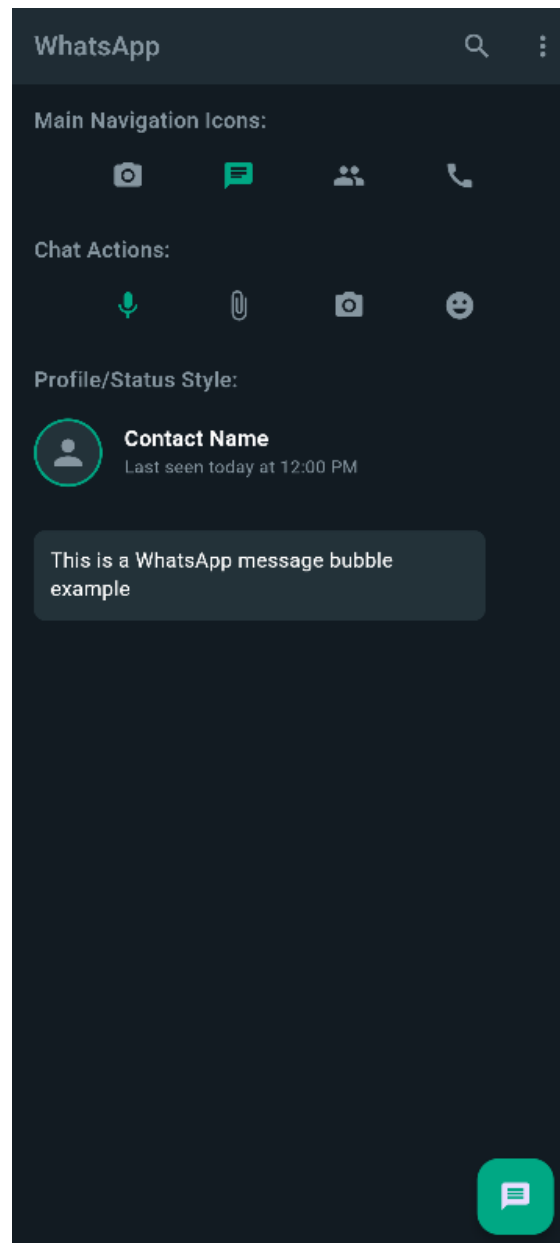
    ),
  ],
),
const SizedBox(height: 32),

// Message Bubble Example
Container(
  margin: const EdgeInsets.only(right: 50),
  padding: const EdgeInsets.all(12),
  decoration: BoxDecoration(
    color: const Color(
      0xFF233239), // WhatsApp dark mode message bubble
    borderRadius: BorderRadius.circular(8),
  ),
  child: Text(
    'This is a WhatsApp message bubble example',
    style: GoogleFonts.heebo(
      fontSize: 15,
      color: Colors.white,
    ),
  ),
),
],
),
),
floatingActionButton: FloatingActionButton(
  backgroundColor: const Color(0xFF00A884),
  onPressed: () {},
  child: const Icon(Icons.message),

```

```
    ),  
    );  
}  
}
```

Output:



MAD & PWA Lab

Journal

Experiment No.	04
Experiment Title.	To create an interactive Form using form widget
Roll No.	24
Name	Niraj Kothawade
Class	D15A/D15B
Subject	MAD & PWA Lab
Lab Outcome	LO2: Design and Develop interactive Flutter App by using widgets, layouts, gestures and animation
Grade:	

MAD & PWA LAB

Experiment 4

Name: Niraj Kothawade

Class: D15A

Roll no: 24

Aim: To create an interactive Form using form widget.

Code:

Login page:

```
import
'package:country_picker/country_picker.d
art';

import 'package:flutter/material.dart';

import
'package:flutter_riverpod/flutter_riverpod
.dart';

import
'package:whatsapp_messenger/common/
extension/custom_theme_extension.dart';

import
'package:whatsapp_messenger/common/
helper/show_alert_dialog.dart';

import
'package:whatsapp_messenger/common/
widgets/custom_elevated_button.dart';

import
'package:whatsapp_messenger/feature/a
uth/controller/auth_controller.dart';

import
'package:whatsapp_messenger/feature/a
uth/widgets/custom_text_field.dart';

import '../../common/utils/coloors.dart';

import
'../../common/widgets/custom_icon_bu
tton.dart';

class LoginPage extends
ConsumerStatefulWidget {

  const LoginPage({super.key});

  @override

  ConsumerState<LoginPage> createState()
=> _LoginPageState();
}

class _LoginPageState extends
ConsumerState<LoginPage> {

  late TextEditingController
countryNameController;

  late TextEditingController
countryCodeController;

  late TextEditingController
phoneNumberController;

  sendCodeToPhone() {

    final phoneNumber =
phoneNumberController.text;

    final countryName =
countryNameController.text;
```

```

        final countryCode =
countryCodeController.text;
    );
}

```

```

    if (phoneNumber.isEmpty) {
        return showAlertDialog(
            context: context,
            message: "Please enter your phone
number",
        );
    } else if (phoneNumber.length < 9) {
        return showAlertDialog(
            context: context,
            message:
                'The phone number you entered is
too short for the country:
$countryName\n\nInclude your area code
if you haven\'t',
        );
    } else if (phoneNumber.length > 10) {
        return showAlertDialog(
            context: context,
            message:
                "The phone number you entered is
too long for the country: $countryName",
        );
    }
}

```

```

        ref.read(authControllerProvider).sendS
msCode(
            context: context,
            phoneNumber:
                "+$countryCode$phoneNumber",

```

```

        showCountryPickerBottomSheet() {
            showCountryPicker(
                context: context,
                showPhoneCode: true,
                favorite: ['ET'],
                countryListTheme:
CountryListThemeData(
                    bottomSheetHeight: 600,
                    backgroundColor:
Theme.of(context).colorScheme.surface,
                    flagSize: 22,
                    borderRadius:
BorderRadius.circular(20),
                    textStyle: TextStyle(color:
context.theme.greyColor),
                    inputDecoration: InputDecoration(
                        labelStyle: TextStyle(color:
context.theme.greyColor),
                        prefixIcon: const Icon(
                            Icons.language,
                            color: Colors.greenDark,
                        ),
                        hintText: 'Search country by code or
name',
                        enabledBorder:
UnderlineInputBorder(
                            borderSide: BorderSide(
                                color:
context.theme.greyColor!.withOpacity(.2),

```

```

    ),
    ),
    focusedBorder: const
UnderlineInputBorder(
    borderSide: BorderSide(
    color: Colors.greenDark,
    ),
    ),
    ),
    ),
    ),
    onSelect: (country) {
    countryNameController.text =
country.name;
    countryCodeController.text =
country.phoneCode;
    },
    );
}

@override
void initState() {
    countryNameController =
TextEditingController(text: 'Ethiopia');
    countryCodeController =
TextEditingController(text: '251');
    phoneNumberController =
TextEditingController();
    super.initState();
}

@override

```

```

void dispose() {
    countryNameController.dispose();
    countryCodeController.dispose();
    phoneNumberController.dispose();
    super.dispose();
}

@override
Widget build(BuildContext context) {
    return Scaffold(
    appBar: AppBar(
    backgroundColor:
Theme.of(context).scaffoldBackgroundCol
or,
    elevation: 0,
    title: Text(
    'Enter your phone number',
    style: TextStyle(
    color:
context.theme.authAppBarTextColor,
    ),
    ),
    centerTitle: true,
    actions: [
    CustomIconButton(
    onPressed: () {},
    icon: Icons.more_vert,
    ),
    ],
    ),

```

```

body: Column(
  children: [
    Padding(
      padding: const
EdgeInsets.symmetric(horizontal: 20,
vertical: 10),
      child: RichText(
        textAlign: TextAlign.center,
        text: TextSpan(
          text: 'WhatsApp will need to
verify your number. ',
          style: TextStyle(
            color: context.theme.greyColor,
            height: 1.5,
          ),
          children: [
            TextSpan(
              text: "What's my number?",
              style: TextStyle(
                color:
context.theme.blueColor,
              ),
            ),
          ],
        ),
      ),
      const SizedBox(height: 10),
      Padding(
        padding: const
EdgeInsets.symmetric(horizontal: 50),

```

```

child: CustomTextField(
  onTap:
showCountryPickerBottomSheet,
  controller:
countryNameController,
  readOnly: true,
  suffixIcon: const Icon(
    Icons.arrow_drop_down,
    color: Colors.greenDark,
    size: 22,
  ),
),
),
const SizedBox(height: 10),
Padding(
  padding: const
EdgeInsets.symmetric(horizontal: 50),
  child: Row(
    children: [
      SizedBox(
        width: 70,
        child: CustomTextField(
          onTap:
showCountryPickerBottomSheet,
          controller:
countryCodeController,
          prefixText: '+',
          readOnly: true,
        ),
      ),
      const SizedBox(width: 10),

```



```

Expanded(
  child: CustomTextField(
    controller:
phoneNumberController,
    hintText: 'phone number',
    textAlign: TextAlign.left,
    keyboardType:
TextInputType.number,
  ),
),
],
),
),
const SizedBox(height: 20),
Text(
  'Carrier charges may apply',
  style: TextStyle(
    color: context.theme.greyColor,
  ),
),
],
),
floatingActionButtonLocation:
FloatingActionButtonLocation.centerFloat,
floatingActionButton:
CustomElevatedButton(
  onPressed: sendCodeToPhone,
  text: 'NEXT',
  buttonWidth: 90,
),
);
}
}

```

Verification page:

```

import 'package:flutter/material.dart';
import
'package:flutter_riverpod/flutter_riverpod
.dart';
import
'package:whatsapp_messenger/common/
extension/custom_theme_extension.dart';
import
'package:whatsapp_messenger/feature/a
uth/controller/auth_controller.dart';
import
'package:whatsapp_messenger/feature/a
uth/widgets/custom_text_field.dart';

class VerificationPage extends
ConsumerWidget {
  const VerificationPage({
    super.key,
    required this.smsCodeId,
    required this.phoneNumber,
  });

  final String smsCodeId;
  final String phoneNumber;

```

```

Future<void> verifySmsCode(
    BuildContext context,
    WidgetRef ref,
    String smsCode,
) async {
    await
    ref.read(authControllerProvider).verifySms
    Code(
        context: context,
        smsCodeId: smsCodeId,
        smsCode: smsCode,
    );
}

```

```

@override
Widget build(BuildContext context,
WidgetRef ref) {
    return Scaffold(
        appBar: AppBar(
            backgroundColor:
            Theme.of(context).scaffoldBackgroundCol
            or,
            elevation: 0,
            centerTitle: true,
            title: Text(
                'Verify your number',
                style: TextStyle(
                    color:
                    context.theme.authAppBarTextColor,
                ),
            ),
        ),
    ),

```

```

actions: [
    IconButton(
        onPressed: () {},
        icon: const Icon(Icons.more_vert),
    ),
],
),
body: SingleChildScrollView(
    padding: const
    EdgeInsets.symmetric(horizontal: 20),
    child: Column(
        children: [
            Padding(
                padding: const
                EdgeInsets.symmetric(horizontal: 10),
                child: RichText(
                    textAlign: TextAlign.center,
                    text: TextSpan(
                        style: TextStyle(color:
                        context.theme.greyColor),
                        children: [
                            const TextSpan(
                                text:
                                    "You've tried to register
                                    before requesting an SMS or Call with your
                                    code.",
                                ),
                            TextSpan(
                                text: " Wrong number?",
                                style: TextStyle(

```

```

        color:
context.theme.blueColor,
    ),
    ),
  ],
),
),
const SizedBox(height: 20),
Container(
  padding: const
EdgeInsets.symmetric(horizontal: 80),
  child: CustomTextField(
    hintText: "--- ---",
    fontSize: 30,
    autoFocus: true,
    keyboardType:
TextInputType.number,
    onChanged: (value) async {
      if (value.length == 6) {
        await verifySmsCode(context,
ref, value);
      }
    },
  ),
),
const SizedBox(height: 20),
Text(
  'Enter 6-digit code',
  style: TextStyle(color:
context.theme.greyColor),

```

```

),
const SizedBox(height: 30),
Row(
  children: [
    Icon(Icons.message, color:
context.theme.greyColor),
    const SizedBox(width: 20),
    Text(
      'Resend SMS',
      style: TextStyle(
        color: context.theme.greyColor,
      ),
    ),
  ],
),
const SizedBox(height: 10),
Divider(
  color:
context.theme.greyColor!.withOpacity(.2),
),
const SizedBox(height: 10),
Row(
  children: [
    Icon(Icons.phone, color:
context.theme.greyColor),
    const SizedBox(width: 20),
    Text(
      'Call Me',
      style: TextStyle(
        color: context.theme.greyColor,

```

```

        ),
      ),
    ],
  ),
],
),

```

```

        ),
      ),
    );
  }
}

```

User_Info page:

```

import 'dart:io';
import 'dart:typed_data';

import 'package:flutter/material.dart';
import
'package:flutter_riverpod/flutter_riverpod
.dart';
import
'package:image_picker/image_picker.dart'
;
import
'package:whatsapp_messenger/common/
extension/custom_theme_extension.dart';
import
'package:whatsapp_messenger/common/
helper/show_alert_dialog.dart';
import
'package:whatsapp_messenger/common/
utils/colors.dart';
import
'package:whatsapp_messenger/common/
widgets/custom_elevated_button.dart';
import
'package:whatsapp_messenger/common/
widgets/custom_icon_button.dart';
import
'package:whatsapp_messenger/common/
widgets/short_h_bar.dart';

```

```

import
'package:whatsapp_messenger/feature/a
uth/controller/auth_controller.dart';

import
'package:whatsapp_messenger/feature/a
uth/pages/image_picker_page.dart';
import
'package:whatsapp_messenger/feature/a
uth/widgets/custom_text_field.dart';

class UserInfoPage extends
ConsumerStatefulWidget {

  const UserInfoPage({super.key,
this.profileImageUrl});

  final String? profileImageUrl;

  @override
  ConsumerState<UserInfoPage>
createState() => _UserInfoPageState();
}

class _UserInfoPageState extends
ConsumerState<UserInfoPage> {

  File? imageCamera;
  Uint8List? imageGallery;

```

```

late TextEditingController
usernameController;

@override
void initState() {
  super.initState();
  usernameController =
TextEditingController();
}

@override
void dispose() {
  usernameController.dispose();
  super.dispose();
}

Future<void> saveUserDataToFirebase()
async {
  String username =
usernameController.text.trim();

  if (username.isEmpty) {
    showAlertDialog(context: context,
message: 'Please provide a username');
    return;
  }

  if (username.length < 3 ||
username.length > 20) {
    showAlertDialog(
      context: context, message:
'Username length should be between 3-
20');

```

```

    return;
  }

  await
ref.read(authControllerProvider).saveUserI
nfoToFirestore(
    username: username,
    profileImage:
      imageCamera ?? imageGallery ??
widget.profileImageUrl ?? "",
    context: context,
  );
}

void imagePickerTypeBottomSheet() {
  showModalBottomSheet(
    context: context,
    builder: (context) {
      return Padding(
        padding: const
EdgeInsets.symmetric(vertical: 10),
        child: Column(
          mainAxisAlignment: MainAxisAlignment.min,
          children: [
            const ShortHBar(),
            Padding(
              padding: const
EdgeInsets.symmetric(horizontal: 20),
              child: Row(
                children: [
                  const Text(

```

```

        'Profile photo',
        style:
            TextStyle(fontSize: 20,
fontWeight: FontWeight.w500),
    ),
    const Spacer(),
    CustomIconButton(
        onPressed: () =>
Navigator.pop(context),
        icon: Icons.close,
    ),
],
),
),
),
    Divider(color:
context.theme.greyColor!.withOpacity(.3))
,
    Row(
        mainAxisAlignment:
MainAxisAlignment.spaceEvenly,
        children: [
            imagePickerIcon(
                onTap: pickImageFromCamera,
                icon:
Icons.camera_alt_rounded,
                text: 'Camera',
            ),
            imagePickerIcon(
                onTap: pickImageFromGallery,
                icon:
Icons.photo_camera_back_rounded,
                text: 'Gallery',

```

```

        ),
    ],
    ),
    ),
    );
}

Future<void> pickImageFromCamera()
async {
    Navigator.pop(context);

    final image = await
ImagePicker().pickImage(source:
ImageSource.camera);

    if (image != null) {
        setState(() {
            imageCamera = File(image.path);
            imageGallery = null;
        });
    }
}

Future<void> pickImageFromGallery()
async {
    Navigator.pop(context);

    final image = await Navigator.push(
        context,
        MaterialPageRoute(builder: (context)
=> const ImagePickerPage()),

```

```

);
if (image != null) {
  setState(() {
    imageGallery = image;
    imageCamera = null;
  });
}
}

Widget imagePickerIcon({
  required VoidCallback onTap,
  required IconData icon,
  required String text,
}) {
  return Column(
    children: [
      CustomIconButton(
        onPressed: onTap,
        icon: icon,
        iconColor: Colors.greenDark,
        minWidth: 50,
        border: Border.all(color:
context.theme.greyColor!.withOpacity(.2))
        ,
      ),
      const SizedBox(height: 5),
      Text(text, style: TextStyle(color:
context.theme.greyColor)),
    ],
  );
}

}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      backgroundColor:
Theme.of(context).colorScheme.surface,
      elevation: 0,
      title: Text(
        'Profile info',
        style: TextStyle(color:
context.theme.authAppBarTextColor),
      ),
      centerTitle: true,
    ),
    body: SingleChildScrollView(
      padding: const
EdgeInsets.symmetric(horizontal: 20),
      child: Column(
        children: [
          Text(
            'Please provide your name and an
optional profile photo',
            textAlign: TextAlign.center,
            style: TextStyle(color:
context.theme.greyColor),
          ),
          const SizedBox(height: 40),
          GestureDetector(

```

```

      onTap:
imagePickerTypeBottomSheet,
      child: Container(
        padding: const EdgeInsets.all(26),
        decoration: BoxDecoration(
          shape: BoxShape.circle,
          color:
context.theme.photoIconBgColor,
          border: Border.all(
            color: (imageCamera == null
&&
              imageGallery == null &&
              widget.profileImageUrl ==
null)
              ? Colors.transparent
              :
context.theme.greyColor!.withOpacity(.4),
          ),
          image: (imageCamera != null ||
            imageGallery != null ||
            widget.profileImageUrl !=
null)
              ? DecorationImage(
                fit: BoxFit.cover,
                image: imageGallery != null
                  ?
MemoryImage(imageGallery!)
                  : widget.profileImageUrl
!= null
                  ?
NetworkImage(widget.profileImageUrl!)

```

```

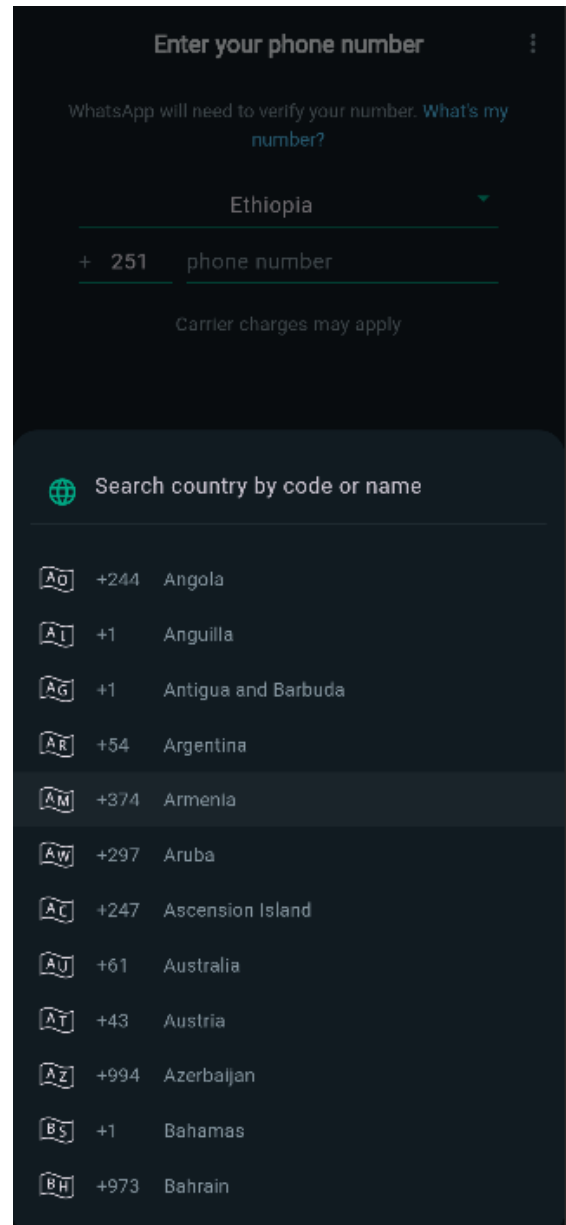
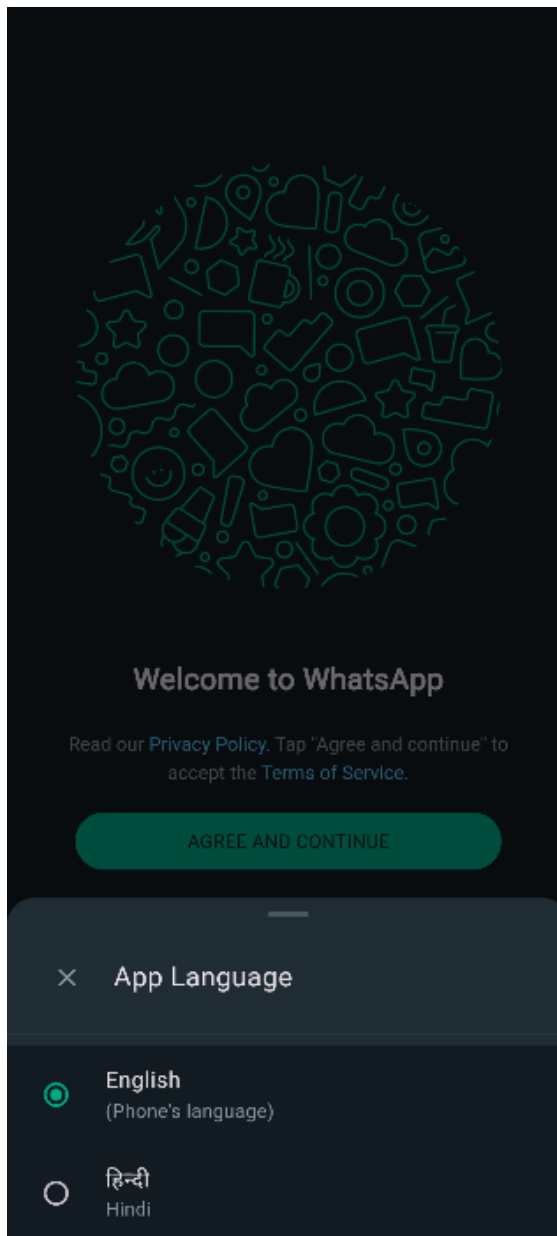
              :
FileImage(imageCamera!) as
ImageProvider,
            )
            : null,
          ),
          child: Icon(
            Icons.add_a_photo_rounded,
            size: 48,
            color: (imageCamera == null &&
              imageGallery == null &&
              widget.profileImageUrl ==
null)
              ?
context.theme.photoIconColor
              : Colors.transparent,
          ),
          ),
          ),
          const SizedBox(height: 40),
          CustomTextField(
            controller: usernameController,
            hintText: 'Type your name here',
            autoFocus: true),
        ],
      ),
      ),
      floatingActionButton:
CustomElevatedButton(
        onPressed: saveUserDataToFirebase,
        text: 'NEXT', buttonWidth: 90),
      );

```



```
}  
}
```

Output:



Enter your phone number

WhatsApp will need to verify your number. [What's my number?](#)

India

+ 91 9356229062

Carrier charges may apply

NEXT

Verify your number

You've tried to register before requesting an SMS or Call with your code. [Wrong number?](#)

Enter 6-digit code

Resend SMS

Call Me

 [Privacy](#) · [Terms](#)

Profile Info

Please provide your name and an optional profile photo



Niraj



MAD & PWA Lab

Journal

Experiment No.	05
Experiment Title.	To apply navigation, routing and gestures in Flutter App
Roll No.	24
Name	Niraj Kothawade
Class	D15A/D15B
Subject	MAD & PWA Lab
Lab Outcome	LO2: Design and Develop interactive Flutter App by using widgets, layouts, gestures and animation
Grade:	

MAD & PWA LAB

Experiment 5

Name: Niraj Kothawade

Class: D15A

Roll no: 24

Aim: To apply navigation, routing and gestures in Flutter App

Theory:

1. Navigation in Flutter

Navigation allows users to move between screens in an app. It is managed using the Navigator and a stack-based system. The most common navigation actions include:

- Push Navigation: Moves to a new screen.
- Pop Navigation: Goes back to the previous screen.
- Named Routes: Uses predefined names for navigation instead of directly creating new screen instances.

2. Routing in Flutter

Routing defines the path and logic for screen transitions. There are two main types:

- Static Routing: Predefined routes set in MaterialApp.
- Dynamic Routing: Routes created dynamically using onGenerateRoute, allowing flexible screen navigation.

3. Gestures in Flutter

Gestures enable user interactions such as tapping, swiping, and dragging. Flutter provides:

- GestureDetector: Detects custom gestures like tap, double-tap, and long press.
- InkWell: Adds ripple effects for a better touch response.

Code: `elcome/widgets/privacy_and_terms.dart';`

Landing page:

```
import 'package:flutter/material.dart';

import
'package:whatsapp_messenger/common/
routes/routes.dart';

import
'package:whatsapp_messenger/common/
utils/coloors.dart';

import
'package:whatsapp_messenger/common/
widgets/custom_elevated_button.dart';

import
'package:whatsapp_messenger/feature/w
elcome/widgets/language_button.dart';

import
'package:whatsapp_messenger/feature/w

class WelcomePage extends
StatelessWidget {

  const WelcomePage({super.key});

  navigateToLoginPage(context) {
    Navigator.of(context)
      .pushNamedAndRemoveUntil(Routes.
login, (route) => false);
  }

  @override
  Widget build(BuildContext context) {
```

```
return Scaffold(  
  body: Column(  
    children: [  
      Expanded(  
        child: Align(  
          alignment:  
Alignment.bottomCenter,  
        child: Padding(  
          padding:  
            const  
EdgeInsets.symmetric(horizontal: 50,  
vertical: 10),  
          child:  
Image.asset('assets/images/circle.png',  
            color: Colors.greenDark),  
        ),  
      ),  
      const SizedBox(height:  
40), Expanded(  
        child: Column(  
          children: [  
            Expanded(  
              child: Image.asset('assets/images/circle.png',  
                color: Colors.greenDark),  
            ),  
            const SizedBox(height: 40),  
            Expanded(  
              child: Image.asset('assets/images/circle.png',  
                color: Colors.greenDark),  
            ),  
          ],  
        ),  
      ),  
    ],  
  ),  
);
```

```

children: [ const Text(
    'Welcome to WhatsApp',
    style: TextStyle(
      fontSize: 22,
      fontWeight: FontWeight.bold,
    ),
  ),
  const PrivacyAndTerms(),
  CustomElevatedButton(
    onPressed: () =>
navigateToLoginPage(context),
    text: 'AGREE AND CONTINUE',
  ),
  const SizedBox(height: 50),
  LanguageButton(),
],
),
),
],
),
);
}
}

```

Authentication page:

```

import 'package:flutter/material.dart';

import
'package:flutter_riverpod/flutter_riverpod
.dart';

```

```

import
'package:whatsapp_messenger/common/
extension/custom_theme_extension.dart';

import
'package:whatsapp_messenger/feature/a
uth/controller/auth_controller.dart';

import
'package:whatsapp_messenger/feature/a
uth/widgets/custom_text_field.dart';

class VerificationPage extends
ConsumerWidget {
  const VerificationPage({
    super.key,
    required this.smsCodeId,
    required this.phoneNumber,
  });

  final String smsCodeId;
  final String phoneNumber;

  Future<void> verifySmsCode(
    BuildContext context,
    WidgetRef ref,
    String smsCode,
  ) async {
    await
ref.read(authControllerProvider).verifySms
Code(
    context: context,
    smsCodeId: smsCodeId,
    smsCode: smsCode,

```

```

    );
}

@override
Widget build(BuildContext context,
WidgetRef ref) {
  return Scaffold(
    appBar: AppBar(
      backgroundColor:
Theme.of(context).scaffoldBackgroundCol
or,
      elevation: 0,
      centerTitle: true,
      title: Text(
        'Verify your number',
        style: TextStyle(
          color:
context.theme.authAppBarTextColor,
        ),
      ),
      actions: [
        IconButton(
          onPressed: () {},
          icon: const Icon(Icons.more_vert),
        ),
      ],
    ),
    body: SingleChildScrollView(
      padding: const
EdgeInsets.symmetric(horizontal: 20),
      child: Column(

```

```

        children: [
          Padding(
            padding: const
EdgeInsets.symmetric(horizontal: 10),
            child: RichText(
              textAlign: TextAlign.center,
              text: TextSpan(
                style: TextStyle(color:
context.theme.greyColor),
                children: [
                  const TextSpan(
                    text:
                      "You've tried to register
before requesting an SMS or Call with your
code.",
                  ),
                  TextSpan(
                    text: " Wrong number?",
                    style: TextStyle(
                      color:
context.theme.blueColor,
                    ),
                  ),
                ],
              ),
            ),
          const SizedBox(height: 20),
          Container(
            padding: const
EdgeInsets.symmetric(horizontal: 80),

```



```

        child: CustomTextField(
            hintText: " ----- ",
            fontSize: 30,
            autoFocus: true,
            keyboardType:
TextInputType.number,
            onChanged: (value) async {
                if (value.length == 6) {
                    await verifySmsCode(context,
ref, value);
                }
            },
        ),
    ),
    const SizedBox(height: 20),
    Text(
        'Enter 6-digit code',
        style: TextStyle(color:
context.theme.greyColor),
    ),
    const SizedBox(height: 30),
    Row(
        children: [
            Icon(Icons.message, color:
context.theme.greyColor),
            const SizedBox(width: 20),
            Text(
                'Resend SMS',
                style: TextStyle(
                    color: context.theme.greyColor,
                ),
            ),
        ],
    ),
    const SizedBox(height: 10),
    Divider(
        color:
context.theme.greyColor!.withOpacity(.2),
    ),
    const SizedBox(height: 10),
    Row(
        children: [
            Icon(Icons.phone, color:
context.theme.greyColor),
            const SizedBox(width: 20),
            Text(
                'Call Me',
                style: TextStyle(
                    color: context.theme.greyColor,
                ),
            ),
        ],
    ),
    const SizedBox(width: 20),
    Text(
        'Call Me',
        style: TextStyle(
            color: context.theme.greyColor,
        ),
    ),
    ],
),
);
}
}

```

Chats page:

```
import
'package:cached_network_image/cached_network_image.dart';

import 'package:flutter/material.dart';

import
'package:flutter_riverpod/flutter_riverpod.dart';

import 'package:intl/intl.dart';

import
'package:whatsapp_messenger/common/extension/custom_theme_extension.dart';

import
'package:whatsapp_messenger/common/models/last_message_model.dart';

import
'package:whatsapp_messenger/common/models/user_model.dart';

import
'package:whatsapp_messenger/common/routes/routes.dart';

import
'package:whatsapp_messenger/common/utils/colors.dart';

import
'package:whatsapp_messenger/feature/chat/controller/chat_controller.dart';

class ChatHomePage extends ConsumerWidget {

  const ChatHomePage({super.key});

  List<LastMessageModel>
  getDemoChats() {

    return [
```

```
      LastMessageModel(
        username: 'Dev Stack',
        contactId: '1',
        profileImageUrl:
        'https://picsum.photos/seed/1/200',
        lastMessage: 'Hi Everyone 🦄',
        timeSent:
        DateTime.now().subtract(const Duration(minutes: 4)),
      ),
      LastMessageModel(
        username: 'Kishor',
        contactId: '2',
        profileImageUrl:
        'https://picsum.photos/seed/2/200',
        lastMessage: 'Hi Kishor',
        timeSent:
        DateTime.now().subtract(const Duration(hours: 1)),
      ),
      LastMessageModel(
        username: 'Dev Server Chat',
        contactId: '3',
        profileImageUrl:
        'https://picsum.photos/seed/3/200',
        lastMessage: 'Alice: The new API is ready for testing',
        timeSent:
        DateTime.now().subtract(const Duration(hours: 2)),
      ),
      LastMessageModel(
```

```

        username: 'Collins',
        contactId: '4',
        imageUrl:
'https://picsum.photos/seed/4/200',
        lastMessage: 'Hey Dev Stack! How\'s
the project going?',
        timeSent:
DateTime.now().subtract(const
Duration(hours: 4)),
    ),
    LastMessageModel(
        username: 'Friends',
        contactId: '5',
        imageUrl:
'https://picsum.photos/seed/5/200',
        lastMessage: 'John: Anyone up for
coffee? 🐼',
        timeSent:
DateTime.now().subtract(const
Duration(hours: 5)),
    ),
    LastMessageModel(
        username: 'Flutter Community',
        contactId: '6',
        imageUrl:
'https://picsum.photos/seed/6/200',
        lastMessage: 'New flutter 3.0 features
announced! 🌈',
        timeSent:
DateTime.now().subtract(const
Duration(hours: 10)),
    ),
    LastMessageModel(

```

```

        username: 'Balram Rathore',
        contactId: '7',
        imageUrl:
'https://picsum.photos/seed/7/200',
        lastMessage: 'happy coding
🟡 🟡 🟡',
        timeSent:
DateTime.now().subtract(const
Duration(hours: 10)),
    ),
    ];
}

@override
Widget build(BuildContext context,
WidgetRef ref) {
    return Scaffold(
        backgroundColor: const
Color(0xFF121B22),
        body: ListView.builder(
            itemCount: getDemoChats().length,
            shrinkWrap: true,
            itemBuilder: (context, index) {
                final lastMessageData =
getDemoChats()[index];

                return Column(
                    children: [
                        ListTile(
                            onTap: () {
                                Navigator.pushNamed(

```

```

        context,
        Routes.chat,
        arguments: UserModel(
            username:
lastMessageData.username,
            uid:
lastMessageData.contactId,
            profileImageUrl:
lastMessageData.profileImageUrl,
            active: true,
            lastSeen: 0,
            phoneNumber: '0',
            groupId: [],
        ),
    );
},
title: Row(
    mainAxisAlignment:
MainAxisAlignment.spaceBetween,
    children: [
        Expanded(
            child: Text(
                lastMessageData.username,
                style: const TextStyle(
                    fontWeight:
FontWeight.w500,
                    color: Colors.white,
                ),
                overflow:
TextOverflow.ellipsis,
            ),

```

```

        ),
        Text(
            DateFormat.Hm().format(last
MessageData.timeSent),
            style: const TextStyle(
                fontSize: 13,
                color: Colors.grey,
            ),
        ),
    ],
),
    subtitle: Padding(
        padding: const
EdgeInsets.only(top: 3),
        child: Row(
            children: [
                const Icon(
                    Icons.done_all,
                    size: 16,
                    color: Colors.greenDark,
                ),
                const SizedBox(width: 3),
                Expanded(
                    child: Text(
                        lastMessageData.lastMessa
ge,
                        maxLines: 1,
                        overflow:
TextOverflow.ellipsis,
                        style: const TextStyle(
                            color: Colors.grey,

```

```

        ),
        ),
        ),
    ],
    ),
    ),
    leading: CircleAvatar(
        backgroundImage:
CachedNetworkImageProvider(
        lastMessageData.profileImage
Url,
    ),
    radius: 24,
    ),
    ),
    Divider(
        color: Colors.grey[900],
        indent: 80,
        height: 0,
    ),
    ],
    );
},
),
floatingActionButton:
FloatingActionButton(
    backgroundColor: Colors.greenDark,
    onPressed: () {
        Navigator.pushNamed(context,
Routes.contact);
    },

```

```

        child: const Icon(
            Icons.message,
            color: Colors.white,
        ),
    ),
    );
}
}

```

Status Page:

```

import 'dart:html' as html;
import 'package:flutter/material.dart';
import
'package:image_picker/image_picker.dart'
;

```

// Enhanced Status Model

```

class Status {
    final String id;
    final String name;
    final String imageUrl; // Changed from
imagePath to imageUrl
    final DateTime timestamp;
    final bool isViewed;
    final StatusType type;
    final String? caption;

    Status({
        required this.id,

```

```

    required this.name,
    required this.imageUrl,
    required this.timestamp,
    this.isViewed = false,
    this.type = StatusType.image,
    this.caption,
  });
}

enum StatusType { image, text }

class StatusHomePage extends
StatefulWidget {
  const StatusHomePage({super.key});

  @override
  State<StatusHomePage> createState() =>
    _StatusHomePageState();
}

class _StatusHomePageState extends
State<StatusHomePage> {
  final ImagePicker _picker = ImagePicker();
  List<Status> recentUpdates = [];
  List<Status> viewedUpdates = [];
  Status? myCurrentStatus;

  @override
  void initState() {
    super.initState();

    _loadDemoStatuses();
  }

  void _loadDemoStatuses() {
    // Load demo statuses with web-
    compatible URLs
    recentUpdates = [
      Status(
        id: '1',
        name: 'Balram Rathore',
        imageUrl:
        'https://picsum.photos/400/600', // Using
        placeholder image
        timestamp: DateTime.now(),
      ),
      Status(
        id: '2',
        name: 'Kishor Kumar',
        imageUrl:
        'https://picsum.photos/400/601', //
        Different placeholder
        timestamp: DateTime.now(),
      ),
    ];

    viewedUpdates = [
      Status(
        id: '3',
        name: 'Saket Sinha',
        imageUrl:
        'https://picsum.photos/400/602',

```

```

        timestamp:
DateTime.now().subtract(const
Duration(hours: 1)),
        isViewed: true,
    ),
];
}

Future<void> _addImageStatus() async {
    try {
        final XFile? image = await
_picker.pickImage(source:
ImageSource.gallery);

        if (image == null) return;

        // Create object URL for web

        final bytes = await
image.readAsBytes();

        final blob = html.Blob([bytes]);

        final url =
html.Url.createObjectUrlFromBlob(blob);

        setState(() {
            myCurrentStatus = Status(
                id: DateTime.now().toString(),
                name: 'My Status',
                imageUrl: url,
                timestamp: DateTime.now(),
                type: StatusType.image,
            );

```

```

        // Add to recent updates
        recentUpdates.insert(0,
myCurrentStatus!);
    });
} catch (e) {
    showDialog(
        context: context,
        builder: (context) => AlertDialog(
            title: const Text('Error'),
            content: Text('Failed to add status:
$e'),
            actions: [
                TextButton(
                    onPressed: () =>
Navigator.pop(context),
                    child: const Text('OK'),
                ),
            ],
        ),
    );
}
}

Future<void> _addTextStatus() async {
    final TextEditingController textController
= TextEditingController();

    final result = await showDialog<String>(
        context: context,
        builder: (context) => AlertDialog(
            backgroundColor: const
Color(0xFF121B22),

```

```

        title: const Text('Add Text Status',
          style: TextStyle(color:
Colors.white)),
        content: TextField(
          controller: textController,
          style: const TextStyle(color:
Colors.white),
          maxLines: 3,
          decoration: const InputDecoration(
            hintText: 'Enter your status...',
            hintStyle: TextStyle(color:
Colors.grey),
            border: OutlineInputBorder(),
          ),
        ),
        actions: [
          TextButton(
            onPressed: () =>
Navigator.pop(context),
            child: const Text('Cancel'),
          ),
          TextButton(
            onPressed: () =>
Navigator.pop(context,
textController.text),
            child: const Text('Post'),
          ),
        ],
      ),
    );

```

```

    if (result != null && result.isNotEmpty) {
      setState(() {
        myCurrentStatus = Status(
          id: DateTime.now().toString(),
          name: 'My Status',
          imageUrl: '', // Empty for text status
          timestamp: DateTime.now(),
          type: StatusType.text,
          caption: result,
        );

        // Add to recent updates
        recentUpdates.insert(0,
myCurrentStatus!);
      });
    }
  }

  void _viewStatus(Status status) {
    Navigator.push(
      context,
      MaterialPageRoute(
        builder: (context) =>
StatusViewPage(status: status),
      ),
    );

    // Mark status as viewed
    if (!status.isViewed &&
recentUpdates.contains(status)) {

```



```

    setState(() {
      final index =
recentUpdates.indexOf(status);

      final viewedStatus = Status(
        id: status.id,
        name: status.name,
        imageUrl: status.imageUrl,
        timestamp: status.timestamp,
        isViewed: true,
        type: status.type,
        caption: status.caption,
      );

      recentUpdates.removeAt(index);
      viewedUpdates.insert(0,
viewedStatus);
    });
  }
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    backgroundColor: const
Color(0xFF121B22),
    body: SingleChildScrollView(
      child: Column(
        crossAxisAlignment:
CrossAxisAlignment.start,
        children: [
          // My Status Section

```

```

ListTile(
  leading: Stack(
    children: [
      CircleAvatar(
        radius: 30,
        backgroundColor: Colors.grey,
        backgroundImage:
myCurrentStatus?.type ==
StatusType.image
          ?
NetworkImage(myCurrentStatus!.imageUr
l)
          : null,
        child: myCurrentStatus == null
          ? const Icon(Icons.person,
            color: Colors.white, size:
35)
          : myCurrentStatus?.type ==
StatusType.text
            ? const
Icon(Icons.text_fields, color: Colors.white)
            : null,
      ),
      Positioned(
        bottom: 0,
        right: 0,
        child: Container(
          padding: const
EdgeInsets.all(4),
          decoration: BoxDecoration(
            color: Colors.green,

```

borderRadius:	},
BorderRadius.circular(12),	),
),	
child: const Icon(	if (recentUpdates.isNotEmpty) ...[
Icons.add,	const Padding(
color: Colors.white,	padding:
size: 20,	EdgeInsets.symmetric(horizontal: 15,
),	vertical: 8),
),	child: Text(
),	'Recent updates',
],	style: TextStyle(
),	color: Colors.grey,
title: const Text(	fontWeight: FontWeight.bold,
'My status',	),
style: TextStyle(	),
color: Colors.white,	),
fontWeight: FontWeight.bold,	ListView.builder(
),	shrinkWrap: true,
),	physics: const
subtitle: Text(	NeverScrollableScrollPhysics(),
myCurrentStatus != null	itemCount:
? 'Tap to view your status'	recentUpdates.length,
: 'Tap to add status update',	itemBuilder: (context, index) =>
style: const TextStyle(color:	StatusTile(
Colors.grey),	status: recentUpdates[index],
),	onTap: () =>
onTap: () {	_viewStatus(recentUpdates[index]),
if (myCurrentStatus != null) {	),
_viewStatus(myCurrentStatus!);	),
}	],

```

    if (viewedUpdates.isNotEmpty) ...[
      const Padding(
        padding:
EdgeInsets.symmetric(horizontal: 15,
vertical: 8),
        child: Text(
          'Viewed updates',
          style: TextStyle(
            color: Colors.grey,
            fontWeight: FontWeight.bold,
          ),
        ),
      ),
    ],
    ListView.builder(
      shrinkWrap: true,
      physics: const
NeverScrollableScrollPhysics(),
      itemCount:
viewedUpdates.length,
      itemBuilder: (context, index) =>
StatusTile(
        status: viewedUpdates[index],
        onTap: () =>
_viewStatus(viewedUpdates[index]),
      ),
    ],
  ],
),
),
floatingActionButton: Column(

```

```

      mainAxisAlignment:
MainAxisAlignment.end,
      children: [
        FloatingActionButton(
          mini: true,
          onPressed: _addTextStatus,
          backgroundColor: Colors.grey[700],
          child: const Icon(Icons.edit),
        ),
        const SizedBox(height: 16),
        FloatingActionButton(
          onPressed: _addImageStatus,
          backgroundColor: Colors.teal[700],
          child: const Icon(Icons.camera_alt),
        ),
      ],
    ),
  );
}
}

```

```

class StatusTile extends StatelessWidget {
  final Status status;
  final VoidCallback onTap;

  const StatusTile({
    super.key,
    required this.status,
    required this.onTap,
  });
}

```

```

@override
Widget build(BuildContext context) {
  return ListTile(
    leading: Container(
      decoration: BoxDecoration(
        shape: BoxShape.circle,
        border: Border.all(
          color: status.isViewed ? Colors.grey
: Colors.green,
          width: 2,
        ),
      ),
    child: CircleAvatar(
      radius: 28,
      backgroundColor: Colors.grey,
      backgroundImage: status.type ==
StatusType.image
        ? NetworkImage(status.imageUrl)
        : null,
    child: status.type == StatusType.text
        ? const Icon(Icons.text_fields,
color: Colors.white)
        : null,
    ),
  ),
  title: Text(
    status.name,
    style: const TextStyle(
      color: Colors.white,

```

```

fontWeight: FontWeight.bold,
    ),
  ),
  subtitle: Text(
    'Today,
    ${status.timestamp.hour}:${status.timesta
mp.minute.toString().padLeft(2, '0')}',
    style: const TextStyle(color:
Colors.grey),
  ),
  onTap: onTap,
);
}
}

```

```

class StatusViewPage extends
StatelessWidget {

```

```

  final Status status;

```

```

  const StatusViewPage({super.key,
required this.status});

```

```

  @override

```

```

Widget build(BuildContext context) {
  return Scaffold(
    backgroundColor: Colors.black,
    appBar: AppBar(
      backgroundColor: Colors.transparent,
      title: Text(status.name),
      iconTheme: const
IconThemeData(color: Colors.white),

```

```

    ),
    body: Center(
      child: status.type == StatusType.image
        ? Image.network(
            status.imageUrl,
            fit: BoxFit.contain,
            loadingBuilder: (context, child,
loadingProgress) {
              if (loadingProgress == null)
return child;

              return const
CircularProgressIndicator();
            },
            errorBuilder: (context, error,
stackTrace) {
              return const Icon(
                Icons.error,
                color: Colors.red,
                size: 48,
              );
            },
        )
      : Container(
          padding: const EdgeInsets.all(20),
          color: Colors.teal,
          child: Center(
            child: Text(
              status.caption ?? "",
              style: const TextStyle(
                color: Colors.white,
                fontSize: 24,

```

```

            ),
            textAlign: TextAlign.center,
          ),
        ),
      ),
    );
  }
}

```

Call page:

```
import 'package:flutter/material.dart';
```

```

class Call {
  final String name;
  final String avatarUrl;
  final DateTime timestamp;
  final bool isIncoming;
  final bool isMissed;
  final bool isVideo;

  Call({
    required this.name,
    required this.avatarUrl,
    required this.timestamp,
    this.isIncoming = true,
    this.isMissed = false,
    this.isVideo = false,
  });
}

```

```

class CallHomePage extends StatefulWidget {
  const CallHomePage({super.key});

  @override
  State<CallHomePage> createState() =>
    _CallHomePageState();
}

```

```

class _CallHomePageState extends
State<CallHomePage> {

  // Expanded demo call data with Picsum
  images

  final List<Call> calls = [

    Call(
      name: "Mike Johnson",
      avatarUrl:
        "https://picsum.photos/seed/mike/200",
      timestamp: DateTime.now().subtract(const
        Duration(minutes: 5)),
      isIncoming: true,
      isVideo: false,
    ),
    Call(
      name: "Sarah Wilson",
      avatarUrl:
        "https://picsum.photos/seed/sarah/200",
      timestamp: DateTime.now().subtract(const
        Duration(minutes: 15)),
      isIncoming: false,
      isVideo: true,
    ),
    Call(

```

```

      name: "Alex Thompson",
      avatarUrl:
        "https://picsum.photos/seed/alex/200",
      timestamp: DateTime.now().subtract(const
        Duration(hours: 2)),
      isIncoming: true,
      isMissed: true,
      isVideo: false,
    ),
    Call(
      name: "Emma Davis",
      avatarUrl:
        "https://picsum.photos/seed/emma/200",
      timestamp: DateTime.now().subtract(const
        Duration(hours: 3)),
      isIncoming: false,
      isVideo: true,
    ),
    Call(
      name: "James Miller",
      avatarUrl:
        "https://picsum.photos/seed/james/200",
      timestamp: DateTime.now().subtract(const
        Duration(hours: 5)),
      isIncoming: true,
      isMissed: false,
      isVideo: true,
    ),
    Call(
      name: "Lisa Brown",
      avatarUrl:
        "https://picsum.photos/seed/lisa/200",
      timestamp: DateTime.now().subtract(const
        Duration(hours: 6)),

```

```

        isIncoming: false,

        isMissed: true,

        isVideo: false,
    ),
    Call(
        name: "David Clark",

        avatarUrl:
"https://picsum.photos/seed/david/200",

        timestamp: DateTime.now().subtract(const
Duration(hours: 8)),

        isIncoming: true,

        isVideo: false,
    ),
    Call(
        name: "Rachel Green",

        avatarUrl:
"https://picsum.photos/seed/rachel/200",

        timestamp: DateTime.now().subtract(const
Duration(days: 1)),

        isIncoming: false,

        isVideo: true,
    ),
    Call(
        name: "Tom Wilson",

        avatarUrl:
"https://picsum.photos/seed/tom/200",

        timestamp: DateTime.now().subtract(const
Duration(days: 1, hours: 3)),

        isIncoming: true,

        isMissed: true,

        isVideo: false,
    ),
    Call(

```

```

        name: "Emily White",

        avatarUrl:
"https://picsum.photos/seed/emily/200",

        timestamp: DateTime.now().subtract(const
Duration(days: 1, hours: 6)),

        isIncoming: false,

        isVideo: true,
    ),
];

```

```

void _showOutgoingCallDialog(
    BuildContext context, String name, bool
isVideo) {
    showDialog(
        context: context,
        barrierDismissible: false,
        builder: (context) => Align(
            alignment: Alignment.topCenter,
            child: OutgoingCallDialog(name: name,
isVideo: isVideo),
        ),
    );
}

```

```

String _getTimeString(DateTime timestamp) {
    final now = DateTime.now();

    final difference =
now.difference(timestamp);

    if (difference.inDays > 0) {
        return "Yesterday,
${timestamp.hour.toString().padLeft(2,
'0')}:${timestamp.minute.toString().padLeft(2,
'0')}";
    }
}

```

```

    } else if (difference.inHours > 0) {
        return "${difference.inHours}h ago";
    } else {
        return
"$${timestamp.hour.toString().padLeft(2,
'0')}:${timestamp.minute.toString().padLeft(2,
'0')}";
    }
}

```

```

@override
Widget build(BuildContext context) {
    return Scaffold(
        backgroundColor: const
Color(0xFF1F2C34),
        body: SafeArea(
            child: Column(
                children: [
                    Expanded(
                        child: ListView.builder(
                            itemCount: calls.length,
                            itemBuilder: (context, index) {
                                final call = calls[index];
                                return ListTile(
                                    leading: CircleAvatar(
                                        radius: 25,
                                        backgroundImage:
NetworkImage(call.avatarUrl),
                                        backgroundColor:
Colors.grey[700],
                                    ),
                                    title: Text(
                                        call.name,

```

```

style: const TextStyle(
    color: Colors.white,
    fontWeight: FontWeight.w500,
),
),
subtitle: Row(
    children: [
        Icon(
            call.isIncoming
                ? Icons.call_received
                : Icons.call_made,
            size: 15,
            color: call.isMissed ? Colors.red
: Colors.green,
        ),
        const SizedBox(width: 4),
        Text(
            _getTimeString(call.timestamp),
            style: TextStyle(
                color: Colors.grey[400],
                fontSize: 13,
            ),
        ),
    ],
),
trailing: IconButton(
    icon: Icon(
        call.isVideo ? Icons.videocam :
Icons.call,
        color: const Color(0xFF00A884),
    ),
    onPressed: () {

```



```

        color: Colors.white,
      },
      fontSize: 16,
    },
    fontWeight: FontWeight.bold,
  ),
),
Text(
  "Calling${isVideo ? ' video' : ''}...",
  style: TextStyle(
    color: Colors.grey[400],
    fontSize: 14,
  ),
),
],
),
),
IconButton(
  icon: const Icon(Icons.close, color:
Colors.red),
  onPressed: () =>
Navigator.of(context).pop(),
),
IconButton(
  icon: Icon(
    isVideo ? Icons.videocam : Icons.call,
    color: Colors.green,
  ),
  onPressed: () =>
Navigator.of(context).pop(),
),
],
),
);

```

Routing Code:

```
import 'package:flutter/material.dart';
import 'package:page_transition/page_transition.dart';
import 'package:whatsapp_messenger/common/models/user_model.dart';
import 'package:whatsapp_messenger/feature/auth/pages/login_page.dart';
import 'package:whatsapp_messenger/feature/auth/pages/user_info_page.dart';
import 'package:whatsapp_messenger/feature/auth/pages/verification_page.dart';
import 'package:whatsapp_messenger/feature/chat/pages/chat_page.dart';
import 'package:whatsapp_messenger/feature/chat/pages/profile_page.dart';
import 'package:whatsapp_messenger/feature/contact/pages/contact_page.dart';
import 'package:whatsapp_messenger/feature/home/pages/home_page.dart';
import 'package:whatsapp_messenger/feature/welcome/pages/welcome_page.dart';
```

```
class Routes {
  static const String welcome = 'welcome';
  static const String login = 'login';
  static const String verification = 'verification';
  static const String userInfo = 'user-info';
  static const String home = 'home';
  static const String contact = 'contact';
  static const String chat = 'chat';
  static const String profile = 'profile';

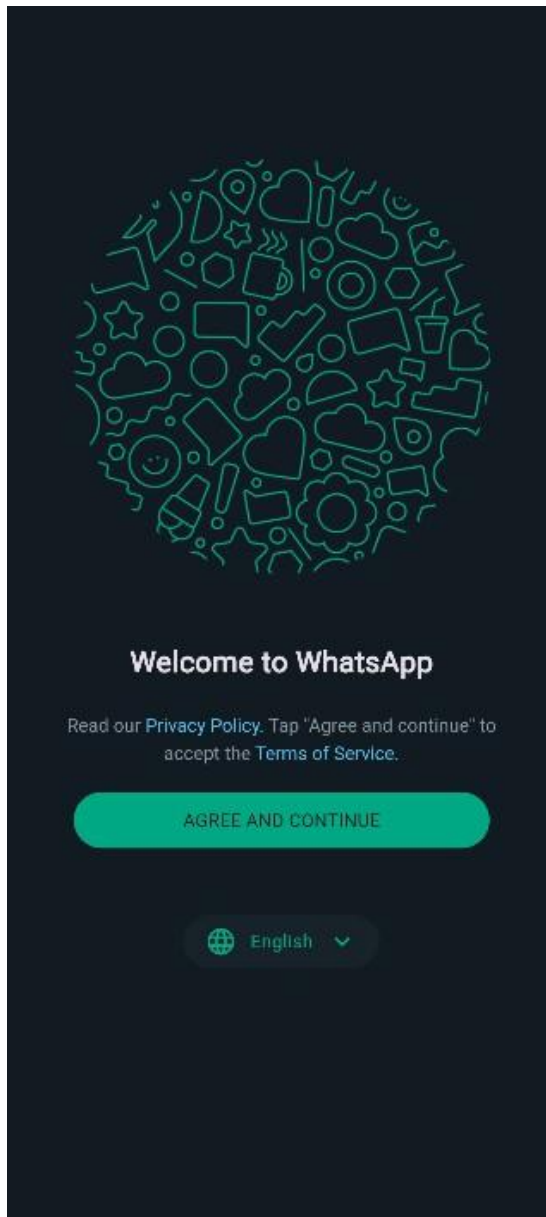
  static Route<dynamic> onGenerateRoute(RouteSettings settings) {
    switch (settings.name) {
      case welcome:
        return PageTransition(
          child: const WelcomePage(),
          type: PageTransitionType.rightToLeft,
          duration: const Duration(milliseconds: 500),
        );
      case login:
        return PageTransition(
          child: const LoginPage(),
          type: PageTransitionType.rightToLeft,
          duration: const Duration(milliseconds: 500),
        );
      case verification:
        final args = settings.arguments as Map<String, dynamic>;
        if (args == null ||
            !args.containsKey('smsCodeId') ||
            !args.containsKey('phoneNumber')) {
          throw ArgumentError("Invalid arguments for VerificationPage");
        }
        return PageTransition(
          child: VerificationPage(
            smsCodeId: args['smsCodeId'],
            phoneNumber: args['phoneNumber'],
          ),
          type: PageTransitionType.fade,
          duration: const Duration(milliseconds: 600),
        );
      case userInfo:
        final String? profileImageUrl = settings.arguments as String?;
```

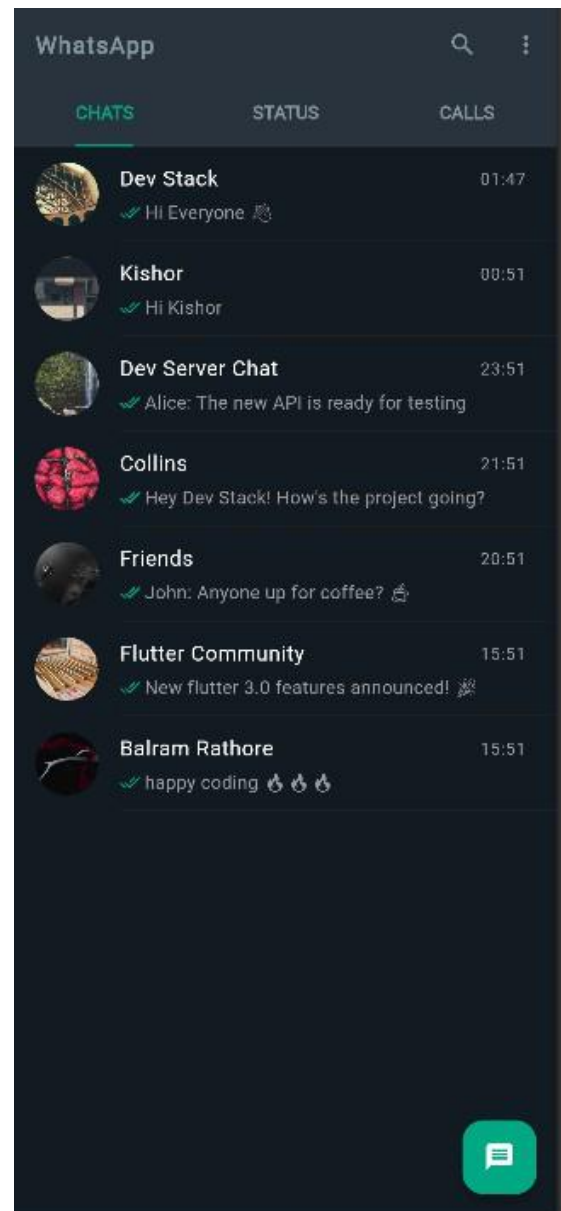
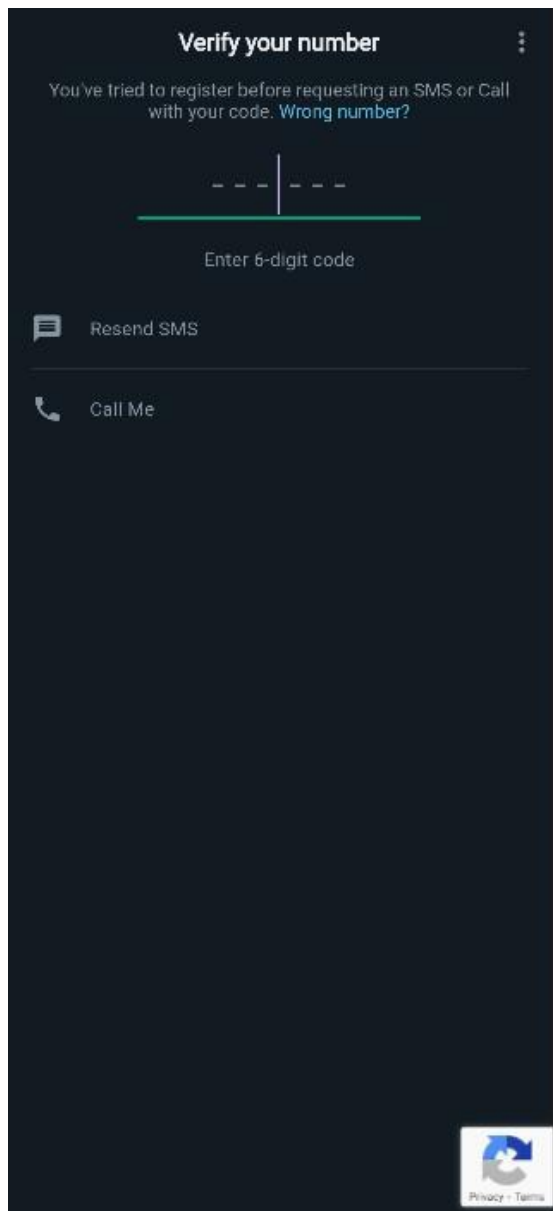
```

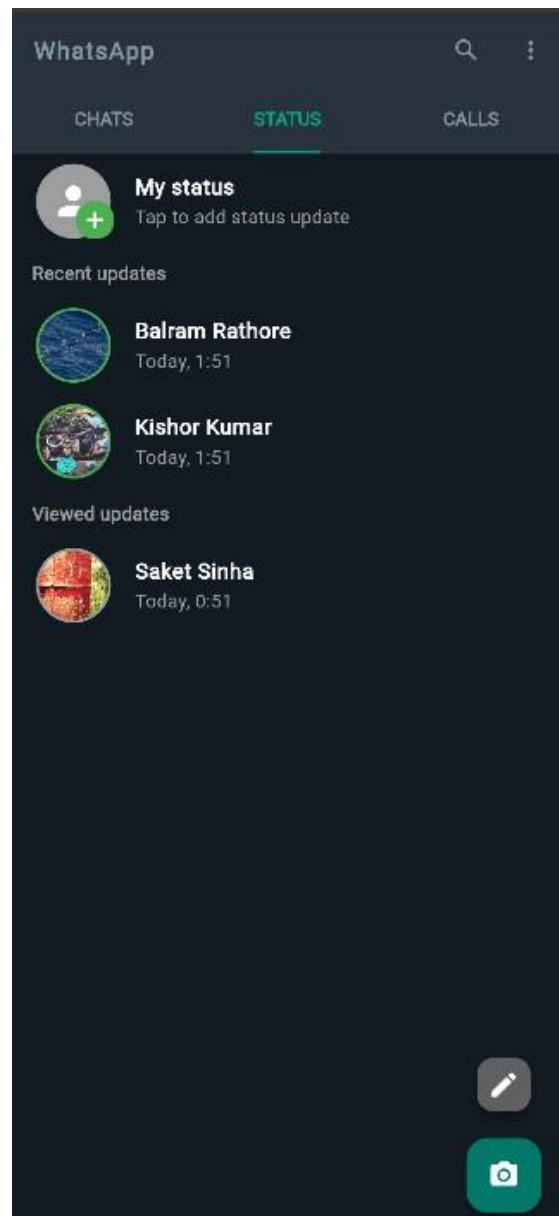
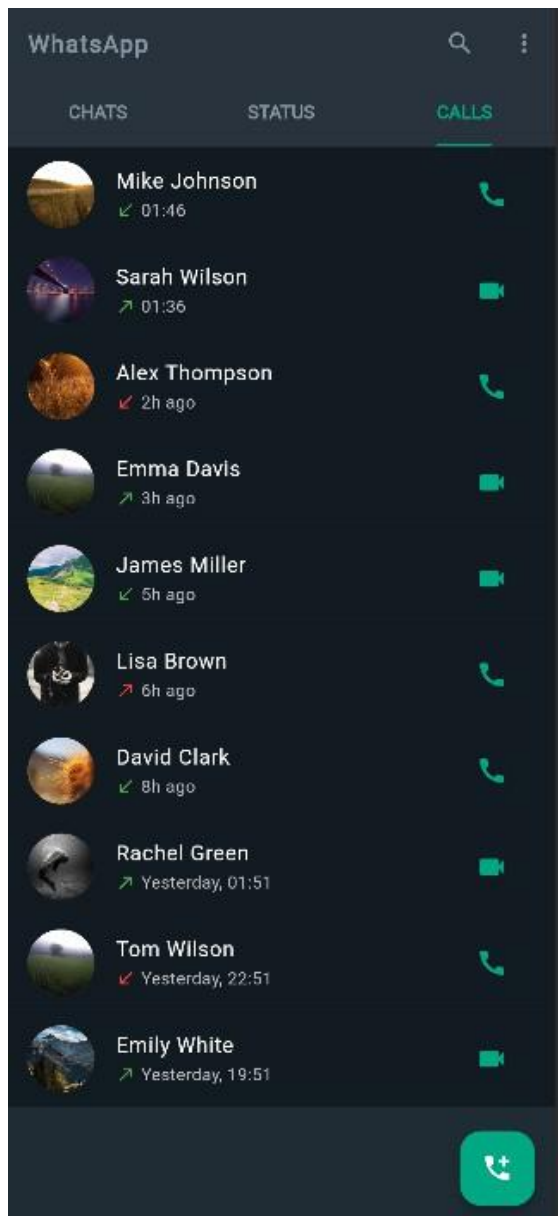
return PageTransition(
  child: UserInfoPage(profileImageUrl: profileImageUrl),
  type: PageTransitionType.rightToLeft,
  duration: const Duration(milliseconds: 500),
);
case home:
return PageTransition(
  child: const HomePage(),
  type: PageTransitionType.fade,
  duration: const Duration(milliseconds: 600),
);
case contact:
return PageTransition(
  child: const ContactPage(),
  type: PageTransitionType.rightToLeft,
  duration: const Duration(milliseconds: 500),
);
case chat:
final user = settings.arguments as UserModel?;
if (user == null) {
  throw ArgumentError("UserModel argument is required for ChatPage");
}
return PageTransition(
  child: ChatPage(user: user),
  type: PageTransitionType.rightToLeftWithFade,
  duration: const Duration(milliseconds: 600),
);
case profile:
final user = settings.arguments as UserModel?;
if (user == null) {
  throw ArgumentError("UserModel argument is required for ProfilePage");
}
return PageTransition(
  child: ProfilePage(user: user),
  type: PageTransitionType.fade,
  duration: const Duration(milliseconds: 800),
);
default:
return MaterialPageRoute(
  builder: (context) => const Scaffold(
    body: Center(
      child: Text('No Page Route Provided'),
    ),
  ),
);
}
}
}

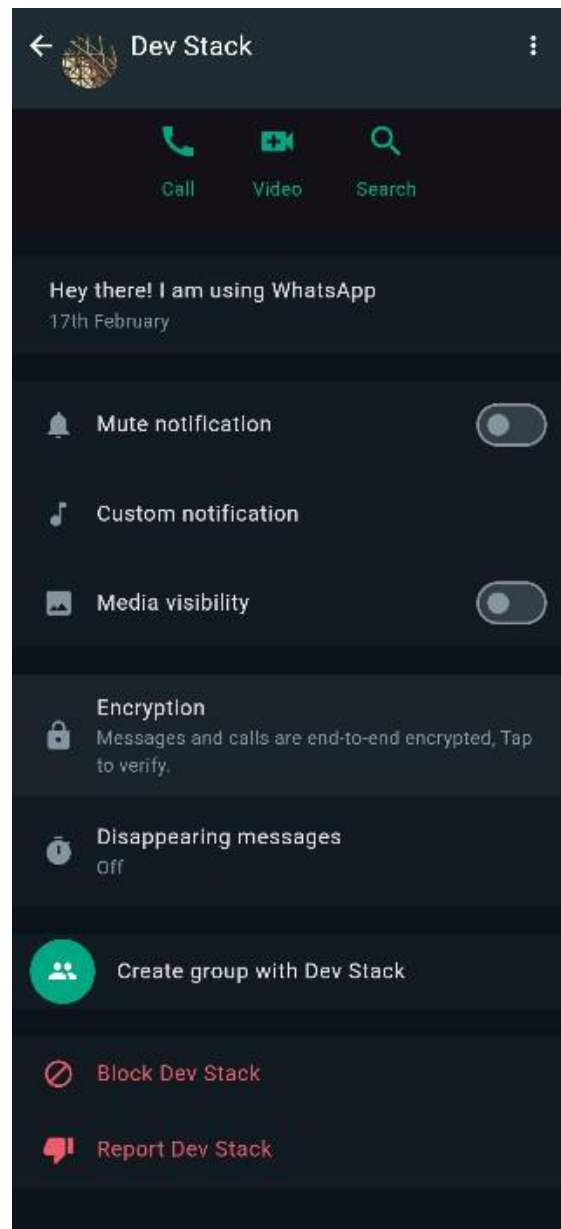
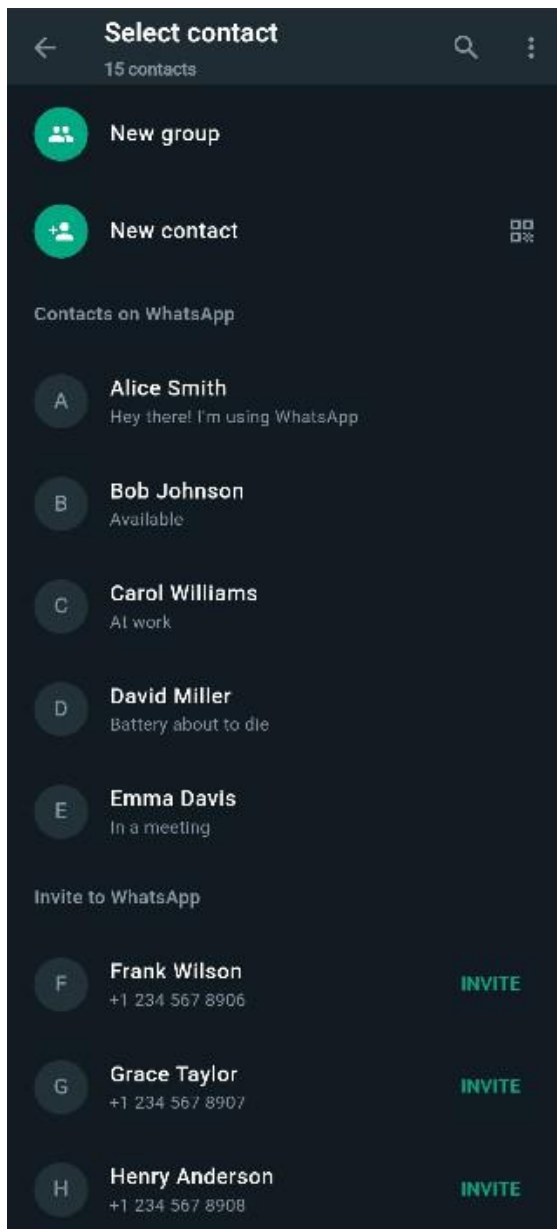
```

Output:









MAD & PWA Lab

Journal

Experiment No.	06
Experiment Title.	To Connect Flutter UI with fireBase database
Roll No.	24
Name	Niraj Kothawade
Class	D15A/D15B
Subject	MAD & PWA Lab
Lab Outcome	LO3: Analyze and Build production ready Flutter App by incorporating backend services and deploying on Android / iOS
Grade:	

EXPERIMENT NO: - 06

Name:- Niraj Kothawade

Class:- D15A

Roll:No:- 24

AIM:- To connect Flutter UI with Firebase database.

Theory: -

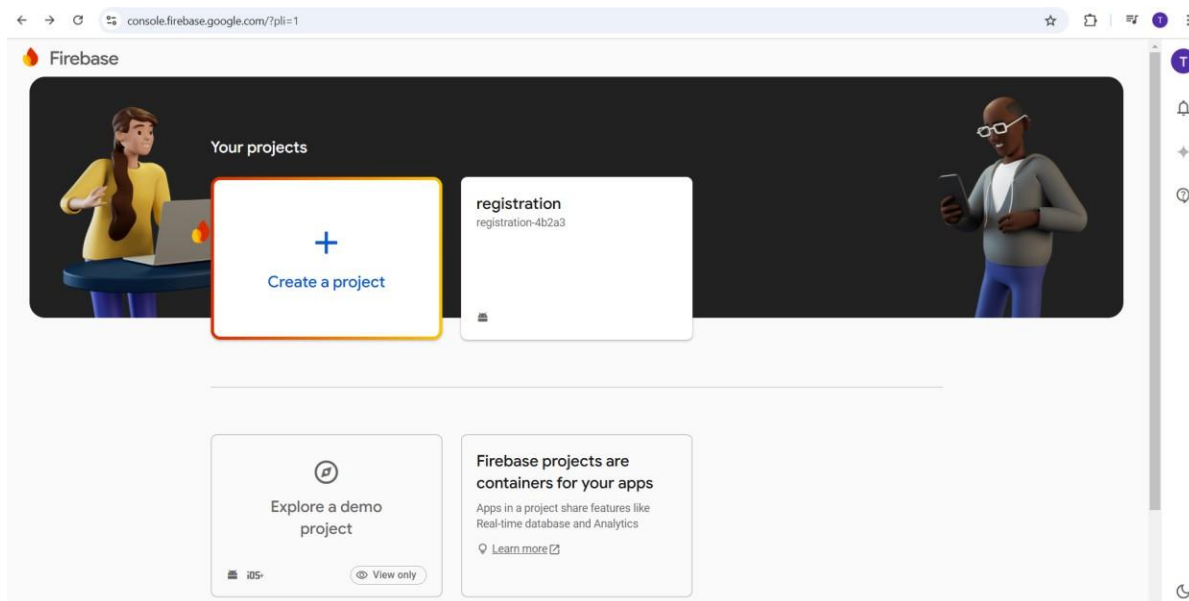
Flutter is an open-source UI toolkit developed by Google for building natively compiled applications for mobile, web, and desktop from a single codebase. Firebase, a Backend-as-a-Service (BaaS) platform, provides real-time database, authentication, and cloud storage services, making it a powerful backend solution for Flutter applications.

By integrating Firebase with Flutter, developers can store and retrieve data in real time, authenticate users, and manage cloud-based data efficiently. This is particularly useful for applications requiring dynamic content updates and user interactions.

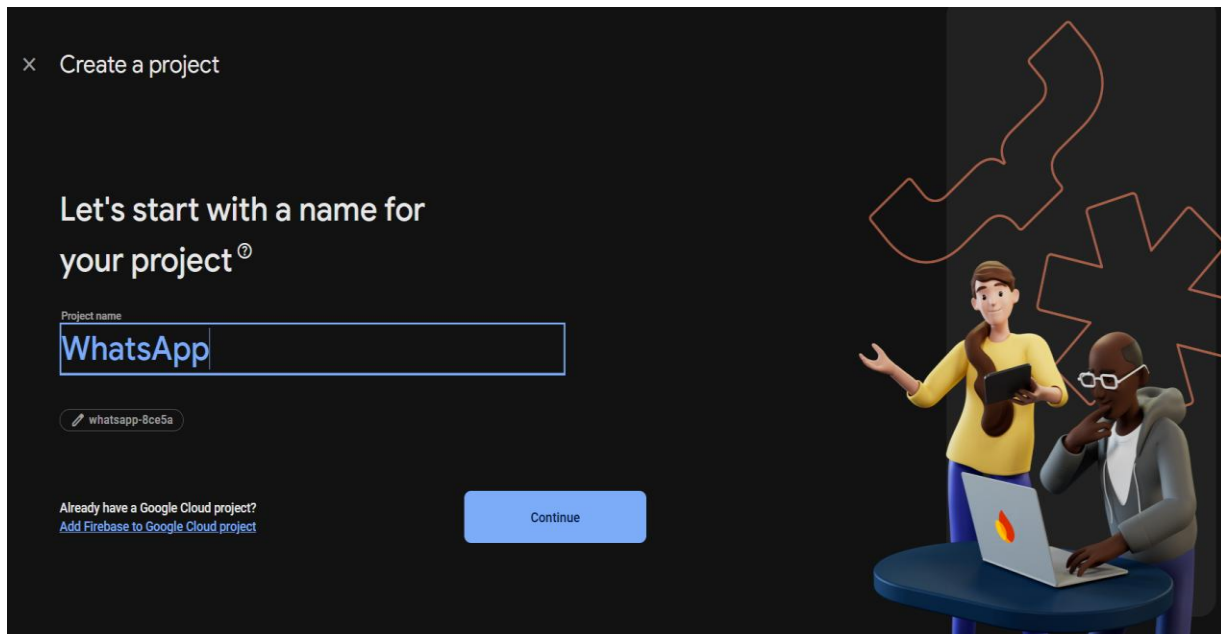
➤ **Steps to Connect Flutter UI with Firebase Database**

Step 1:

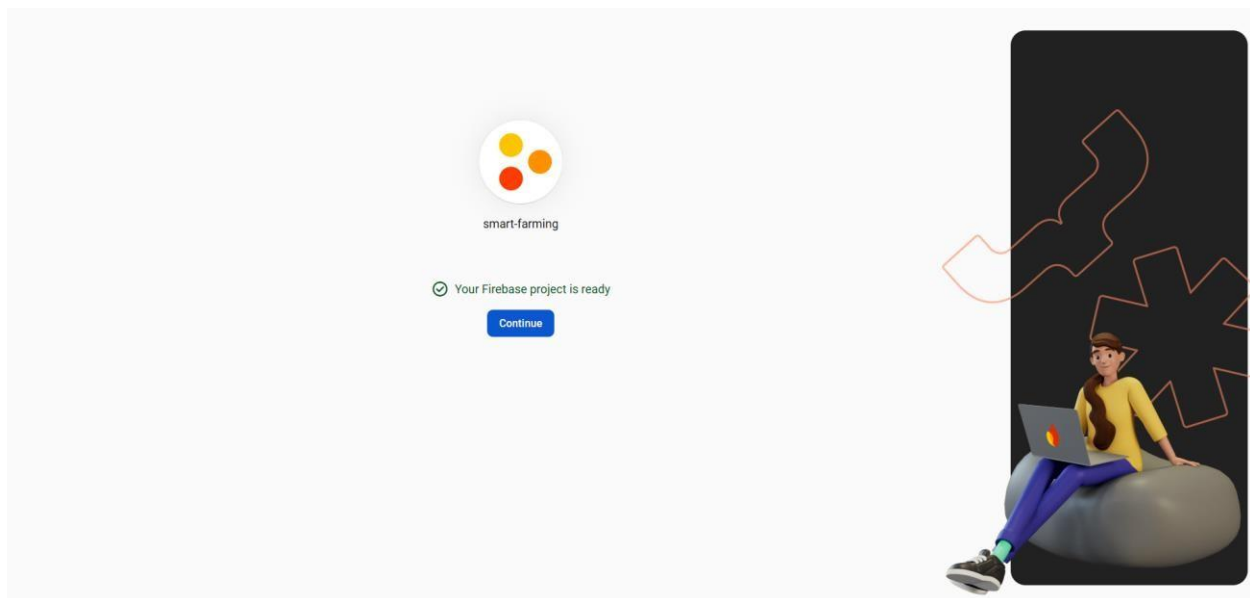
1.1) Go to Firebase Console and Create a Firebase Project



1.2) Click on Create a Project and give it a suitable name.

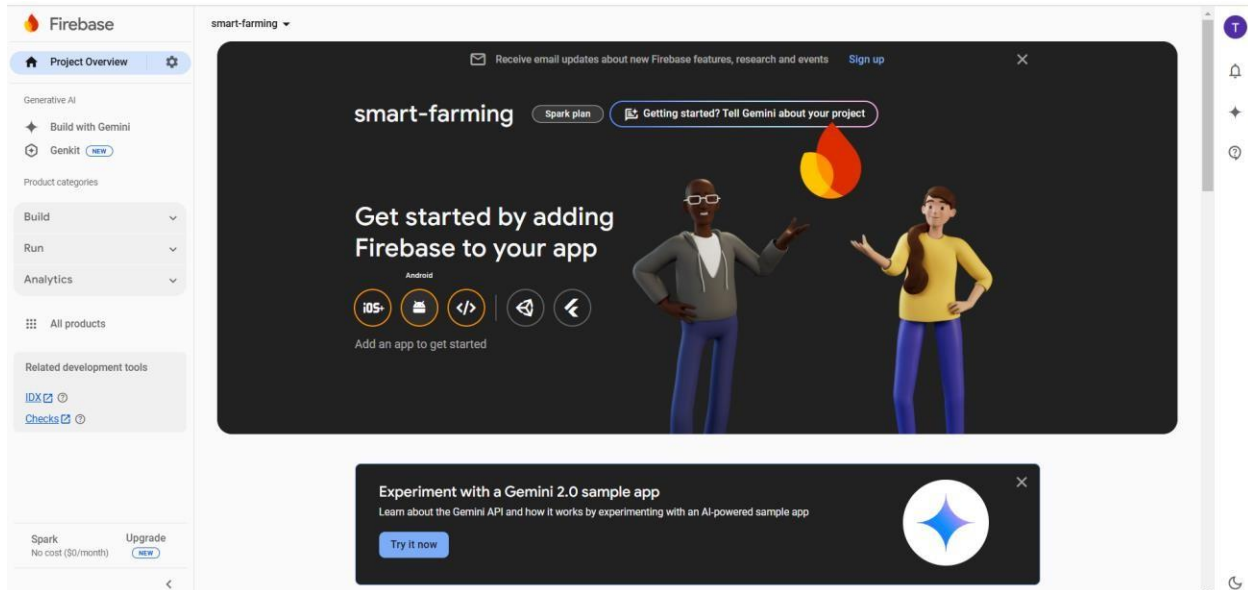


1.3) Enable Google Analytics (optional) & Click continue and complete the setup

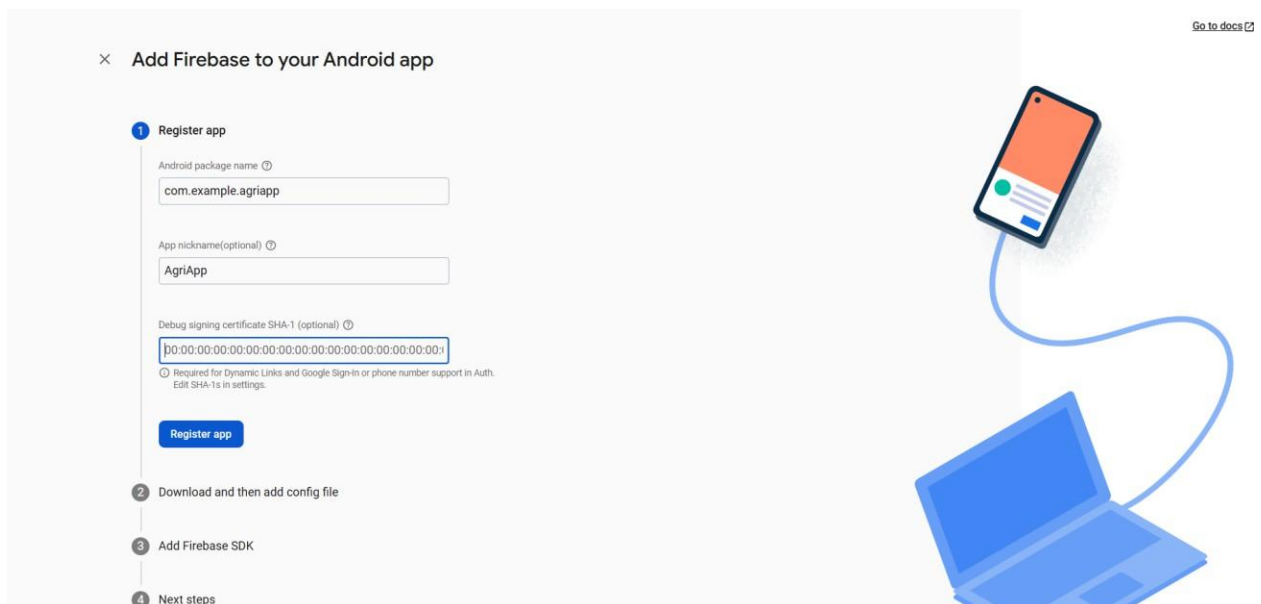


Step 2:- Add Firebase to Your Flutter App

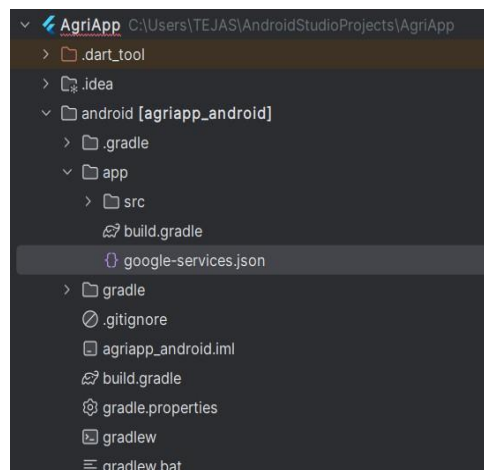
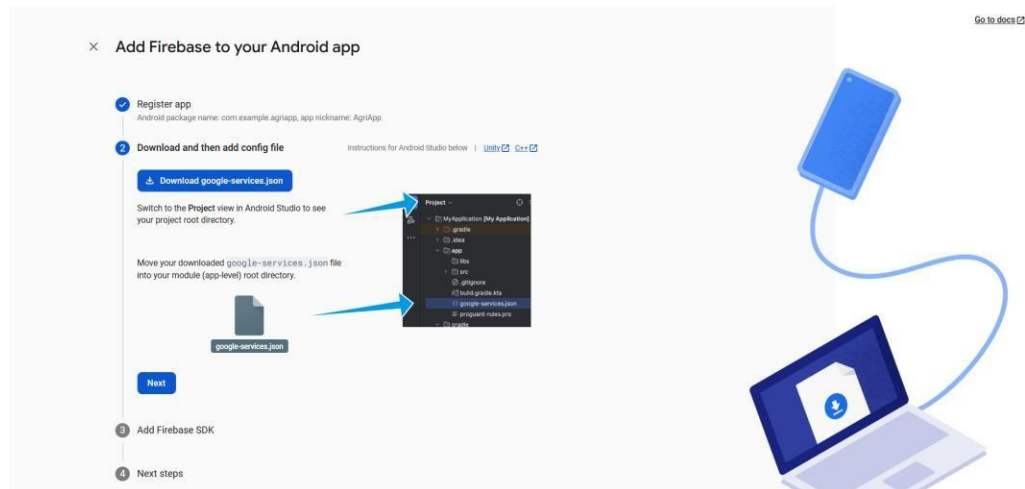
2.1) Click on Android/iOS/Web based on your Flutter application



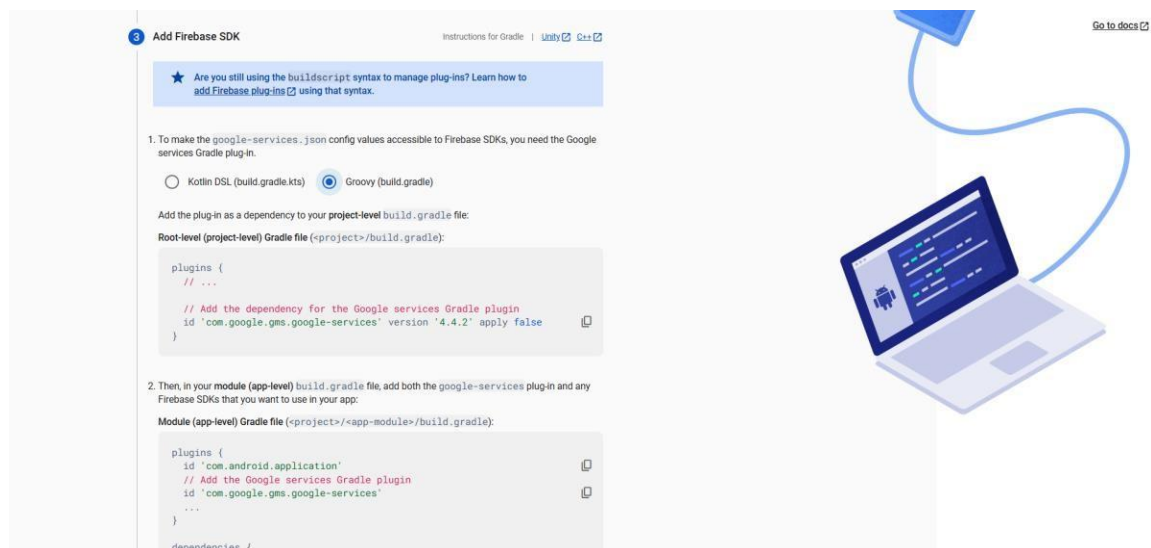
2.2) Register your app with a unique package name (found in android/app/build.gradle for Android).

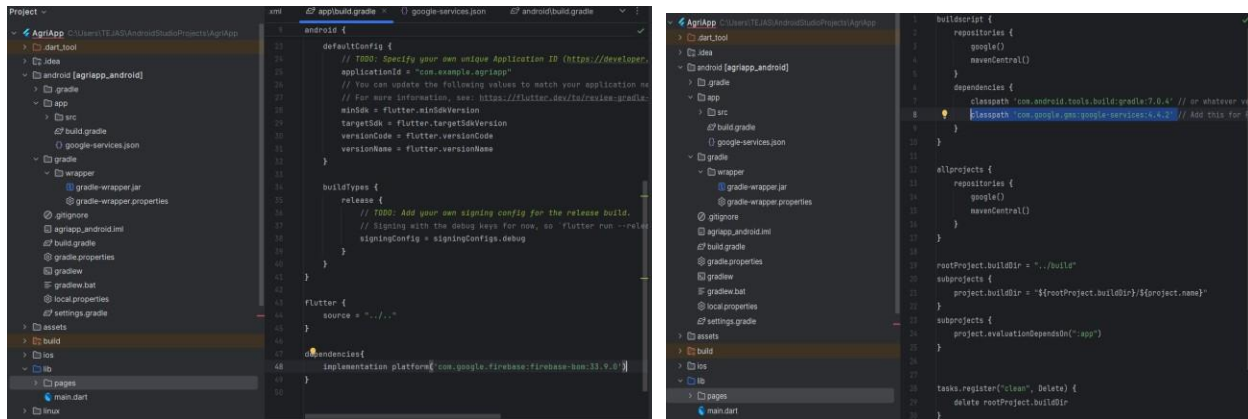


2.3) Download the google-services.json (for Android) & place the JSON file inside android/app/ directory.



2.4) Add Firebase SDK dependencies to android/build.gradle





Step 3: - Add Firebase Authentication to Your App

3.1) Add Firebase Authentication Dependencies

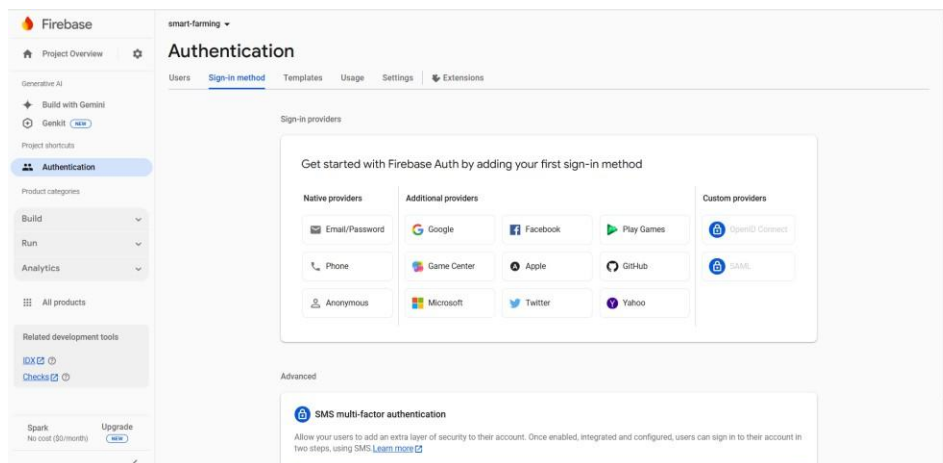
```
dependencies:
  flutter:
    sdk: flutter
  firebase_core: ^3.11.0
  firebase_auth: ^5.4.2 # For authentication
  cloud_firestore: ^5.6.3 # For Firestore, if you need it
  firebase_messaging: ^15.2.2
  http: ^0.13.3
  image_picker: ^1.0.4
  tf_lite_flutter: ^0.11.0
  image: ^3.2.0
  url_launcher: ^6.1.14
```

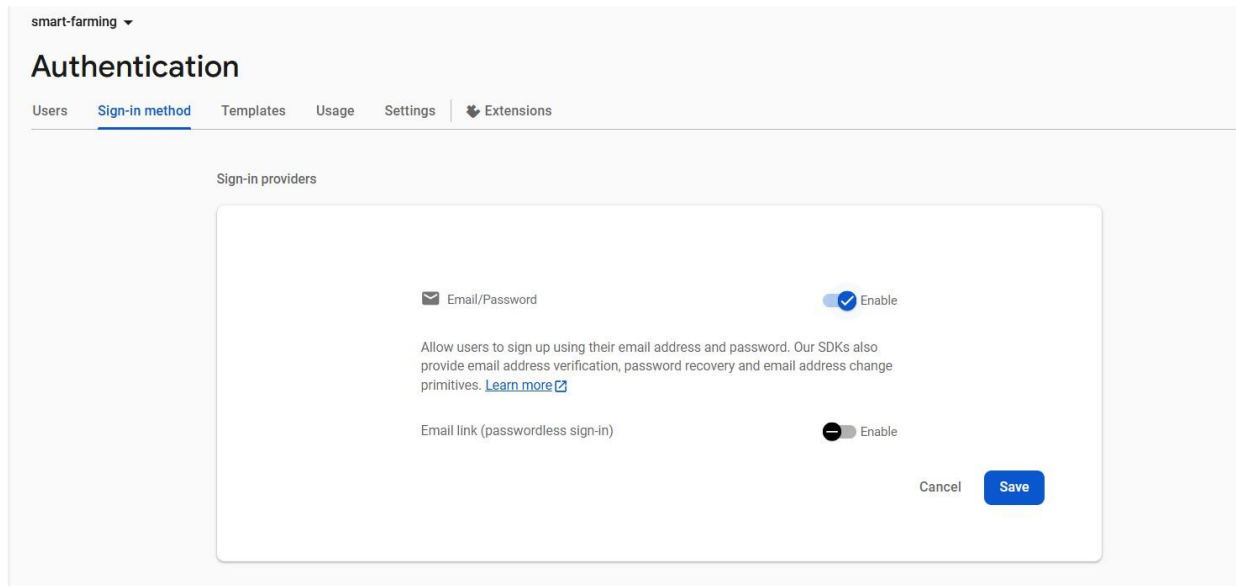
3.2) Enable Authentication in Firebase Console

Go to **Firebase Console** → **Authentication**.

Click on **Sign-in method** and enable **Email/Password** (or any other method like Google).

Click Save





3.3) Implement Authentication in Flutter

Modify main.dart

```
import 'package:firebase_core/firebase_core.dart';
import 'package:firebase_auth/firebase_auth.dart';

void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp();
  runApp(MyApp());
}
```

Step 4: -Configure Firebase Realtime Database

- 4.1) Go to Firebase Console → Realtime Database.
- 4.2) Click **Create Database** → Choose location → Set rules (for development, set read/write to true).
- 4.3) Click **Publish**.

Code:-

Login page:

```
import
'package:country_picker/country_picker.dart';
import 'package:flutter/material.dart';
import
'package:flutter_riverpod/flutter_riverpod.dart';
import
'package:whatsapp_messenger/common/extension/custom_theme_extension.dart';
import
'package:whatsapp_messenger/common/helper/show_alert_dialog.dart';
import
'package:whatsapp_messenger/common/widgets/custom_elevated_button.dart';
import
'package:whatsapp_messenger/feature/auth/controller/auth_controller.dart';
import
'package:whatsapp_messenger/feature/auth/widgets/custom_text_field.dart';

import '../common/utills/colours.dart';
import
'../common/widgets/custom_icon_button.dart';

class LoginPage extends
ConsumerStatefulWidget {
  const LoginPage({super.key});

  @override
  ConsumerState<LoginPage> createState()
=> _LoginPageState();
}

class _LoginPageState extends
ConsumerState<LoginPage> {
```

```
late TextEditingController
countryNameController;
late TextEditingController
countryCodeController;
late TextEditingController
phoneNumberController;

sendCodeToPhone() {
  final phoneNumber =
phoneNumberController.text;
  final countryName =
countryNameController.text;
  final countryCode =
countryCodeController.text;

  if (phoneNumber.isEmpty) {
    return showAlertDialog(
      context: context,
      message: "Please enter your phone
number",
    );
  } else if (phoneNumber.length < 9) {
    return showAlertDialog(
      context: context,
      message:
        "The phone number you entered is
too short for the country:
$countryName\n\nInclude your area code if
you haven't",
    );
  } else if (phoneNumber.length > 10) {
    return showAlertDialog(
      context: context,
      message:
        "The phone number you entered is
too long for the country: $countryName",
    );
  }
}
```



```

        ref.read(authControllerProvider).sendSms
        Code(
            context: context,
            phoneNumber:
            "+$countryCode$phoneNumber",
        );
    }

```

```

showCountryPickerBottomSheet() {
    showCountryPicker(
        context: context,
        showPhoneCode: true,
        favorite: ['ET'],
        countryListTheme:
        CountryListThemeData(
            bottomSheetHeight: 600,
            backgroundColor:
            Theme.of(context).colorScheme.surface,
            flagSize: 22,
            borderRadius:
            BorderRadius.circular(20),
            textStyle: TextStyle(color:
            context.theme.greyColor),
            inputDecoration: InputDecoration(
                labelStyle: TextStyle(color:
                context.theme.greyColor),
                prefixIcon: const Icon(
                    Icons.language,
                    color: Colors.greenDark,
                ),
                hintText: 'Search country by code or
                name',
                enabledBorder:
                UnderlineInputBorder(
                    borderSide: BorderSide(
                        color:
                        context.theme.greyColor!.withOpacity(.2),
                    ),
                ),
                focusedBorder: const
                UnderlineInputBorder(
                    borderSide: BorderSide(

```

```

            color: Colors.greenDark,
        ),
    ),
    ),
    ),
    onSelect: (country) {
        countryNameController.text =
        country.name;
        countryCodeController.text =
        country.phoneCode;
    },
);
}

@override
void initState() {
    countryNameController =
    TextEditingController(text: 'Ethiopia');
    countryCodeController =
    TextEditingController(text: '251');
    phoneNumberController =
    TextEditingController();
    super.initState();
}

@override
void dispose() {
    countryNameController.dispose();
    countryCodeController.dispose();
    phoneNumberController.dispose();
    super.dispose();
}

@override
Widget build(BuildContext context) {
    return Scaffold(
        appBar: AppBar(
            backgroundColor:
            Theme.of(context).scaffoldBackgroundColo
            r,
            elevation: 0,

```

```

        title: Text(
          'Enter your phone number',
          style: TextStyle(
            color:
context.theme.authAppBarTextColor,
          ),
        ),
        centerTitle: true,
        actions: [
          CustomIconButton(
            onPressed: () {},
            icon: Icons.more_vert,
          ),
        ],
      ),
      body: Column(
        children: [
          Padding(
            padding: const
EdgeInsets.symmetric(horizontal: 20,
vertical: 10),
            child: RichText(
              textAlign: TextAlign.center,
              text: TextSpan(
                text: 'WhatsApp will need to
verify your number. ',
                style: TextStyle(
                  color: context.theme.greyColor,
                  height: 1.5,
                ),
              children: [
                TextSpan(
                  text: "What's my number?",
                  style: TextStyle(
                    color:
context.theme.blueColor,
                  ),
                ),
              ],
            ),
          ),
        ],
      ),

```

```

        ),
      ),
      const SizedBox(height: 10),
      Padding(
        padding: const
EdgeInsets.symmetric(horizontal: 50),
        child: CustomTextField(
          onTap:
showCountryPickerBottomSheet,
          controller: countryNameController,
          readOnly: true,
          suffixIcon: const Icon(
            Icons.arrow_drop_down,
            color: Colors.greenDark,
            size: 22,
          ),
        ),
      ),
      const SizedBox(height: 10),
      Padding(
        padding: const
EdgeInsets.symmetric(horizontal: 50),
        child: Row(
          children: [
            SizedBox(
              width: 70,
              child: CustomTextField(
                onTap:
showCountryPickerBottomSheet,
                controller:
countryCodeController,
                prefixText: '+',
                readOnly: true,
              ),
            ),
            const SizedBox(width: 10),
            Expanded(
              child: CustomTextField(
                controller:
phoneNumberController,

```

```

        hintText: 'phone number',
        textAlign: TextAlign.left,
        keyboardType:
TextInputType.number,
    ),
    ),
  ],
),
),
const SizedBox(height: 20),
Text(
  'Carrier charges may apply',
  style: TextStyle(
    color: context.theme.greyColor,
  ),

```

```

    ),
  ],
),
floatingActionButtonLocation:
FloatingActionButtonLocation.centerFloat,
floatingActionButton:
CustomElevatedButton(
  onPressed: sendCodeToPhone,
  text: 'NEXT',
  buttonWidth: 90,
),
);
}
}

```

UserInfo Page:

```

import 'dart:io';
import 'dart:typed_data';

import 'package:flutter/material.dart';
import
'package:flutter_riverpod/flutter_riverpod.da
rt';
import
'package:image_picker/image_picker.dart';
import
'package:whatsapp_messenger/common/exte
nsion/custom_theme_extension.dart';
import
'package:whatsapp_messenger/common/hel
per/show_alert_dialog.dart';
import
'package:whatsapp_messenger/common/util
s/colors.dart';
import
'package:whatsapp_messenger/common/wid
gets/custom_elevated_button.dart';
import
'package:whatsapp_messenger/common/wid
gets/custom_icon_button.dart';

```

```

import
'package:whatsapp_messenger/common/wid
gets/short_h_bar.dart';
import
'package:whatsapp_messenger/feature/auth/
controller/auth_controller.dart';
import
'package:whatsapp_messenger/feature/auth/
pages/image_picker_page.dart';
import
'package:whatsapp_messenger/feature/auth/
widgets/custom_text_field.dart';

class UserInfoPage extends
ConsumerStatefulWidget {
  const UserInfoPage({super.key,
this.profileImageUrl});

  final String? profileImageUrl;

  @override
  ConsumerState<UserInfoPage>
createState() => _UserInfoPageState();
}

```

```

class _UserInfoPageState extends
ConsumerState<UserInfoPage> {
  File? imageCamera;
  Uint8List? imageGallery;
  late TextEditingController
usernameController;

  @override
  void initState() {
    super.initState();
    usernameController =
TextEditingController();
  }

  @override
  void dispose() {
    usernameController.dispose();
    super.dispose();
  }

  Future<void> saveUserDataToFirebase()
async {
    String username =
usernameController.text.trim();

    if (username.isEmpty) {
      showAlertDialog(context: context,
message: 'Please provide a username');
      return;
    }
    if (username.length < 3 || username.length
> 20) {
      showAlertDialog(
        context: context, message: 'Username
length should be between 3-20');
      return;
    }

    await

```

```

ref.read(authControllerProvider).saveUserIn
foToFirestore(
  username: username,
  profileImage:
    imageCamera ?? imageGallery ??
widget.profileImageUrl ?? "",
  context: context,
);
}

void imagePickerTypeBottomSheet() {
  showModalBottomSheet(
    context: context,
    builder: (context) {
      return Padding(
        padding: const
EdgeInsets.symmetric(vertical: 10),
        child: Column(
          mainAxisAlignment: MainAxisAlignment.min,
          children: [
            const ShortHBar(),
            Padding(
              padding: const
EdgeInsets.symmetric(horizontal: 20),
              child: Row(
                children: [
                  const Text(
                    'Profile photo',
                    style:
                      TextStyle(fontSize: 20,
fontWeight: FontWeight.w500),
                  ),
                  const Spacer(),
                  CustomIconButton(
                    onPressed: () =>
Navigator.pop(context),
                    icon: Icons.close,
                  ),
                ],
              ),
            ),
          ],
        ),
      ),
    ),
  ),
)

```

```

    ),
    Divider(color:
context.theme.greyColor!.withOpacity(.3)),
    Row(
      mainAxisAlignment:
MainAxisAlignment.spaceEvenly,
      children: [
        imagePickerIcon(
          onTap: pickImageFromCamera,
          icon:
Icons.camera_alt_rounded,
          text: 'Camera',
        ),
        imagePickerIcon(
          onTap: pickImageFromGallery,
          icon:
Icons.photo_camera_back_rounded,
          text: 'Gallery',
        ),
      ],
    ),
  ],
),
);
},
);
}

```

```

Future<void> pickImageFromCamera()
async {
  Navigator.pop(context);
  final image = await
ImagePicker().pickImage(source:
ImageSource.camera);
  if (image != null) {
    setState() {
      imageCamera = File(image.path);
      imageGallery = null;
    };
  }
}

```

```

Future<void> pickImageFromGallery()
async {
  Navigator.pop(context);
  final image = await Navigator.push(
    context,
    MaterialPageRoute(builder: (context) =>
const ImagePickerPage()),
  );
  if (image != null) {
    setState() {
      imageGallery = image;
      imageCamera = null;
    };
  }
}

```

```

Widget imagePickerIcon({
  required VoidCallback onTap,
  required IconData icon,
  required String text,
}) {
  return Column(
    children: [
      CustomIconButton(
        onPressed: onTap,
        icon: icon,
        iconColor: Colors.greenDark,
        minWidth: 50,
        border: Border.all(color:
context.theme.greyColor!.withOpacity(.2)),
      ),
      const SizedBox(height: 5),
      Text(text, style: TextStyle(color:
context.theme.greyColor)),
    ],
  );
}

```

```

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      backgroundColor:
Theme.of(context).colorScheme.surface,
      elevation: 0,
      title: Text(
        'Profile info',
        style: TextStyle(color:
context.theme.authAppBarTextColor),
      ),
      centerTitle: true,
    ),
    body: SingleChildScrollView(
      padding: const
EdgeInsets.symmetric(horizontal: 20),
      child: Column(
        children: [
          Text(
            'Please provide your name and an
optional profile photo',
            textAlign: TextAlign.center,
            style: TextStyle(color:
context.theme.greyColor),
          ),
          const SizedBox(height: 40),
          GestureDetector(
            onTap:
imagePickerTypeBottomSheet,
            child: Container(
              padding: const EdgeInsets.all(26),
              decoration: BoxDecoration(
                shape: BoxShape.circle,
                color:
context.theme.photoIconBgColor,
                border: Border.all(
                  color: (imageCamera == null
&&
imageGallery == null &&
widget.profileImageUrl ==

```

```

null)
              ? Colors.transparent
              :
context.theme.greyColor!.withOpacity(.4),
            ),
            image: (imageCamera != null ||
imageGallery != null ||
widget.profileImageUrl !=
null)
              ? DecorationImage(
                fit: BoxFit.cover,
                image: imageGallery != null
?
MemoryImage(imageGallery!)
: widget.profileImageUrl
!= null
?
NetworkImage(widget.profileImageUrl!)
:
FileImage(imageCamera!) as
ImageProvider,
              )
              : null,
          ),
          child: Icon(
            Icons.add_a_photo_rounded,
            size: 48,
            color: (imageCamera == null &&
imageGallery == null &&
widget.profileImageUrl ==
null)
              ?
context.theme.photoIconColor
              : Colors.transparent,
          ),
        ),
        const SizedBox(height: 40),
        CustomTextField(
          controller: usernameController,
          hintText: 'Type your name here',

```

```

        autoFocus: true),
    ],
  ),
),
floatingActionButton:
CustomElevatedButton(
  onPressed: saveUserDataToFirestore,

```

```

text: 'NEXT', buttonWidth: 90),
  );
}
}

```

Verification Page:

```

import 'package:flutter/material.dart';
import
'package:flutter_riverpod/flutter_riverpod.dart';
import
'package:whatsapp_messenger/common/extension/custom_theme_extension.dart';
import
'package:whatsapp_messenger/feature/auth/controller/auth_controller.dart';
import
'package:whatsapp_messenger/feature/auth/widgets/custom_text_field.dart';

class VerificationPage extends
ConsumerWidget {
  const VerificationPage({
    super.key,
    required this.smsCodeId,
    required this.phoneNumber,
  });

  final String smsCodeId;
  final String phoneNumber;

  Future<void> verifySmsCode(
    BuildContext context,
    WidgetRef ref,
    String smsCode,
  ) async {
    await
    ref.read(authControllerProvider).verifySmsCode
    (
      context: context,
      smsCodeId: smsCodeId,

```

```

      smsCode: smsCode,
    );
  }

  @override
  Widget build(BuildContext context,
  WidgetRef ref) {
    return Scaffold(
      appBar: AppBar(
        backgroundColor:
        Theme.of(context).scaffoldBackgroundColor,
        elevation: 0,
        centerTitle: true,
        title: Text(
          'Verify your number',
          style: TextStyle(
            color:
            context.theme.authAppBarTextColor,
          ),
        ),
        actions: [
          IconButton(
            onPressed: () {},
            icon: const Icon(Icons.more_vert),
          ),
        ],
      ),
      body: SingleChildScrollView(
        padding: const
        EdgeInsets.symmetric(horizontal: 20),
        child: Column(
          children: [
            Padding(
              padding: const

```

```

EdgeInsets.symmetric(horizontal: 10),
  child: RichText(
    textAlign: TextAlign.center,
    text: TextSpan(
      style: TextStyle(color:
context.theme.greyColor),
    children: [
      const TextSpan(
        text:
          "You've tried to register before
requesting an SMS or Call with your code.",
      ),
      TextSpan(
        text: " Wrong number?",
        style: TextStyle(
          color: context.theme.blueColor,
        ),
      ),
    ],
  ),
),
const SizedBox(height: 20),
Container(
  padding: const
EdgeInsets.symmetric(horizontal: 80),
  child: CustomTextField(
    hintText: "- - - - -",
    fontSize: 30,
    autoFocus: true,
    keyboardType:
TextInputType.number,
    onChanged: (value) async {
      if (value.length == 6) {
        await verifySmsCode(context, ref,
value);
      }
    },
  ),
),
const SizedBox(height: 20),
Text(
  'Enter 6-digit code',

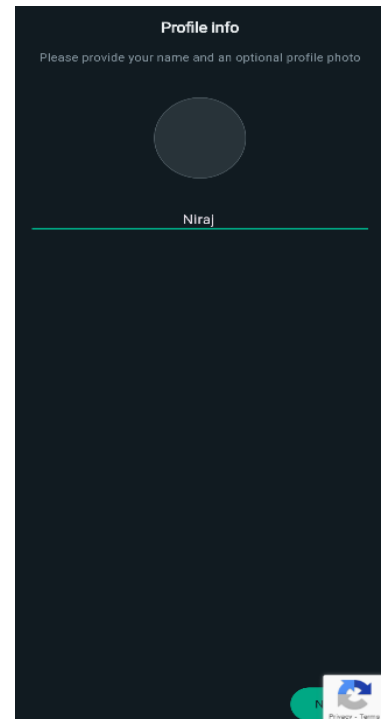
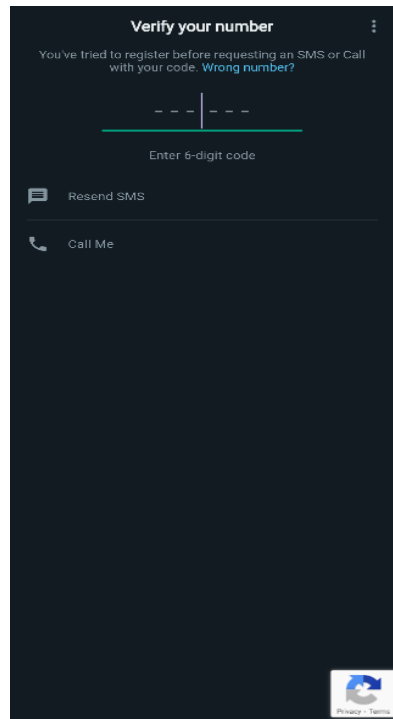
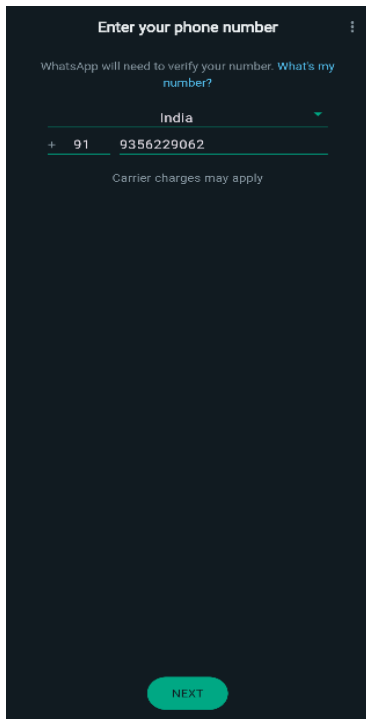
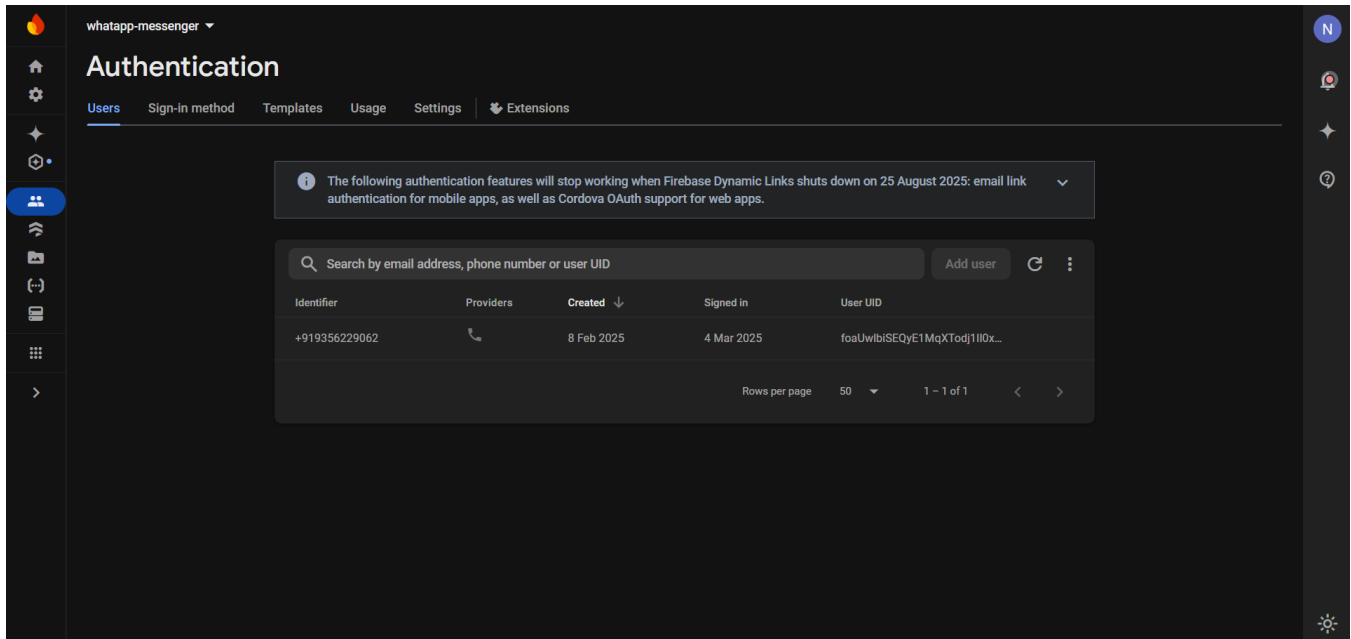
```

```

    style: TextStyle(color:
context.theme.greyColor),
  ),
  const SizedBox(height: 30),
  Row(
    children: [
      Icon(Icons.message, color:
context.theme.greyColor),
      const SizedBox(width: 20),
      Text(
        'Resend SMS',
        style: TextStyle(
          color: context.theme.greyColor,
        ),
      ),
    ],
  ),
  const SizedBox(height: 10),
  Divider(
    color:
context.theme.greyColor!.withOpacity(.2),
  ),
  const SizedBox(height: 10),
  Row(
    children: [
      Icon(Icons.phone, color:
context.theme.greyColor),
      const SizedBox(width: 20),
      Text(
        'Call Me',
        style: TextStyle(
          color: context.theme.greyColor,
        ),
      ),
    ],
  ),
),
);
}
}

```


After Registering, the user details is saved in the database.



User details get fetched from the database in the profile page.

The screenshot shows the Google Cloud Firestore console for a project named 'whatapp-messenger'. The 'Data' tab is selected, and the breadcrumb path is 'users > foaUwlbISEQyE1MqXTodj1lI0xm1'. The document is expanded, showing the following fields:

- active: true
- groupId
- lastSeen: 1741102062460
- phoneNumber: "+919356229062"
- profileImageUrl: ""
- uid: "foaUwlbISEQyE1MqXTodj1lI0xm1"
- username: "ABC"

The URL at the bottom of the browser window is: https://console.firebase.google.com/u/0/project/whatapp-messenger-8275a/firestore7fb_gclid=CjwKCAIA5pq-BhBuEiwAvkzVZctzmR79BRs232e4HoELVSN04ihCycmdUkn0f9hih7OW3Pfr-ECuRoChAUQAvD_BwE

MAD & PWA Lab

Journal

Experiment No.	07
Experiment Title.	To write meta data of your Ecommerce PWA in a Web app manifest file to enable “add to homescreen feature”.
Roll No.	24
Name	Niraj Kothawade
Class	D15A/D15B
Subject	MAD & PWA Lab
Lab Outcome	LO4: Understand various PWA frameworks and their requirements
Grade:	

Experiment No. 7

Niraj Kothawade

D15A - 24

Aim: To write meta data of your Ecommerce PWA in a Web app manifest file to enable “add to homescreen feature”.

Online Reference:

<https://developer.mozilla.org/en-US/docs/Web/Manifest>

<https://www.geeksforgeeks.org/making-a-simple-pwa-under-5-minutes/>

Theory:

Regular Web App

A regular web app is a website that is designed to be accessible on all mobile devices such that the content gets fit as per the device screen. It is designed using a web technology stack (HTML, CSS, JavaScript, Ruby, etc.) and operates via a browser. They offer various native-device features and functionalities. However, it entirely depends on the browser the user is using. In other words, it might be possible that you can access a native-device feature on Chrome but not on Safari or Mozilla Firefox because the browsers are incompatible with that feature.

Progressive Web App

Progressive Web App (PWA) is a regular web app, but some extras enable it to deliver an excellent user experience. It is a perfect blend of desktop and mobile application experience to give both platforms to the end-users.

Difference between PWAs vs. Regular Web Apps:

A Progressive Web is different and better than a Regular Web app with features like:

1. Native Experience

Though a PWA runs on web technologies (HTML, CSS, JavaScript) like a Regular web app, it gives user experience like a native mobile application. It can use most native device features, including push notifications, without relying on the browser or any other entity. It offers a seamless and integrated user experience that it is quite tough for one to differentiate between a PWA and a Native application by considering its look and feel.

2. Ease of Access

Unlike other mobile apps, PWAs do not demand longer download time and make memory space available for installing the applications. The PWAs can be shared and installed by a link, which cuts down the number of steps to install and use. These applications can easily keep an app icon on the user's home screen, making the app easily accessible to the users and helps the brands remain in the users' minds, and improving the chances of interaction.

3. Faster Services

PWAs can cache the data and serve the user with text stylesheets, images, and other web content even before the page loads completely. This lowers the waiting time for the end-users and helps the brands improve the user engagement and retention rate, which eventually adds value to their business.

4. Engaging Approach

As already shared, the PWAs can employ push notifications and other native device features more efficiently. Their interaction does not depend on the browser user uses. This eventually improves the chances of notifying the user regarding your services, offers, and other options related to your brand and keeping them hooked to your brand. In simpler words, PWAs let you maintain the user engagement and retention rate.

5. Updated Real-Time Data Access

Another plus point of PWAs is that these apps get updated on their own. They do not demand the end-users to go to the App Store or other such platforms to download the update and wait until installed.

In this app type, the web app developers can push the live update from the server, which reaches the apps residing on the user's devices automatically. Therefore, it is easier for the mobile app developer to provide the best of the updated functionalities and services to the end-users without forcing them to update their app.

6. Discoverable

PWAs reside in web browsers. This implies higher chances of optimizing them as per the Search Engine Optimization (SEO) criteria and improving the Google rankings like that in websites and other web apps.

7. Lower Development Cost

Progressive web apps can be installed on the user device like a native device, but it does not demand submission on an App Store. This makes it far more cost-effective than native mobile applications while offering the same set of functionalities.

Pros and cons of the Progressive Web App

The main features are:

- **Progressive** — They work for every user, regardless of the browser chosen because they are built at the base with progressive improvement principles.
- **Responsive** — They adapt to the various screen sizes: desktop, mobile, tablet, or dimensions that can later become available.
- **App-like** — They behave with the user as if they were native apps, in terms of interaction and navigation.
- **Updated** — Information is always up-to-date thanks to the data update process offered by service workers.
- **Secure** — Exposed over HTTPS protocol to prevent the connection from displaying information or altering the contents.
- **Searchable** — They are identified as “applications” and are indexed by search engines.
- **Reactivable** — Make it easy to reactivate the application thanks to capabilities such as web notifications.
- **Installable** — They allow the user to “save” the apps that he considers most useful with the corresponding icon on the screen of his mobile terminal (home screen) without having to face all the steps and problems related to the use of the app store.
- **Linkable** — Easily shared via URL without complex installations.

- **Offline** — Once more it is about putting the user before everything, avoiding the usual error message in case of weak or no connection. The PWA are based on two particularities: first of all the ‘skeleton’ of the app, which recalls the page structure, even if its contents do not respond and its elements include the header, the page layout, as well as an illustration that signals that the page is loading.

Weaknesses refer to:

- IOS support from version 11.3 onwards;
- Greater use of the device battery;
- Not all devices support the full range of PWA features (same speech for iOS and Android operating systems);
- It is not possible to establish a strong re-engagement for iOS users (URL scheme, standard web notifications);
- Support for offline execution is however limited;
- Lack of presence on the stores (there is no possibility to acquire traffic from that channel);
- There is no “body” of control (like the stores) and an approval process;
- Limited access to some hardware components of the devices;
- Little flexibility regarding “special” content for users (eg loyalty programs, loyalty, etc.).

Code:

manifest.json:-

```
{
  "name": "Xtrack",
  "short_name": "Xtrack",
  "start_url": "ani.html",
  "scope": "./",
  "icons": [
    {
      "src": "contract.png",
      "sizes": "192x192",
      "type": "image/png"
    },
    {
```

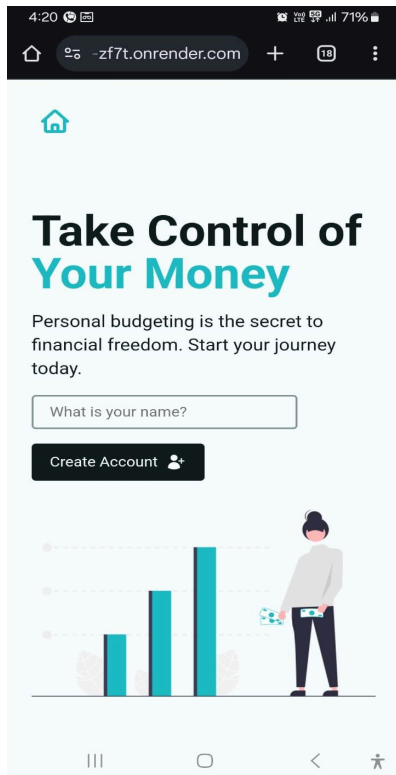
```
    "src": "contract.png",  
    "sizes": "512x512",  
    "type": "image/png"  
  }  
],  
  "theme_color": "#ffd31d",  
  "background_color": "#333",  
  "display": "standalone"  
}
```

Add the link tag to link to the manifest.json file

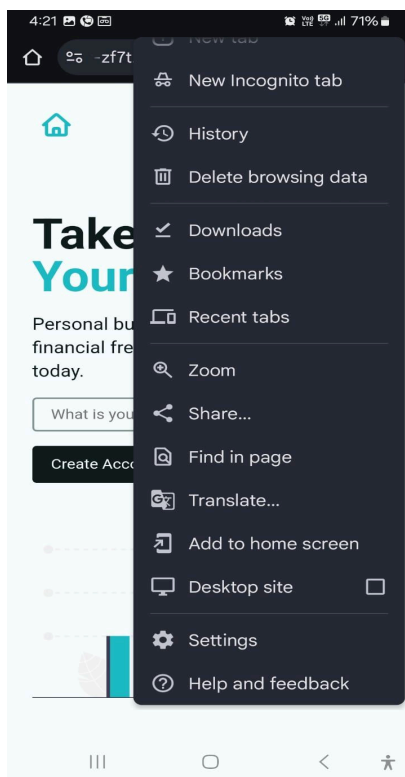
```
<html>  
  <head>  
    <link rel="manifest" href="manifest.json">  
    <script src="myscript.js"></script>  
    <title>Xtrack</title>  
    <style>
```

Output with Steps:

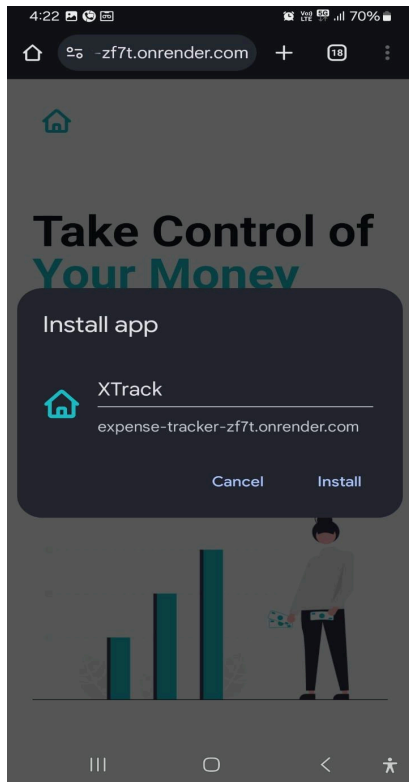
1. Go to google chrome and open your website link:



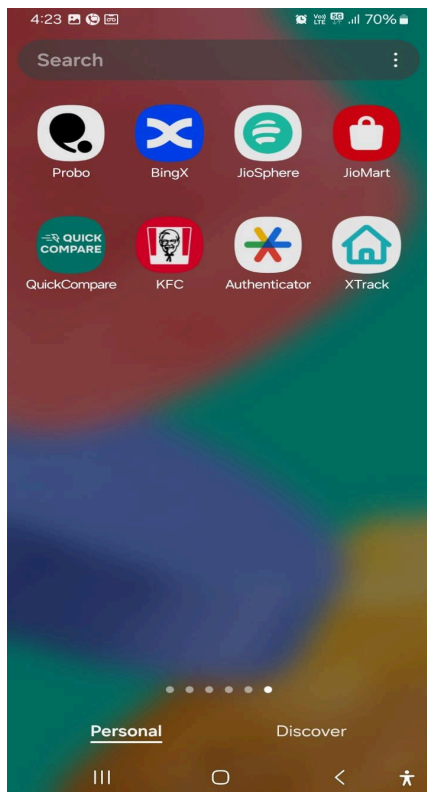
2. Then click on three dots at top right corner, find and click on add to home screen:



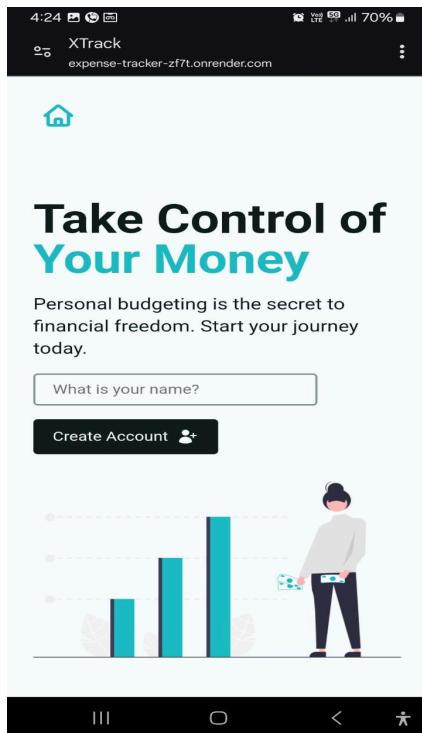
3. Then click on install:



4. Once installed you will see application in your mobile:



5. Final Output:



Conclusion:

Hence, we learnt how to write a metadata of our E-commerce website PWA in a Web App Manifest File to enable add to homescreen feature.

MAD & PWA Lab

Journal

Experiment No.	08
Experiment Title.	To code and register a service worker, and complete the install and activation process for a new service worker for the E-commerce PWA
Roll No.	24
Name	Niraj Kothawade
Class	D15A/D15B
Subject	MAD & PWA Lab
Lab Outcome	LO5: Design and Develop a responsive User Interface by applying PWA Design techniques
Grade:	

Experiment No. 8

Niraj Kothawade
D15A -24

Aim: To code and register a service worker, and complete the install and activation process for a new service worker for the E-commerce PWA.

Theory:

Service Worker

Service Worker is a script that works on browser background without user interaction independently. Also, It resembles a proxy that works on the user side. With this script, you can track network traffic of the page, manage push notifications and develop “offline first” web applications with Cache API.

Things to note about Service Worker:

- A service worker is a programmable network proxy that lets you control how network requests from your page are handled.
- Service workers only run over HTTPS. Because service workers can intercept network requests and modify responses, "man-in-the-middle" attacks could be very bad.
- The service worker becomes idle when not in use and restarts when it's next needed. You cannot rely on a global state persisting between events. If there is information that you need to persist and reuse across restarts, you can use IndexedDB databases.

What can we do with Service Workers?

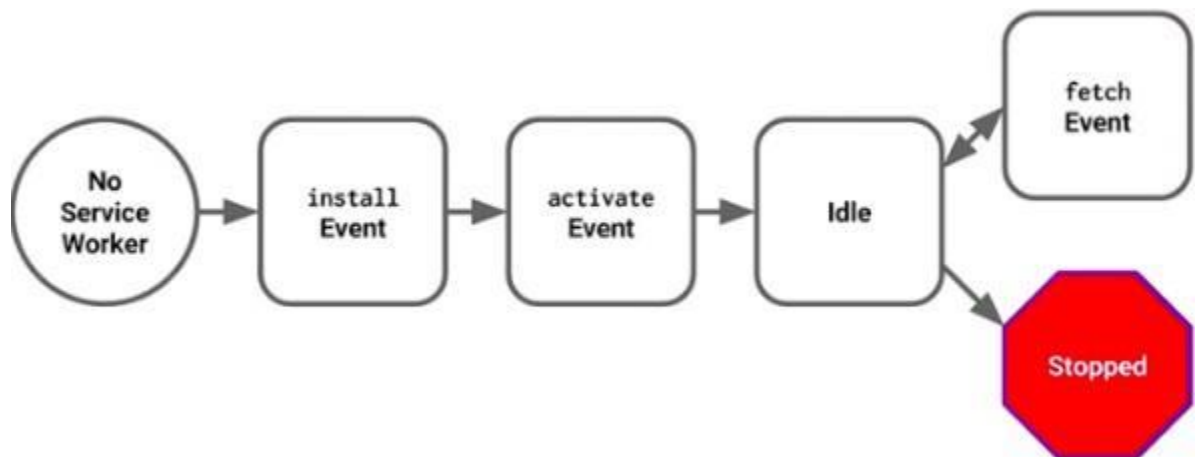
- You can dominate **Network Traffic**
You can manage all network traffic of the page and do any manipulations. For example, when the page requests a CSS file, you can send plain text as a response or when the page requests an HTML file, you can send a png file as a response. You can also send a true response too.
- You can **Cache**
You can cache any request/response pair with Service Worker and Cache API and you can access these offline content anytime.

- You can manage **Push Notifications**
You can manage push notifications with Service Worker and show any information message to the user.
- You can **Continue**
Although Internet connection is broken, you can start any process with Background Sync of Service Worker.

What can't we do with Service Workers?

- You can't access the **Window**
You can't access the window, therefore, You can't manipulate DOM elements. But, you can communicate to the window through post Message and manage processes that you want.
- You can't work it on **80 Port**
Service Worker just can work on HTTPS protocol. But you can work on localhost during development.

Service Worker Cycle



A service worker goes through three steps in its life cycle:

- Registration
- Installation
- Activation

Registration

To install a service worker, you need to register it in your main JavaScript code.

Registration tells the browser where your service worker is located, and to start installing it in the background. Let's look at an example:

main.js

```
if ('serviceWorker' in navigator) {  
  navigator.serviceWorker.register('/  
  service-worker.js')  
  .then(function(registration) {  
    console.log('Registration successful, scope is:', registration.scope);  
  })  
  .catch(function(error) {  
    console.log('Service worker registration failed, error:', error);  
  })  
}
```

This code starts by checking for browser support by examining **navigator.serviceWorker**. The service worker is then registered with `navigator.serviceWorker.register`, which returns a promise that resolves when the service worker has been successfully registered. The scope of the service worker is then logged with `registration.scope`. If the service worker is already installed, `navigator.serviceWorker.register` returns the registration object of the currently active service worker.

The scope of the service worker determines which files the service worker controls, in other words, from which path the service worker will intercept requests. The default scope is the location of the service worker file, and extends to all directories below. So if `service-worker.js` is located in the root directory, the service worker will control requests from all files at this domain.

You can also set an arbitrary scope by passing in an additional parameter when registering. For

example: `main.js`

```
navigator.serviceWorker.register('/service-worker.js', {  
  scope: '/app/'  
});
```

In this case we are setting the scope of the service worker to `/app/`, which means the service worker will control requests from pages like `/app/`, `/app/lower/` and `/app/lower/lower`, but not from pages like `/app` or `/`, which are higher.

If you want the service worker to control higher pages e.g. `/app` (without the trailing slash) you can indeed change the scope option, but you'll also need to set the Service-Worker-Allowed HTTP Header in your server config for the request serving the service worker script.

`main.js`

```
navigator.serviceWorker.register('/app/service-worker.js'  
  , { scope: '/app'  
  })
```


Installation

Once the browser registers a service worker, installation can be attempted. This occurs if the service worker is considered to be new by the browser, either because the site currently doesn't have a registered service worker, or because there is a byte difference between the new service worker and the previously installed one.

A service worker installation triggers an install event in the installing service worker. We can include an install event listener in the service worker to perform some task when the service worker installs. For instance, during the install, service workers can precache parts of a web app so that it loads instantly the next time a user opens it (see caching the application shell). So, after that first load, you're going to benefit from instant repeat loads and your time to interactivity is going to be even better in those cases. An example of an installation event listener looks like this:

service-worker.js

```
// Listen for install event, set callback  
self.addEventListener('install', function(event) {  
  // Perform some task  
});
```

Activation

Once a service worker has successfully installed, it transitions into the activation stage. If there are any open pages controlled by the previous service worker, the new service worker enters a waiting state. The new service worker only activates when there are no longer any pages loaded that are still using the old service worker. This ensures that only one version of the service worker is running at any given time.

When the new service worker activates, an activate event is triggered in the activating service worker. This event listener is a good place to clean up outdated caches (see the Offline Cookbook for an example).

service-worker.js

```
self.addEventListener('activate', function(event) {  
  // Perform some task  
});
```

Once activated, the service worker controls all pages that load within its scope, and starts listening for events from those pages. However, pages in your app that were loaded before the service worker activation will not be under service worker control. The new service worker will only take over when you close and reopen your app, or if the service worker calls **clients.claim()**. Until then, requests from this page will not be intercepted by the new service worker. This is intentional as a way to ensure consistency in your site.

Code

sw.js

```
self.addEventListener("install", function (event)
  { event.waitUntil(preLoad());
});

var filesToCache = [
  '/',
  '/menu',
  '/contactUs',
  '/offline.html',
];

var preLoad = function () {
  return caches.open("offline").then(function (cache) {
    // caching index and important routes
    return cache.addAll(filesToCache);
  });
};

self.addEventListener("fetch", function (event) {
  event.respondWith(checkResponse(event.request).catch(function
  () {
    return returnFromCache(event.request);
  }));
  event.waitUntil(addToCache(event.request));
});

var checkResponse = function (request) {
  return new Promise(function (fulfill, reject)
  {
    fetch(request).then(function (response)
    { if (response.status !== 404) {
```

```

        fulfill(response)
    } else {
        reject();
    }
}, reject);
});
};

```

```

var addToCache = function (request) {
    return caches.open("offline").then(function (cache)
    { return fetch(request).then(function (response)
    {
        return cache.put(request, response);
    });
    });
};

```

```

var returnFromCache = function (request) {
    return caches.open("offline").then(function (cache) {
        return cache.match(request).then(function
        (matching) {
            if (!matching || matching.status === 404)
            { return cache.match("offline.html");
            } else {
                return matching;
            }
        });
    });
};
}

```

The image shows a web browser window with a landing page for 'Take Control of Your Money'. The page features a teal house icon, the title 'Take Control of Your Money' in large black and teal text, and a subtitle 'Personal budgeting is the secret to financial freedom. Start your journey today.' Below this is a form with the placeholder text 'What is your name?' and a 'Create Account' button with a user icon. At the bottom, there is a bar chart with three bars of increasing height and an illustration of a person holding a wallet.

Overlaid on the right side of the browser window is the Chrome DevTools 'Application' tab. The left sidebar shows the 'Storage' section expanded, with 'Cache storage' selected. The main panel displays the 'http://localhost:5173' origin. Under 'Cache storage', the 'expense-tracker-v1...' entry is selected. A table lists the cached resources:

#	Name	Response-Type	Content-Type	Content-Length	Time Cached	Vary Header
0	/	basic	text/html	720	25/03/2025, 1...	
1	/dist/assets/index-BPUHWWJbD.js	basic	text/javascript	1,427,684	25/03/2025, 1...	
2	/dist/assets/index-BvWQCeOo.css	basic	text/javascript	24,413	25/03/2025, 1...	
3	/index.html	basic	text/html	721	25/03/2025, 1...	
4	/manifest.json	basic	application/json	318	25/03/2025, 1...	

Below the table, a message states 'No cache entry selected' and 'Select a cache entry above to preview'. At the bottom, it says 'Total entries: 5'.

Sample Code with Output

Index.html

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/favicon.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <link rel="manifest" href="manifest.json" />
    <title>XTrack</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

main.jsx

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import App from './App.jsx'
import './index.css'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)

// Service Worker Registration
if ('serviceWorker' in navigator) {
  window.addEventListener('load', () => {
    navigator.serviceWorker.register('public/service-worker.js') // Path to your service worker
    file
      .then((registration) => {
        console.log('[Service Worker] Registered with scope:', registration.scope);

        // Check if the service worker is active
        if (registration.active) {
          console.log('[Service Worker] Active');
```

```

    }

    // Check if the service worker is installing
    if (registration.installing) {
        console.log('[Service Worker] Installing');
    }

    // Check if the service worker is waiting
    if (registration.waiting) {
        console.log('[Service Worker] Waiting');
    }

    // Listen for state changes
    registration.addEventListener('updatefound', () => {
        console.log('[Service Worker] Update found');
        const installingWorker = registration.installing;
        if (installingWorker) {
            installingWorker.addEventListener('statechange', () => {
                if (installingWorker.state === 'installed') {
                    if (navigator.serviceWorker.controller) {
                        console.log('[Service Worker] New content is available and will be used when
all tabs for this page are closed.');
```

// You can prompt the user to reload here if needed

```

                    } else {
                        console.log('[Service Worker] Content is cached for offline use.');
```

}

```

                }
            });
        }
    });
}

.catch((error) => {
    console.error('[Service Worker] Registration failed:', error);
});
});
} else {
    console.warn('[Service Worker] Not supported in this browser.');
```

}

service-worker.js

```
// public/service-worker.js
```

```
const CACHE_NAME = 'expense-tracker-v1'; // Change this when you update the service worker
```

```
const urlsToCache = [
```

```
  '/', // Cache the root URL (index.html)
```

```
  '/index.html',
```

```
  '/manifest.json', // if you have one
```

```
  // Correct paths to your built assets
```

```
  'dist/assets/index-BPUHWJbD.js', // Replace with the actual path
```

```
  'dist/assets/index-BvWQCeOo.css',
```

```
];
```

```
// Install Event: Cache static assets
```

```
self.addEventListener('install', (event) => {
```

```
  console.log('[Service Worker] Install');
```

```
  event.waitUntil(
```

```
    caches.open(CACHE_NAME)
```

```
      .then((cache) => {
```

```
        console.log('[Service Worker] Caching app shell');
```

```
        return cache.addAll(urlsToCache);
```

```
      })
```

```
      .catch((error) => {
```

```
        console.error('[Service Worker] Caching failed:', error);
```

```
      })
```

```
  );
```

```
});
```

```
// Activate Event: Clean up old caches
```

```
self.addEventListener('activate', (event) => {
```

```
  console.log('[Service Worker] Activate');
```

```
  event.waitUntil(
```

```
    caches.keys().then((cacheNames) => {
```

```
      return Promise.all(
```

```
        cacheNames.map((cacheName) => {
```

```
          if (cacheName !== CACHE_NAME) {
```

```
            console.log('[Service Worker] Deleting old cache:', cacheName);
```

```
            return caches.delete(cacheName);
```

```
          }
```

```
        })
```

```
      );
```

```
    });
```

```
);  
});  
  
// Fetch Event: Serve from cache or network  
self.addEventListener('fetch', (event) => {  
  console.log('[Service Worker] Fetch', event.request.url);  
  event.respondWith(  
    caches.match(event.request)  
      .then((response) => {  
        // Cache hit - return response  
        if (response) {  
          console.log('[Service Worker] Found in cache:', event.request.url);  
          return response;  
        }  
        // Not in cache - return fetch request  
        console.log('[Service Worker] Not found in cache, fetching:', event.request.url);  
        return fetch(event.request);  
      })  
  );  
});
```

Conclusion: Successfully installed and activated the process of new service worker for pwa.

MAD & PWA Lab

Journal

Experiment No.	09
Experiment Title.	To implement Service worker events like fetch, sync and push for E-commerce PWA
Roll No.	24
Name	Niraj Kothawade
Class	D15A/D15B
Subject	MAD & PWA Lab
Lab Outcome	LO5: Design and Develop a responsive User Interface by applying PWA Design techniques
Grade:	

Experiment No. 9

Niraj Kothawade
D15A - 24

Aim: To implement Service worker events like fetch, sync and push for E-commerce PWA.

Theory:

Service Worker

Service Worker is a script that works on browser background without user interaction independently. Also, It resembles a proxy that works on the user side. With this script, you can track network traffic of the page, manage push notifications and develop “offline first” web applications with Cache API.

Things to note about Service Worker:

- A service worker is a programmable network proxy that lets you control how network requests from your page are handled.
- Service workers only run over HTTPS. Because service workers can intercept network requests and modify responses, "man-in-the-middle" attacks could be very bad.
- The service worker becomes idle when not in use and restarts when it's next needed. You cannot rely on a global state persisting between events. If there is information that you need to persist and reuse across restarts, you can use IndexedDB databases.
- Service workers make extensive use of promises, so if you're new to promises, then you should stop reading this and check out Promises, an introduction.

Fetch Event

You can track and manage page network traffic with this event. You can check existing cache, manage “cache first” and “network first” requests and return a response that you want.

Of course, you can use many different methods but you can find in the following example a “cache first” and “network first” approach. In this example, if the request's

and current location's origin are the same (Static content is requested.), this is called "cacheFirst" but if you request a targeted external URL, this is called "networkFirst".

- **CacheFirst** - In this function, if the received request has cached before, the cached response is returned to the page. But if not, a new response requested from the network.
- **NetworkFirst** - In this function, firstly we can try getting an updated response from the network, if this process completed successfully, the new response will be cached and returned. But if this process fails, we check whether the request has been cached before or not. If a cache exists, it is returned to the page, but if not, this is up to you. You can return dummy content or information messages to the page.

```
self.addEventListener("fetch", function (event) {
  const req = event.request;
  const url = new URL(req.url);

  if (url.origin === location.origin) {
    event.respondWith(cacheFirst(req));
  }
  else {
    event.respondWith(networkFirst(req));
  }
});

async function cacheFirst(req) {
  return await caches.match(req) || fetch(req);
}

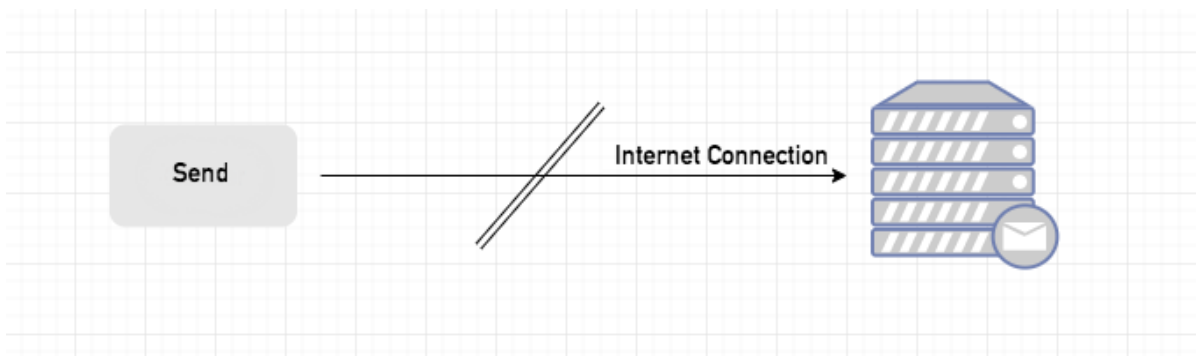
async function networkFirst(req) {
  const cache = await caches.open("pwa-dynamic");
  try {
    const res = await fetch(req);
    cache.put(req, res.clone());
    return res;
  } catch (error) {
    const cachedResponse = await cache.match(req);
    return cachedResponse || await caches.match("./noconnection.json");
  }
}
```

Sync Event

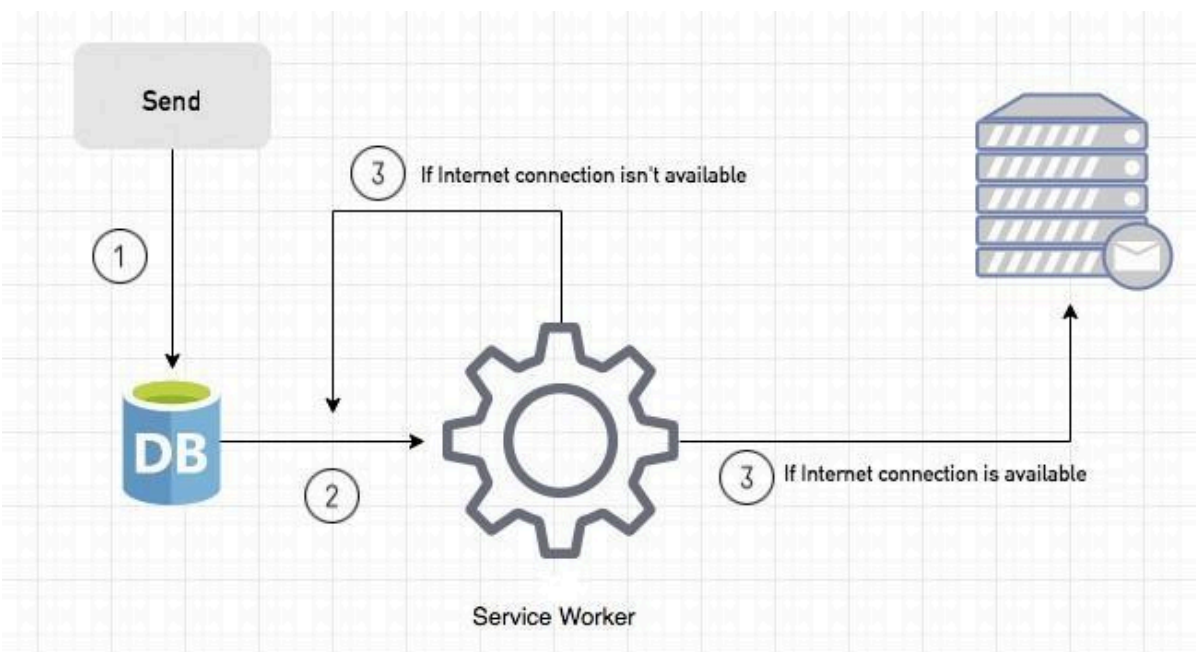
Background Sync is a Web API that is used to delay a process until the Internet connection is stable. We can adapt this definition to the real world; there is an e-mail client application that works on the browser and we want to send an email with this tool. Internet connection is broken while we are writing e-mail content and we didn't realize it. When completing the writing, we click the send button.

Here is a job for the Background Sync.

The following view shows the classical process of sending email to us. If the Internet Connection is broken, we can't send any content to Mail Server.



Here, you can create any scenario for yourself. A sample is in the following for this case.



1. When we click the “send” button, email content will be saved to IndexedDB.
2. Background Sync registration.
3. **If the Internet connection is available**, all email content will be read and sent to Mail Server.

If the Internet connection is unavailable, the service worker waits until the connection is available even though the window is closed. When it is available, email content will be sent to Mail Server.

You can see the working process within the following code block.

Event Listener for Background Sync Registration

```
document.querySelector("button").addEventListener("click", async () => {
  var swRegistration = await navigator.serviceWorker.register("sw.js");
  swRegistration.sync.register("helloSync").then(function () {
    console.log("helloSync success [main.js]");
  });
});
```

Event Listener for sw.js

```
self.addEventListener('sync', event => {
  if (event.tag == 'helloSync') {
    console.log("helloSync [sw.js]");
  }
});
```

Push Event

This is the event that handles push notifications that are received from the server. You can apply any method with received data.

We can check in the following example.

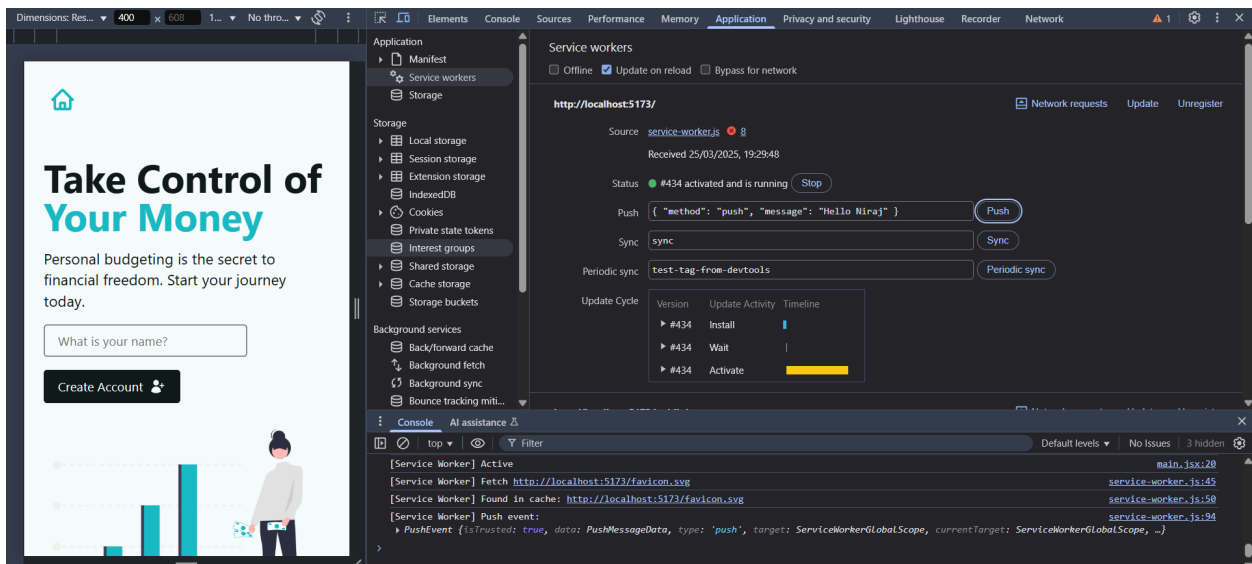
“Notification.requestPermission();” is the necessary line to show notification to the user. If you don’t want to show any notification, you don’t need this line.

In the following code block is in sw.js file. You can handle push notifications with this event. In this example, I kept it simple. We send an object that has “method” and

“message” properties. If the method value is “pushMessage”, we open the information notification with the “message” property.

```
self.addEventListener('push', event => {  
  if (event && event.data) {  
    var data = event.data.json();  
    if (data.method === "pushMessage") {  
      event.waitUntil(self.registration.showNotification("Test App", {  
        body: data.message  
      }));  
    }  
  }  
});
```

You can use Application Tab from Chrome Developer Tools for testing push notification.



Code:

service-worker.js

```
// public/service-worker.js

const CACHE_NAME = 'expense-tracker-v1'; // Change this when you update the service
worker

const urlsToCache = [
  '/', // Cache the root URL (index.html)
  '/index.html',
  '/manifest.json', // if you have one
  // Correct paths to your built assets
  'dist/assets/index-BPUHWJbD.js', // Replace with the actual path
  'dist/assets/index-BvWQCeOo.css',
];

self.addEventListener('install', (event) => {
  console.log('[Service Worker] Install');
  event.waitUntil(
    caches.open(CACHE_NAME)
      .then((cache) => {
        console.log('[Service Worker] Caching app shell');
        return cache.addAll(urlsToCache);
      })
      .catch((error) => {
        console.error('[Service Worker] Caching failed:', error);
      })
  );
});

self.addEventListener('activate', (event) => {
  console.log('[Service Worker] Activate');
  event.waitUntil(
    caches.keys().then((cacheNames) => {
```

```

return Promise.all(
  cacheNames.map((cacheName) => {
    if (cacheName !== CACHE_NAME) {
      console.log('[Service Worker] Deleting old cache:', cacheName);
      return caches.delete(cacheName);
    }
  })
);
});

// Fetch Event: Cache-first strategy
self.addEventListener('fetch', (event) => {
  console.log('[Service Worker] Fetch', event.request.url);
  event.respondWith(
    caches.match(event.request)
      .then((response) => {
        if (response) {
          console.log('[Service Worker] Found in cache:', event.request.url);
          return response;
        }
        return fetch(event.request)
          .then((response) => {
            if (!response || response.status !== 200 || response.type !== 'basic') {
              return response;
            }
            const responseToCache = response.clone();
            caches.open(CACHE_NAME)
              .then((cache) => {
                cache.put(event.request, responseToCache);
              });
            return response;
          });
      });
  );
});

```



```

    })

    .catch((error) => {

        console.error('[Service Worker] Fetch failed:', error);

        // Handle network errors (e.g., show offline message)

    });

})

);

});

// Sync Event: Handle background sync
self.addEventListener('sync', (event) => {

    console.log('[Service Worker] Sync event:', event.tag);

    if (event.tag === 'sync-expenses') {

        event.waitUntil(

            // Your background sync logic here (e.g., send expenses to server)
            syncExpenses()

                .then(() => {

                    console.log('[Service Worker] Expenses synced successfully');

                })

                .catch((error) => {

                    console.error('[Service Worker] Expense sync failed:', error);

                    // Handle sync errors (e.g., retry later)

                })

        );

    }

});

// Push Event: Handle push notifications
self.addEventListener('push', function(event) {

    console.log('[Service Worker] Push event:', event);

    let data;

    try {

```

```

    data = event.data ? event.data.json() : {};
  } catch (error) {
    console.error('Error parsing push notification data:', error);
    data = { title: 'Error', message: 'Could not parse notification data.' };
  }

  if (Notification.permission === 'granted') {
    event.waitUntil(
      self.registration.showNotification(data.title || 'Xtrack', {
        body: data.message || 'No message provided.',
        icon: data.icon || '/your-notification-icon.png', // Provide a default icon
      })
    );
  } else {
    console.log('Notification permission not granted. Notification will not be
shown.');
```

```

//Helper function for sync event
```

```

async function syncExpenses() {
  // Replace this with your actual expense syncing logic
  // This example just simulates a delay
  return new Promise((resolve) => {
    setTimeout(resolve, 2000); // Simulate a 2-second delay
  });
}
```

Output:

Fetchevent

The screenshot shows a web application on the left and Chrome DevTools on the right. The web application has a header with a home icon and a main heading "Take Control of Your Money". Below the heading is a subheading "Personal budgeting is the secret to financial freedom. Start your journey today." and a form with a text input "What is your name?" and a "Create Account" button. At the bottom, there is a bar chart and an illustration of a person. The Chrome DevTools interface shows the "Application" tab with the "Service workers" section expanded. The service worker is registered at "http://localhost:5173/" and is currently active. The "Update Cycle" section shows a timeline with three phases: "Install", "Wait", and "Activate". The "Console" tab shows the following log messages:

```
[Service Worker] Registered with scope: http://localhost:5173/public/
[Service Worker] Active
[Service Worker] Fetch http://localhost:5173/favicon.svg
[Service Worker] Found in cache: http://localhost:5173/favicon.svg
Fetch Successfull
```

Sync event

The screenshot shows a web application on the left and Chrome DevTools on the right. The web application has a header with the text "welcome back, Niraj". Below the header is a subheading "Personal budgeting is the secret to financial freedom." and a form with a text input "Budget Name" (with the example "e.g., Groceries") and a text input "Amount" (with the example "e.g., ₹350"). Below the form is a "Create Budget" button. The Chrome DevTools interface shows the "Application" tab with the "Service workers" section expanded. The service worker is registered at "http://localhost:5173/" and is currently active. The "Update Cycle" section shows a timeline with three phases: "Install", "Wait", and "Activate". The "Console" tab shows the following log messages:

```
[Service Worker] Fetch http://localhost:5173/favicon.svg
[Service Worker] Found in cache: http://localhost:5173/favicon.svg
Fetch Successfull
[Service Worker] Sync event: sync-expenses
[Service Worker] Expenses synced successfully
```

Push event

The screenshot displays a web browser window with a PWA interface on the left and the Chrome DevTools Application panel on the right. The PWA interface, titled "Take Control of Your Money", features a home icon, a text input for a name, a "Create Account" button, and a bar chart. The DevTools Application panel shows the "Service workers" section for the URL `http://localhost:5173/`. It indicates that the service worker `service-worker.js` is active and running on port 8. The "Push" event is configured with the message `{ "method": "push", "message": "Hello Niraj" }`. The "Sync" event is set to `sync`, and the "Periodic sync" event is set to `test-tag-from-devtools`. The "Update Cycle" table shows the status of the service worker:

Version	Update Activity	Timeline
#434	Install	
#434	Wait	
#434	Activate	

. The Console panel at the bottom shows the following log messages:

```
[Service Worker] Active
[Service Worker] Fetch http://localhost:5173/favicon.svg
[Service Worker] Found in cache: http://localhost:5173/favicon.svg
[Service Worker] Push event:
  PushEvent {isTrusted: true, data: PushMessageData, type: 'push', target: ServiceWorkerGlobalScope, currentTarget: ServiceWorkerGlobalScope, ...}
```

Conclusion: Successfully implemented Service worker events like fetch, sync and push for PWA.

MAD & PWA Lab

Journal

Experiment No.	10
Experiment Title.	To study and implement deployment of Ecommerce PWA to GitHub Pages.
Roll No.	24
Name	Niraj Kothawade
Class	D15A/D15B
Subject	MAD & PWA Lab
Lab Outcome	LO5: Design and Develop a responsive User Interface by applying PWA Design techniques
Grade:	

Experiment No. 10

MAD & PWA Lab

Niraj Kothawade
D15A - 24

Aim:

To study and implement deployment of Ecommerce PWA to GitHub Pages.

Theory:

GitHub Pages

Public web pages are freely hosted and easily published. Public webpages hosted directly from your GitHub repository. Just edit, push, and your changes are live.

GitHub Pages provides the following key features:

1. Blogging with Jekyll
2. Custom URL
3. Automatic Page Generator

Reasons for favoring this over Firebase:

1. Free to use
2. Right out of github
3. Quick to set up

GitHub Pages is used by Lyft, CircleCI, and HubSpot.

GitHub Pages is listed in 775 company stacks and 4401 developer stacks.

Pros

1. Very familiar interface if you are already using GitHub for your projects.
2. Easy to set up. Just push your static website to the gh-pages branch and your website is ready.
3. Supports Jekyll out of the box.
4. Supports custom domains. Just add a file called CNAME to the root of your site, add an A record in the site's DNS configuration, and you are done.

Cons

1. The code of your website will be public, unless you pay for a private repository.
2. Currently, there is no support for HTTPS for custom domains. It's probably coming soon though.

3. Although Jekyll is supported, plug-in support is rather spotty.

Firebase

The Realtime App Platform. Firebase is a cloud service designed to power real-time, collaborative applications. Simply add the Firebase library to your application to gain access to a shared data structure; any changes you make to that data are automatically synchronized with the Firebase cloud and with other clients within milliseconds.

Some of the features offered by Firebase are:

1. Add the Firebase library to your app and get access to a shared data structure. Any changes made to that data are automatically synchronized with the Firebase cloud and with other clients within milliseconds.
2. Firebase apps can be written entirely with client-side code, update in real-time out-of-the-box, interoperate well with existing services, scale automatically, and provide strong data security.
3. Data Accessibility- Data is stored as JSON in Firebase. Every piece of data has its own URL which can be used in Firebase's client libraries and as a REST endpoint. These URLs can also be entered into a browser to view the data and watch it update in real-time.

Reasons for favoring over GitHub Pages:

1. Realtime backend made easy
2. Fast and responsive

Instacart, 9GAG, and Twitch are some of the popular companies that use Firebase. Firebase has a broader approval, being mentioned in 1215 company stacks & 4651 developers stacks.

Pros

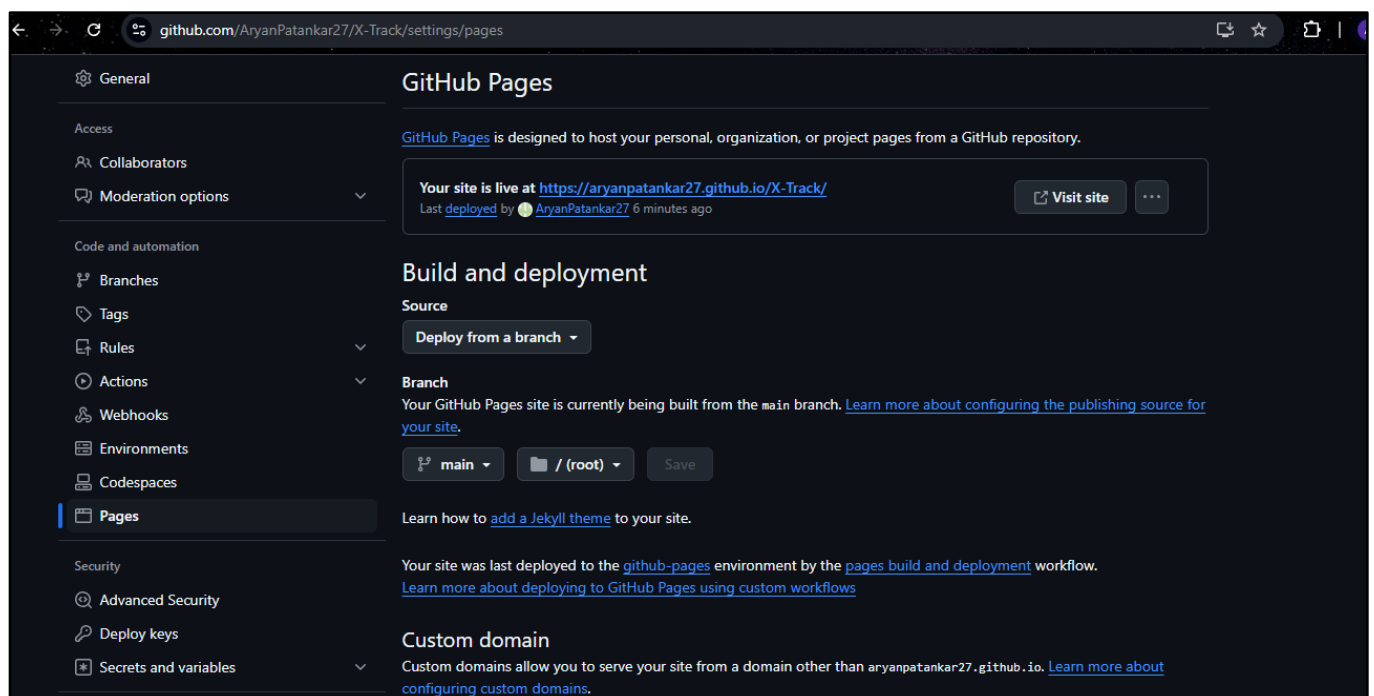
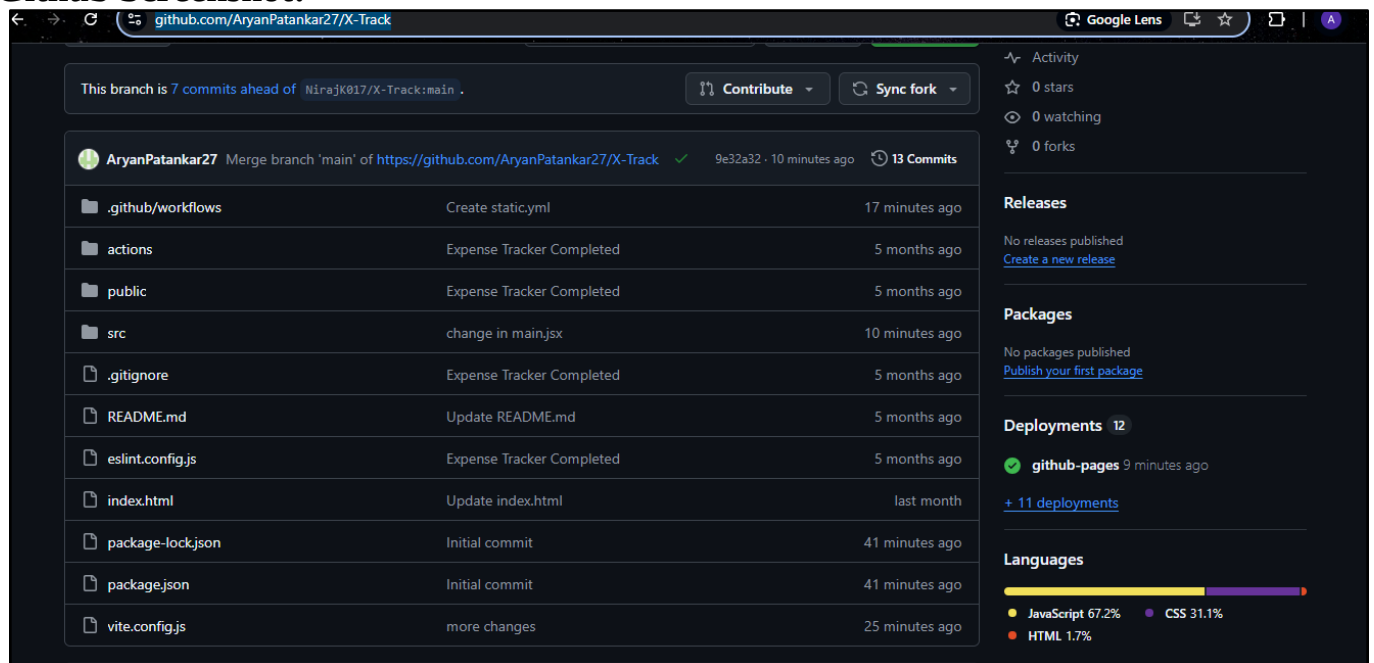
1. Hosted by Google. Enough said.
2. Authentication, Cloud Messaging, and a whole lot of other handy services will be available to you.
3. A real-time database will be available to you, which can store 1 GB of data.
4. You'll also have access to a blob store, which can store another 1 GB of data.
5. Support for HTTPS. A free certificate will be provisioned for your custom domain within 24 hours.

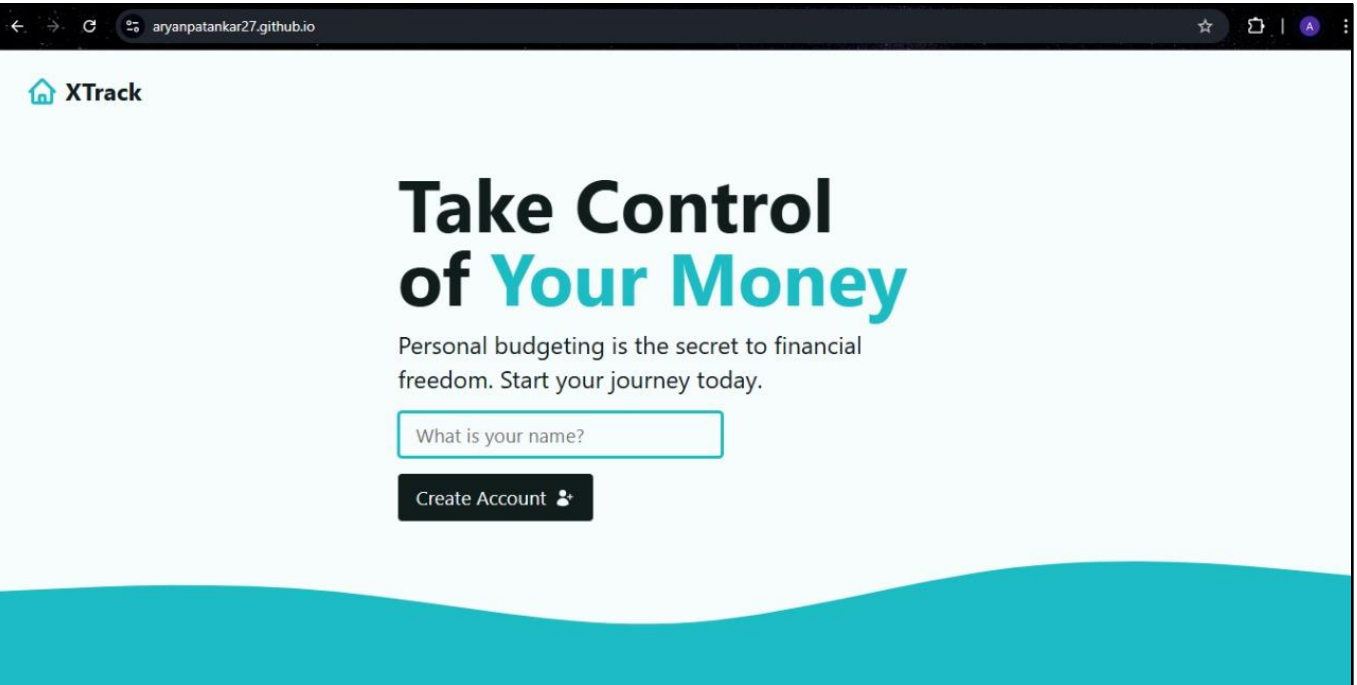
Cons

1. Only 10 GB of data transfer is allowed per month. But this is not really a big problem, if you use a CDN or AMP.
2. Command-line interface only.
3. No in-built support for any static site generator.

Link to our GitHub repository: <https://github.com/AryanPatankar27/X-Track>

Github Screenshot:





MAD & PWA Lab

Journal

Experiment No.	11
Experiment Title.	To use google Lighthouse PWA Analysis Tool to test the PWA functioning.
Roll No.	24
Name	Niraj Kothawade
Class	D15A/D15B
Subject	MAD & PWA Lab
Lab Outcome	LO6: Develop and Analyze PWA Features and deploy it over app hosting solution
Grade:	

Experiment 11

Niraj Kothawade

D15A - 24

Aim : To use google Lighthouse PWA Analysis Tool to test the PWA functioning.

Theory :

Google Lighthouse :

Google Lighthouse is a tool that lets you audit your web application based on a number of parameters including (but not limited to) performance, based on a number of metrics, mobile compatibility, Progressive Web App (PWA) implementations, etc. All you have to do is run it on a page or pass it a URL, sit back for a couple of minutes and get a very elaborate report, not much short of one that a professional auditor would have compiled in about a week. The best part is that you have to set up almost nothing to get started. Let's begin by looking at some of the top features and audit criteria used by Lighthouse.

Key Features and Audit Metrics

Google Lighthouse has the option of running the Audit for Desktop as well as mobile version of your page(s). The top metrics that will be measured in the Audit are:

Performance: This score is an aggregation of how the page fared in aspects such as (but not limited to) loading speed, time taken for loading for basic frame(s), displaying meaningful content to the user, etc. To a layman, this score is indicative of how decently the site performs, with a score of 100 meaning that you figure in the 98th percentile, 50 meaning that you figure in the 75th percentile and so on.

PWA Score (Mobile): Thanks to the rise of Service Workers, app manifests, etc., a lot of modern web applications are moving towards the PWA paradigm, where the objective is to make the application behave as close as possible to native mobile applications. Scoring points are based on the Baseline PWA checklist laid down by Google which includes Service Worker implementation(s), viewport handling, offline functionality, performance in script-disabled environments, etc.

Accessibility: As you might have guessed, this metric is a measure of how accessible your website is, across a plethora of accessibility features that can be implemented in your page (such as the 'aria-' attributes like aria-required, audio captions, button names, etc.). Unlike the other metrics though, Accessibility metrics score on a pass/fail basis i.e. if all possible elements of the page are not screen-reader friendly (HTML5 introduced features that would make pages easy to interpret for screen readers used by visually challenged people like tag names, tags such as <section>, <article>, etc.), you get a 0 on that score. The aggregate of these scores is your Accessibility metric score.

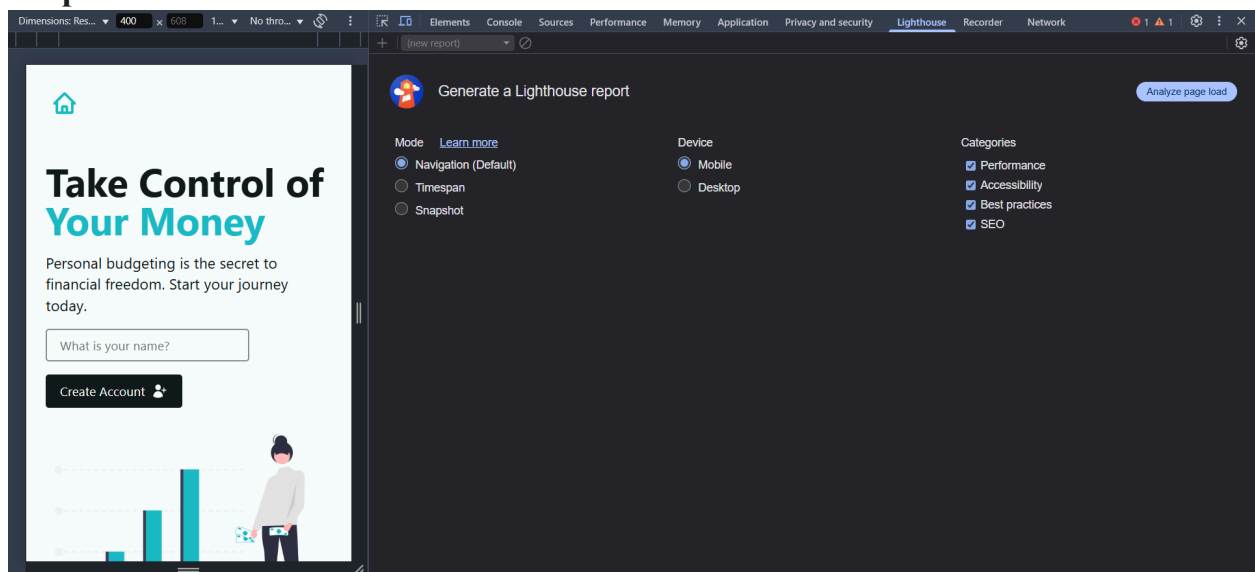
Best Practices: As any developer would know, there are a number of practices that have been deemed 'best' based on empirical data. This metric is an aggregation of many such points, including but not limited to: Use of HTTPS
Avoiding the use of deprecated code elements like tags, directives, libraries, etc. Password input with paste-into disabled
Geo-Location and cookie usage alerts on load, etc.

Improvement In Code:

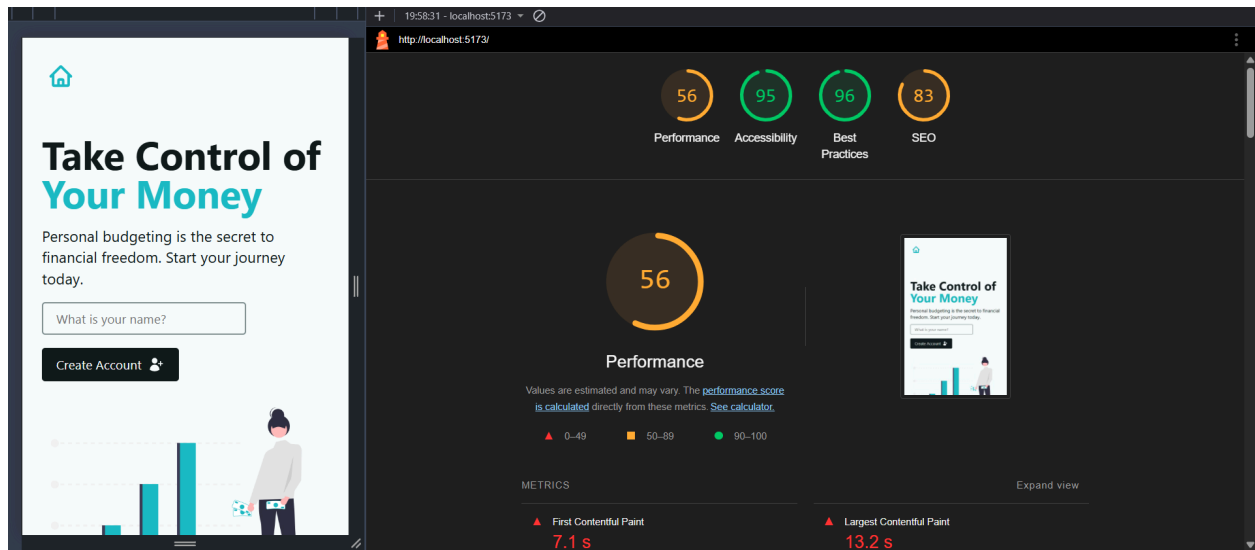
Index.html

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/favicon.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <meta name="description" content="A concise and accurate description of your Expense Tracker app. This will appear in search engine results.">
    <link rel="manifest" href="manifest.json" />
    <title>XTrack</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

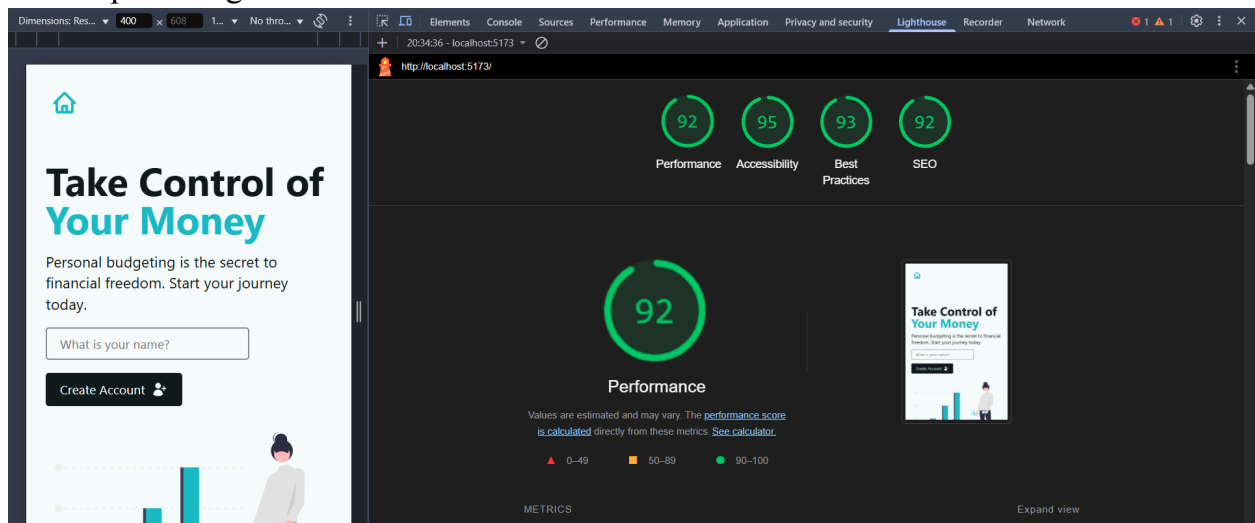
Output:



Before Optimizing Code:



After Optimizing Code:



Conclusion: Thus we successfully used google Lighthouse PWA Analysis Tool for testing the PWA functioning.

MAD & PWA Lab

Journal

Experiment No.	Assignment-1
Assignment 1 Questions	1. Flutter Overview: Explain the key features and advantages of using Flutter for mobile app development. Discuss how the Flutter framework differs from traditional approaches and why it has gained popularity in the developer community. 2. Widget Tree and Composition: Describe the concept of the widget tree in Flutter. Explain how widget composition is used to build complex user interfaces. Provide examples of commonly used widgets and their roles in creating a widget tree. 3. State Management in Flutter: Discuss the importance of state management in Flutter applications. Compare and contrast the different state management approaches available in Flutter, such as setState, Provider, and Riverpod. Provide scenarios where each approach is suitable. 4. Firebase Integration in Flutter: Explain the process of integrating Firebase with a Flutter application. Discuss the benefits of using Firebase as a backend solution. Highlight the Firebase services commonly used in Flutter development and provide a brief overview of how data synchronization is achieved.
Roll No.	24
Name	Niraj Kothawade
Class	D15A/D15B
Subject	MAD & PWA Lab
Lab Outcome	LO1: Understand cross platform mobile application development using Flutter framework LO2: Design and Develop interactive Flutter App by using widgets, layouts, gestures and animation LO3: Analyze and Build production ready Flutter App by incorporating backend services and deploying on Android / iOS
Grade:	

MAD & PWA Lab

Journal

Experiment No.	Assignment-2
Assignment 2 Questions	1. Define Progressive Web App (PWA) and explain its significance in modern web development. Discuss the key characteristics that differentiate PWAs from traditional mobile apps 2. Define responsive web design and explain its importance in the context of Progressive Web Apps. Compare and contrast responsive, fluid, and adaptive web design approaches. 3. Describe the lifecycle of Service Workers, including registration, installation, and activation phases. 4. Explain the use of IndexedDB in the Service Worker for data storage.
Roll No.	24
Name	Niraj Kothawade
Class	D15A/D15B
Subject	MAD & PWA Lab
Lab Outcome	LO4: Understand various PWA frameworks and their requirements LO5: Design and Develop a responsive User Interface by applying PWA Design techniques LO6: Develop and Analyze PWA Features and deploy it over app hosting solutions
Grade:	